GO

CHEAT SHEETS

A Complete Reference for Modern Go

Types & Strings · Control Flow · Errors as Values · Functions & Closures · Slices & Maps · Interfaces · Generics · Files & I/O · Modules · Concurrency · Testing & Tooling · net/http

112 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

001–004 Introduction & Setup
005–007 Output & Comments
008–015 Variables & Data Types
016–021 Strings
022–026 Operators & Pointers
027–032 Control Flow

CORE LANGUAGE

033–037 Error Handling
038–044 Functions
045–054 Data Structures
055–064 Types, Methods & Interfaces
065–071 Files & I/O
072–080 Packages, Modules & Standard Library

ECOSYSTEM & SYSTEMS

081–084 External Libraries
085–090 Advanced Topics
091–098 Concurrency
099–104 Testing & Tooling
105–110 Web & Networking
111–112 Quick Reference

GO

Table of Contents

112 sheets · syntax · examples · notes

INTRODUCTION & SETUP 001 – 004

001 Introduction & History
002 Installation & Toolcha…
003 The go Command
004 Program Structure

OUTPUT & COMMENTS 005 – 007

005 Printing with fmt
006 Format Verbs
007 Comments & Doc Co…

VARIABLES & DATA TYPES 008 – 015

008 Variables & Declarati…
009 Constants & iota
010 Integers
011 Floats & Complex Nu…
012 Booleans & Zero Valu…
013 Type Conversion
014 Type Definitions & Ali…
015 Bytes & Runes

STRINGS 016 – 021

016 String Basics
017 Strings & UTF-8
018 strings Package — Se…
019 strings Package — Tra…
020 strings.Builder & bytes
021 strconv

OPERATORS & POINTERS 022 – 026

022 Arithmetic Operators
023 Comparison & Logica…
024 Bitwise Operators
025 Assignment & Preced…
026 Pointers

CONTROL FLOW 027 – 032

027 if / else
028 switch
029 for Loops
030 for range
031 Labels, break, continu…
032 defer

ERROR HANDLING 033 – 037

033 Errors as Values
034 Wrapping & Inspectin…
035 Custom & Sentinel Er…
036 panic & recover
037 Error Handling Patter…

FUNCTIONS 038 – 044

038 Defining & Calling Fun…
039 Variadic Functions
040 Function Values & Ty…
041 Closures
042 Scope & Shadowing
043 Package Initialization
044 Functional Options & …

DATA STRUCTURES 045 – 054

045 Arrays
046 Slices — Basics
047 Slices — Internals & G…
048 slices Package
049 Maps — Basics
050 Maps — Sets & Patter…
051 Structs
052 Sorting & Searching
053 Stacks, Queues & He…
054 Iterators

TYPES, METHODS & INTERFACES 055 – 064

055 Methods & Receivers
056 Embedding & Compo…
057 Encapsulation & Con…
058 Interfaces
059 any, Type Assertions …
060 Interface Gotchas
061 Standard Library Inte…
062 Generics — Basics
063 Generics — Constrain…
064 Generic Methods & A…

FILES & I/O 065 – 071

065 io.Reader & io.Writer
066 Reading Files
067 Writing Files
068 Paths & the Filesystem
069 embed & io/fs
070 JSON
071 CSV, XML & Other En…

PACKAGES, MODULES & STANDARD LIBRARY 072 – 080

072 Packages & Imports
073 Modules & go.mod
074 Dependencies & Wor…
075 math & math/rand/v2
076 time
077 os, exec & Signals
078 Command-Line Flags
079 Logging: log & slog
080 Hashing, Crypto & UU…

EXTERNAL LIBRARIES 081 – 084

081 testify
082 Cobra & Viper (CLIs)
083 Web Routers: chi, Gin …
084 Database Libraries: p…

ADVANCED TOPICS 085 – 090

085 Regular Expressions
086 Reflection
087 database/sql
088 Build Tags & Cross-C…
089 Memory & Garbage C…
090 unsafe & cgo

CONCURRENCY 091 – 098

091 Goroutines
092 Channels
093 select
094 sync Primitives
095 sync/atomic
096 context
097 Concurrency Patterns
098 Races, Deadlocks & L…

TESTING & TOOLING 099 – 104

099 Testing — Basics
100 Testing — Helpers & F…
101 Benchmarks, Fuzzing …
102 Profiling & Tracing
103 Formatting, Vet & Lint…
104 Debugging with Delve

WEB & NETWORKING 105 – 110

105 HTTP Server — Basics
106 Handlers & Middlewa…
107 Building JSON APIs
108 HTTP Client
109 Production HTTP Serv…
110 TCP, UDP & net

QUICK REFERENCE 111 – 112

111 Quick Reference — S…
112 Quick Reference — C…

Sheet 001

Introduction & History

Google, 2009 · Griesemer / Pike / Thompson · Go 1 compatibility promise · 6-month release cycle · Go 1.27 (Aug 2026) · simplicity, composition, concurrency · go.dev / pkg.go.dev

WHAT IS GO?

Go is a compiled, statically typed language created at **Google** in 2007 and released as open source in 2009. It was designed to keep large codebases simple: a small spec, fast builds, one formatting style, and **built-in concurrency** with goroutines and channels. Go produces a single static binary, which is why it dominates cloud tooling (Docker, Kubernetes, Terraform) and network services.

KEY FACTS

- Designers: Robert Griesemer, Rob Pike and Ken Thompson
- **Go 1 compatibility promise** (2012): code written for Go 1.x keeps compiling on later 1.x releases
- New release every 6 months (Feb / Aug); this set targets **Go 1.27** (Aug 2026)
- Source files end in `.go`; docs at `go.dev`, packages at `pkg.go.dev`

HELLO, WORLD

```go
// hello.go
package main            // executables live in package main

import "fmt"            // stdlib formatting package

func main() {           // entry point: no args, no result
    fmt.Println("Hello, World!")   // Hello, World!
}

// $ go run hello.go       → compile + run in one step
// $ go build -o hello .   → static binary ./hello
// $ ./hello               → Hello, World!
```

CODE EXPLAINED

- → `package main` — marks a program (not a library) package
- → `import "fmt"` — imports are paths; unused imports are compile errors
- → `func main()` — runs when the binary starts; returning ends the program
- → `go run` — builds to a temp dir and runs; `go build` keeps the binary
- → Formatting is not a choice: run `gofmt` (tabs, brace on same line)

COMING FROM…

- **Python** — compiled to one binary; no interpreter
- **Java** — no JVM or classes; fast startup
- **C** — garbage collected, memory-safe, built-in concurrency

RELEASE TIMELINE

VERSION	HIGHLIGHT
Go 1.0 (2012)	stable language + compatibility promise
Go 1.11 (2018)	modules (`go.mod`)
Go 1.18 (2022)	generics, fuzzing, workspaces
Go 1.22 (2024)	per-iteration loop vars, `range` over int
Go 1.23 (2024)	range-over-func iterators
Go 1.27 (2026)	generic methods, `encoding/json/v2`

WHERE GO IS USED

AREA	EXAMPLES
Cloud / infra	Docker, Kubernetes, Terraform
Networking	Caddy, CoreDNS, Traefik
CLIs	`gh`, Hugo, `kubectl`
Databases	CockroachDB, etcd, InfluxDB
Observability	Prometheus, Grafana backend

⚠ COMMON MISTAKES

- Calling it "Golang" in code or imports; the language is **Go** (golang is only the domain/search term)
- Expecting classes, exceptions, or inheritance; Go uses structs, interfaces, and error values instead
- Fighting `gofmt`; brace placement is enforced by the grammar (automatic semicolons)

✓ TIPS & BEST PRACTICES

- Try snippets in the **Go Playground** (`go.dev/play`) without installing anything
- Work through **A Tour of Go** (`go.dev/tour`) and **Effective Go** first
- Read the release notes at `go.dev/doc/go1.N` when upgrading

QUICK SUMMARY

- `package main` + `func main()` — every program's entry point
- `go run` / `go build` — run now vs produce a binary
- Go 1 promise — old code keeps compiling
- Strengths — simplicity, fast builds, concurrency, static binaries

Installation & Toolchain

INSTALLING GO

Go ships as a single toolchain: the compiler, linker, formatter, test runner and module tool are all behind the `go` command. Install it from **go.dev/dl** or a package manager, then check `go version`. Since Go 1.21 the `go` command can also **download a newer toolchain on demand** when a module's `go.mod` asks for one, controlled by `GOTOOLCHAIN`.

ENVIRONMENT VARIABLES

- `GOROOT` — where the toolchain is installed (rarely set by hand)
- `GOPATH` — module cache + installed binaries; defaults to `~/go`
- `GOBIN` — where `go install` puts binaries (default `$GOPATH/bin`); add it to `PATH`
- `GOTOOLCHAIN` — `auto` (default), `local`, or a version like `go1.27.0`

INSTALL & VERIFY

```
# macOS
brew install go            # or the .pkg from go.dev/dl
# Linux (tarball)
sudo tar -C /usr/local -xzf go1.27.0.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin:$HOME/go/bin
# Windows
winget install GoLang.Go

go version                 # go1.27.0 darwin/arm64
go env GOPATH GOROOT       # /Users/me/go  /opt/…
go env -w GOBIN=$HOME/bin # persist a setting
go env -u GOBIN            # unset it again

# editor support (VS Code Go extension installs this)
go install golang.org/x/tools/gopls@latest
```

CODE EXPLAINED

→ `brew install go` — simplest on macOS; upgrades with `brew upgrade go`

→ `$HOME/go/bin` — must be on `PATH` to run tools you `go install`

→ `go env` — prints every setting; `-w` writes to the user config file

→ `gopls` — the official language server (completion, rename, diagnostics)

→ No virtualenvs needed: dependencies are per-module via `go.mod`

COMING FROM…

- **Python** — no venv; deps are per-module in `go.mod`
- **Node** — no `node_modules`; modules cache in `GOMODCACHE`
- **Java** — one toolchain binary, no JDK/Maven split

TOOLCHAIN SELECTION

SETTING	EFFECT
`go 1.27`	`go.mod` minimum version for the module
`toolchain go1.27.1`	`go.mod` preferred toolchain
`GOTOOLCHAIN=auto`	download newer toolchain if `go.mod` needs it
`GOTOOLCHAIN=local`	never download; fail if too old
`GOTOOLCHAIN=go1.26.3`	force a specific version

USEFUL GO ENV KEYS

KEY	TYPICAL VALUE
`GOOS / GOARCH`	`darwin` / `arm64`
`GOMODCACHE`	`~/go/pkg/mod`
`GOCACHE`	build cache dir
`GOPROXY`	`https://proxy.golang.org,direct`
`CGO_ENABLED`	1 (0 = pure-Go builds)

⚠ COMMON MISTAKES

- Forgetting to add `$(go env GOPATH)/bin` to `PATH`, so installed tools "aren't found"
- Setting `GOPATH` to the project folder (old pre-modules habit); with modules, code can live anywhere
- Installing two Go versions (brew + tarball) and running the wrong one; check `which go`

✓ TIPS & BEST PRACTICES

- Pin a team's toolchain with a `toolchain` line in `go.mod` instead of README instructions
- Use `go env -json` in scripts to read settings reliably
- Install extra versions side by side: `go install golang.org/dl/go1.26.3@latest`

QUICK SUMMARY

- `go version` — confirm the install
- `go env [-w]` — read or persist settings
- `GOPATH/bin` — where `go install` binaries land
- `GOTOOLCHAIN` — automatic toolchain switching

Sheet 003

The go Command

go run · go build -o · go install pkg@version · go fmt · go vet · go doc (incl. pkg@version, -ex) · go clean -cache · go help

ONE COMMAND FOR EVERYTHING

The go command builds, runs, tests, formats, documents and manages dependencies. Most subcommands take **package patterns**: `.` is the current package and `./...` is it plus every package below it. Builds are cached, so repeated `go build` and `go test` runs are fast.

PACKAGE PATTERNS

- `.` — package in the current directory
- `./...` — current package and all sub-packages
- `example.com/app/cmd/...` — every package under that import path
- `pkg@version` — a specific module version (`go install` , `go run` , `go doc`)

DAILY COMMANDS

```
go run .                    # build + run this package
go run ./cmd/srv -port 8080 # args after pkg → program
go build                    # binary named after dir
go build -o bin/app ./cmd/app  # choose output path
go install ./cmd/app        # build + copy to $GOBIN
go install golang.org/x/tools/cmd/stringer@latest

go fmt ./...                # gofmt every package
go vet ./...                # static checks
go test ./...               # run all tests

go doc strings.Cut          # docs for one symbol
go doc -all net/http | less # whole package
go doc rsc.io/quote@v1.5.2  # pkg@version (1.27)
go doc -ex sort.Slice       # with examples (1.27)

go clean -cache -testcache  # clear caches
go help build               # help on a subcommand
```

CODE EXPLAINED

- → `go run .` — runs a whole package, not just one file
- → `-o` — output file or directory for `go build`
- → `go install pkg@version` — install a tool without touching `go.mod`
- → `go vet` — also runs automatically (a subset) during `go test`
- → `go doc` — offline docs straight from source comments

COMING FROM…

- **npm / pip** — `go` does build, test, format, deps in one tool
- **Maven/Gradle** — no build file; packages are directories
- **cargo** — `go run` ≈ `cargo run` , `go vet` ≈ `cargo clippy` (lite)

BUILD & RUN

COMMAND	DOES
`go run`	compile to temp + execute
`go build`	compile; keep binary
`go install`	build into `$GOBIN`
`go generate`	run `//go:generate` lines
`go clean`	remove objects / caches

QUALITY

COMMAND	DOES
`go fmt`	reformat source
`go vet`	suspicious-code checks
`go test`	tests, benchmarks, fuzz
`go fix`	apply modernizers
`go tool cover`	coverage report

MODULES

COMMAND	DOES
`go mod init`	create `go.mod`
`go get`	add / upgrade a dependency
`go mod tidy`	sync requirements
`go list -m all`	show module graph
`go work`	multi-module workspace

⚠ COMMON MISTAKES

- `go run main.go` in a multi-file package fails with "undefined"; run the package with `go run .`
- Putting flags after the package for `go build` (`go build . -o x`); build flags go **before** the package
- Using `go get` to install tools; since 1.17 that's `go install tool@version`

✓ TIPS & BEST PRACTICES

- Add `-v` to see which packages build; `-x` prints every command run
- `go list -f '{{.Dir}}' ./...` scripts over packages with templates
- Use `go build -race` / `go test -race` while developing concurrent code

QUICK SUMMARY

- `go run .` / `go build` — run vs compile
- `./...` — this package and everything below
- `go fmt` + `go vet` — format and lint before committing
- `go doc` — documentation at the terminal

Program Structure

ANATOMY OF A GO PROGRAM

Every Go file starts with a **package clause**, then imports, then top-level declarations in any order. An executable needs `package main` with a `func main()`. Visibility is decided by the **first letter**: `Capitalized` names are exported from the package, `lowercase` ones are private to it. There are no header files and no forward declarations.

RULES THE COMPILER ENFORCES

- Unused **imports** and unused **local variables** are compile errors
- Semicolons are inserted automatically at line ends, so `{` must stay on the same line
- All files in one directory share one package name
- `os.Exit(n)` sets the exit code; returning from `main` exits with 0

A COMPLETE PROGRAM

```
package main                    // 1. package clause

import (                        // 2. grouped imports
    "fmt"
    "os"
    "strings"
)

const greeting = "hello"        // 3. package-level decls

func shout(s string) string {   // lowercase = unexported
    return strings.ToUpper(s) + "!"
}

func main() {                   // 4. entry point
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: greet NAME")
        os.Exit(2)              // skips deferred calls
    }
    fmt.Println(shout(greeting), os.Args[1])  // HELLO! Bob
}
```

CODE EXPLAINED

→ `import ( … )` — factored import block; gofmt sorts it

→ `const` / `var` at top level — visible to the whole package

→ `shout` — unexported; `Shout` would be callable from other packages

→ `os.Args[1]` — command-line args; `os.Args[0]` is the program name

→ `os.Exit` — immediate exit; use `return` from `main` when possible

COMING FROM…

- **Java** — no class wrapper; `main` is a plain function
- **Python** — braces, not indentation; unused imports fail the build
- **C** — no headers or forward declarations

GO SYNTAX AT A GLANCE

FEATURE	GO FORM
Statement end	newline (auto `;`)
Blocks	`{ }` always required
Conditions	no parentheses: `if x > 0 {`
Export	Capital first letter
Entry point	`func main()` in `package main`

TYPICAL PROJECT LAYOUT

PATH	PURPOSE
`go.mod`	module path + Go version
`main.go`	`package main` for a single binary
`cmd/app/main.go`	one dir per binary
`internal/…`	private packages

⚠ COMMON MISTAKES

- Putting `{` on the next line; the auto-inserted `;` makes it a syntax error
- Leaving debug imports or variables; delete them or use `_ = x` temporarily
- Calling `os.Exit` in a function that has `defer`s; the deferred cleanup never runs

✓ TIPS & BEST PRACTICES

- Let your editor run `goimports` on save: it formats **and** adds/removes imports
- Keep `main` thin: parse flags, then call `run()` `error` and exit based on its result
- Group related declarations with `const ( … )` and `var ( … )` blocks

QUICK SUMMARY

- `package main` + `func main()` — executable program
- Capitalized — exported; lowercase — package-private
- Unused imports/vars — compile errors
- `os.Exit(code)` — non-zero exit status

Printing with fmt

THE FMT PACKAGE

`fmt` handles formatted I/O. Functions come in families by **destination**: `Print*` writes to stdout, `Sprint*` returns a string, `Fprint*` writes to any `io.Writer`, and `Append*` appends to a byte slice. Within each family, the plain form joins operands, `…ln` adds spaces and a newline, and `…f` takes a format string.

SPACING RULES

- `Println` — always a space between operands + trailing newline
- `Print` — adds spaces only between operands when **neither** is a string
- `Printf` — no newline unless you write `\n`
- All return `(n int, err error)`; usually ignored for stdout

PRINT FAMILY

```
name, n := "Go", 3
fmt.Print("a", "b", 1, 2)           // ab1 2
fmt.Println("a", "b", 1, 2)         // a b 1 2
fmt.Printf("%s has %d\n", name, n)  // Go has 3

s := fmt.Sprintf("%05.1f", 3.14159) // "003.1"
msg := fmt.Sprint("x=", 42)         // "x=42"

fmt.Fprintln(os.Stderr, "warning!") // to stderr
fmt.Fprintf(w, "<h1>%s</h1>", name) // any io.Writer

buf := []byte("log: ")
buf = fmt.Appendf(buf, "%d items", n) // "log: 3 items"

err := fmt.Errorf("open %q: %w", "a.txt", os.ErrNotExist)
fmt.Println(err)  // open "a.txt": file does not exist

println("debug", n)                 // builtin → stderr
```

CODE EXPLAINED

- → `Sprintf` — build a string; the workhorse for messages
- → `Fprint*` — any `io.Writer`: files, buffers, HTTP responses
- → `Appendf` — append to `[]byte` for zero-extra-allocation logging
- → `Errorf` — formatted errors; `%w` wraps another error
- → `println` — builtin, unstable output format; never in real code

COMING FROM…

- **Python** — `print()` ≈ `fmt.Println`; f-strings ≈ `Sprintf`
- **C** — `printf` verbs, but `%v` prints anything
- **JS** — `console.log` ≈ `fmt.Println` (no objects auto-expanded)

FUNCTION MATRIX

DESTINATION	PLAIN	…LN	…F
stdout	Print	Println	Printf
string	Sprint	Sprintln	Sprintf
io.Writer	Fprint	Fprintln	Fprintf
[]byte	Append	Appendln	Appendf

FMT VS LOG

USE	CHOOSE
Program output	`fmt.Println`
Diagnostics + timestamp	`log.Println`
Structured logs	`log/slog`
Fatal + exit 1	`log.Fatal`
User-facing errors	`fmt.Fprintln(os.Stderr, …)`

⚠ COMMON MISTAKES

- `fmt.Println("x = %d", x)` prints the verb literally; formatting needs `Printf`
- Forgetting `\n` with `Printf`, so the next output lands on the same line
- Building strings in a loop with `Sprintf` + `+`; use `strings.Builder` (sheet 020)

✓ TIPS & BEST PRACTICES

- Run `go vet`: it checks `Printf` verbs against argument types
- Use `%v` when unsure and `%+v` on structs to see field names
- Write to `os.Stderr` for errors so piping stdout stays clean

QUICK SUMMARY

- `Print` / `Println` / `Printf` — stdout
- `Sprintf` — format to a string
- `Fprintf(w, …)` — format to any writer
- `Errorf("…: %w", err)` — wrap an error

Format Verbs

%v %+v %#v · %T · %d %b %o %x %X · %f %.2f %e %g · %s %q · %p · width & flags %-10s %05d %d · %%

FORMAT VERBS

Format strings use % verbs like C's `printf`, but Go adds **generic verbs**: %v prints any value in a default format, %+v adds struct field names, and %#v prints Go syntax. Width, precision and flags go between % and the verb. A wrong verb never crashes; it prints `%!d(string=hi)` instead, and `go vet` catches it.

FLAGS

- `-` left-align · `0` zero-pad · `+` always sign · ` ` (space) pad for sign
- `#` alternate form: `0x` for hex, Go syntax for `%#v`
- `*` width/precision from an argument: `%*d`
- `%[2]d` explicit argument index

VERBS IN ACTION

```
type P struct{ X, Y int }
p := P{1, 2}
fmt.Printf("%v", p)         // {1 2}
fmt.Printf("%+v", p)        // {X:1 Y:2}
fmt.Printf("%#v", p)        // main.P{X:1, Y:2}
fmt.Printf("%T", p)         // main.P
fmt.Printf("%d %b %o", 255, 5, 8)    // 255 101 10
fmt.Printf("%x %X %#x", 255, 255, 255) // ff FF 0xff
fmt.Printf("%.2f %e", 3.14159, 1234.5) // 3.14 1.234500e+03
fmt.Printf("%g %g", 0.000012, 100.0)   // 1.2e-05 100
fmt.Printf("%s %q %c", "go", "go", 'G') // go "go" G
fmt.Printf("[%6d][%-6d]", 42, 42)   // [    42][42    ]
fmt.Printf("[%06d][%+d]", 42, 42)   // [000042][+42]
fmt.Printf("[%8.3f]", 3.14159)      // [   3.142]
fmt.Printf("[%*d]", 5, 7)           // [    7]
fmt.Printf("%t %p", true, &p)       // true 0xc000012345
fmt.Printf("%d%%", 50)              // 50%
fmt.Printf("%d", "hi")              // %!d(string=hi)
```

CODE EXPLAINED

- → `%v` — default format for any type; maps print sorted by key
- → `%T` — the dynamic type, great for debugging `any` values
- → `%q` — quoted, escaped string; also works on runes
- → `%8.3f` — width 8, 3 decimals; `%-8s` left-aligns
- → `%!d(…)` — bad verb/arg pair reported inline, not a panic

COMING FROM…

- **C** — same `%d %s %x`, plus `%v`, `%T`, `%q`
- **Python** — `{:>8.2f}` ≈ `%8.2f`
- **Java** — `String.format` ≈ `fmt.Sprintf`

GENERAL

VERB	PRINTS
%v	default format
%+v	struct with field names
%#v	Go-syntax literal
%T	type
%t	bool
%p	pointer address

NUMBERS

VERB	PRINTS
%d	decimal
%b / %o / %O	binary / octal / 0o octal
%x / %X	hex lower / upper
%f / %.2f	decimal float / 2 places
%e / %g	scientific / compact
%c / %U	rune char / U+XXXX

STRINGS

VERB	PRINTS
%s	plain string / []byte
%q	double-quoted, escaped
%x	hex dump of bytes
%10s	right-aligned width 10
%-10s	left-aligned width 10
%.3s	first 3 bytes

⚠ COMMON MISTAKES

- Using `%s` on an `int`; you get `%!s(int=5)`, use `%d` or `%v`
- `%.3s` truncates **bytes**, which can split a multi-byte UTF-8 character
- Passing a pointer to `%v` and expecting the value; structs print as `&{…}`, other pointers as an address

✓ TIPS & BEST PRACTICES

- Implement `String() string` on a type and `%v` / `%s` will use it
- Use `%q` in error messages so empty strings and spaces are visible: `""`
- `%#v` is a quick way to dump a value as copy-pasteable Go code

QUICK SUMMARY

- `%v` / `%+v` / `%#v` — default / fields / Go syntax
- `%d %x %f %s %q` — the everyday verbs
- `%-8s %08d %.2f` — width, padding, precision
- `go vet` — catches verb/argument mismatches

Comments & Doc Comments

// line · / / block · doc comment on declarations · package comment & doc.go · [pkg.Name] doc links · Deprecated: · //go: directives · // Output: in examples

COMMENTS & DOCUMENTATION

Go has `//` line comments and `/* */` block comments. A comment placed **directly above** a top-level declaration (no blank line) is its **doc comment**: `go doc`, gopls hovers and pkg.go.dev all render it. Doc comments start with the name being documented and are full sentences. Special `//go:` lines (no space after `//`) are **directives** to the toolchain, not comments.

DOC COMMENT SYNTAX

- Start with the identifier: `// Parse reads …`
- `[strings.Cut]` and `[Config]` become links in rendered docs
- Indented lines render as code; `# Heading` lines become headings
- `// Deprecated: use X instead.` paragraph marks an API deprecated

COMMENTS IN A PACKAGE

```go
// Package shapes computes areas of simple geometric shapes.
//
// See [math.Pi] for the constant used by [Circle].
package shapes

// Circle is a circle with radius R.
type Circle struct{ R float64 }

// Area returns the area of c.
func (c Circle) Area() float64 {
    return math.Pi * c.R * c.R /* πr² */
}

// Deprecated: use [Circle.Area] instead.
func OldArea(c Circle) float64 { return c.Area() }

//go:generate stringer -type=Color  ← no space after //
//go:embed version.txt
var version string
```

CODE EXPLAINED

→ `// Package shapes …` — package comment, usually in `doc.go`

→ `[math.Pi]` — doc link to another package's symbol

→ `Deprecated:` — tools (gopls, staticcheck) warn on use

→ `//go:generate` — run by `go generate`; `//go:embed` read by the compiler

→ Block comments are rare; mostly for disabling code or license headers

COMING FROM…

- **Java** — no `/** @param */`; plain sentences starting with the name
- **Python** — doc comments go above, not inside like docstrings
- **C#** — no XML doc tags; `[Name]` makes links

COMMON DIRECTIVES

DIRECTIVE	PURPOSE
`//go:build linux`	build constraint (top of file)
`//go:generate cmd`	command for `go generate`
`//go:embed pattern`	embed files into a var
`//go:debug name=val`	set a GODEBUG default
`//nolint:rule`	linter suppression (golangci-lint)

WHERE DOCS SHOW UP

TOOL	HOW
`go doc pkg.Name`	terminal output
`gopls`	hover text in editors
`pkg.go.dev`	public HTML docs
`Example funcs`	`// Output:` verified by tests

⚠ COMMON MISTAKES

- A blank line between the comment and the declaration; it's no longer the doc comment
- Writing `// go:generate` with a space; it becomes an ordinary comment and silently does nothing
- Doc comments that restate the signature (`// GetName gets the name`) instead of explaining behavior

✓ TIPS & BEST PRACTICES

- Put long package docs in a dedicated `doc.go` file
- Use `Example` test functions with `// Output:` for docs that are checked by `go test`
- Run `go doc -all .` to preview how your package documentation reads

QUICK SUMMARY

- `//` — everyday comments; `/* */` — rare
- Comment right above a decl — its doc comment
- `[Name]` links, `Deprecated:` notices
- `//go:` — compiler/tool directives

Variables & Declarations

var x int · var x = 1 · x := 1 · multiple a, b := 1, 2 · var (…) blocks · zero values · _ blank identifier · package vs local vars

DECLARING VARIABLES

Go has two declaration forms. `var name T = value` works everywhere and lets you omit either the type or the value. The **short form** `name := value` infers the type and only works **inside functions**. Every variable is initialized: if you don't give a value it gets its type's **zero value**, so there is no such thing as an uninitialized variable.

RULES

- `:=` is function-scope only; package level needs `var`
- `:=` declares; `=` assigns to an existing variable
- Unused **local** variables are compile errors (package vars are fine)
- Names are `camelCase`; exported names start with a capital

ALL DECLARATION FORMS

```
var count int               // 0 (zero value)
var name = "Gopher"         // type inferred: string
var ratio float64 = 1.5     // explicit type + value

var (                       // grouped block
    host = "localhost"
    port = 8080
    debug bool              // false
)

func main() {
    n := 42                 // short declaration, int
    a, b := 1, "two"        // multiple, mixed types
    x, y := 10, 20
    x, y = y, x             // swap: 20 10
    _, err := os.Open("f") // _ discards a value
    n, z := 7, 3            // ok: z is new, n is reassigned
    fmt.Println(n, a, b, z, err != nil)
}
```

CODE EXPLAINED

→ `var x T` — zero value; `var x = v` — type inferred

→ `var ( … )` — groups related declarations

→ `:=` — needs at least one **new** name on the left

→ `_` — blank identifier; value is discarded

→ `x, y = y, x` — right side evaluated first, so swaps work

COMING FROM…

- **JS** — `:=` ≈ `let x =`; `var` is not hoisted or function-scoped
- **Python** — must declare before use; types fixed after
- **Java** — type comes after the name: `var n int`

WHICH FORM?

SITUATION	USE
Package level	`var x = …`
Local with value	`x := …`
Local, zero value	`var x T`
Specific type	`var x int64 = 5`
Ignore a result	`_ = f()`

NAMING CONVENTIONS

STYLE	EXAMPLE
camelCase local	`userCount`
Exported	`MaxRetries`
Acronyms stay upper	`userID`, `HTTPClient`
Short in small scopes	`i`, `n`, `err`
No underscores	not `user_count`

⚠ COMMON MISTAKES

- Using `:=` at package level: `syntax error: non-declaration statement outside function body`
- `x := 1` inside an `if` block **shadows** an outer `x` instead of assigning it
- `a, b := f()` when both already exist: "no new variables on left side of :="

✓ TIPS & BEST PRACTICES

- Prefer `:=` inside functions; use `var x T` when the zero value is the point
- Keep variable scope small: declare right before first use
- Use `go vet -vettool` shadow analyzer or golangci-lint `govet shadow` to catch shadowing

QUICK SUMMARY

- `var x T` / `var x = v` — long form, anywhere
- `x := v` — short form, functions only
- zero values — every variable starts initialized
- `_` — discard unwanted values

Constants & iota

CONSTANTS

`const` declares values fixed at **compile time**: numbers, strings, booleans and runes only. Untyped constants have arbitrary precision and get a type only when used. `iota` is a counter that restarts at 0 in every `const ( … )` block and increments per line, the standard way to build enums.

CONSTANT FACTS

- Untyped constants convert implicitly: `time.Sleep(2 * time.Second)` works
- Typed constants (`const n int = 5`) follow normal type rules
- A line with no expression repeats the previous expression with the next `iota`

IOTA ENUMS & FLAGS

```
const big = 1 << 100        // OK: untyped, never stored
const MaxUsers int = 500    // typed constant

type Weekday int
const (
    Sunday Weekday = iota       // 0
    Monday                      // 1 (repeats the expr)
    Tuesday                     // 2
)

type Perm uint8
const (
    Read  Perm = 1 << iota      // 1
    Write                       // 2
    Exec                        // 4
)
p := Read | Write               // 3; p&Exec == 0

const (
    _  = iota                   // skip 0
    KB = 1 << (10 * iota)       // 1024, then MB = 1048576
)
```

CODE EXPLAINED

→ `Weekday = iota` — defined type keeps enum values distinct from plain ints

→ Empty lines in the block repeat the last expression

→ `1 << iota` — powers of two for bit flags

→ `_ = iota` — skip a value (often to reserve 0 as "unset")

→ `big` overflows `int64` but compiles because it stays untyped

COMING FROM…

- **Java / C#** — no `enum` keyword; `type` + `iota` instead
- **JS** — `const` is compile-time only, not a frozen variable
- **C** — `iota` replaces manual enum numbering

IOTA PATTERNS

EXPRESSION	VALUES
`iota`	0, 1, 2, 3
`1 << iota`	1, 2, 4, 8
`_ = iota`	skips 0
`1 << (10 * iota)`	1, 1024, 1048576

ALLOWED IN CONST

KIND	EXAMPLE
`integer`	`const N = 10`
`float`	`const E = 2.718`
`string`	`const Name = "go"`
`rune`	`const Tab = '\t'`
`bool`	`const Debug = false`

⚠ COMMON MISTAKES

- Expecting `iota` to continue across `const` blocks; it restarts at 0 in each block
- `const t = time.Now()`: function calls aren't constant (only builtins like `len` on constants)
- Reordering enum lines renumbers them; stored values change meaning

✓ TIPS & BEST PRACTICES

- Generate `String()` methods with `//go:generate stringer -type=Weekday`
- Reserve `0` as an "unknown" value so the zero value is detectable

QUICK SUMMARY

- `const` — compile-time numbers, strings, bools, runes
- untyped constants — arbitrary precision, adapt on use
- `iota` — 0, 1, 2… per `const` block
- `type X int` + `iota` — Go's enum idiom

Integers

INTEGER TYPES

Go has sized signed (`int8`…`int64`) and unsigned (`uint8`…`uint64`) integers plus `int` and `uint`, which are **64 bits on 64-bit platforms**. Integer overflow is well defined: values **wrap around** silently. Different integer types never mix in one expression without an explicit conversion, even `int` and `int64`.

INTEGER NOTES

- `int` is the default for counts, indexes and `len()` results
- `byte` = `uint8`, `rune` = `int32` (aliases, see sheet 015)
- `uintptr` holds a pointer as an integer (unsafe code only)
- Integer division by a constant zero is a compile error; by a variable, a runtime panic

LITERALS, LIMITS & OVERFLOW

```
dec := 1_000_000          // underscores for readability
hex := 0xFF               // 255
oct := 0o755              // 493 (0755 also works)
bin := 0b1010             // 10

var small int8 = 127
small++                   // -128: wraps around
var u uint8 = 0
u--                       // 255

fmt.Println(math.MaxInt64)  // 9223372036854775807
fmt.Println(math.MinInt64)  // -9223372036854775808
fmt.Println(math.MaxUint32) // 4294967295
fmt.Println(strconv.IntSize) // 64 (bits in int)

var a int32 = 5
var b int64 = 10
// c := a + b             // error: mismatched types
c := int64(a) + b         // 15
```

CODE EXPLAINED

- → `1_000_000` — digit separators, also in hex/binary
- → `0o755` — explicit octal prefix (clearer than `0755`)
- → `small++` past 127 — wraps to -128, no panic
- → `math.MaxInt64` etc. — limits as untyped constants
- → Mixed types need conversion: `int64(a) + b`

COMING FROM…

- **Python** — ints are fixed-size and wrap; no bigints by default
- **Java** — unsigned types exist; `int` is 64-bit on 64-bit
- **JS** — real integers, not floats

SIGNED

TYPE	RANGE
`int8`	-128 … 127
`int16`	-32,768 … 32,767
`int32 / rune`	±2.1 × 10^9
`int64`	±9.2 × 10^{18}
`int`	same as int64 on 64-bit

UNSIGNED

TYPE	RANGE
`uint8 / byte`	0 … 255
`uint16`	0 … 65,535
`uint32`	0 … 4.29 × 10^9
`uint64`	0 … 1.8 × 10^{19}
`uint / uintptr`	platform size

CHOOSING

NEED	USE
`General counting`	`int`
`Raw bytes`	`byte`
`Unicode code point`	`rune`
`Fixed-size format`	`int32`, `uint64`
`Huge numbers`	`math/big.Int`

⚠ COMMON MISTAKES

- Using `uint` for sizes to "prevent negatives"; `len(s) - 1` on an empty slice then wraps to a huge number
- Assuming overflow panics; it silently wraps, so validate before arithmetic on untrusted input
- Writing `0755` and reading it as decimal 755; prefer `0o755`

✓ TIPS & BEST PRACTICES

- Default to `int`; pick sized types only for binary formats, protocols or memory layout
- Use `math/bits.Add64` / `Mul64` when you need to detect overflow (carry out)
- Use `math/big` for arbitrary-precision integers (crypto, factorials)

QUICK SUMMARY

- `int` — 64-bit default integer
- `int8…int64`, `uint8…uint64` — sized types
- overflow — wraps, no error
- mixed types — convert explicitly

Floats & Complex Numbers

FLOATING-POINT & COMPLEX

`float64` is the default floating-point type (IEEE-754 double precision); `float32` is for memory-sensitive data like graphics. Floats can't represent most decimals exactly, so **never compare floats with ==** after arithmetic. Dividing a float by zero gives `+Inf` or `NaN` rather than panicking. Go also has built-in `complex64` / `complex128` numbers.

FLOAT FACTS

- Untyped float literals default to `float64`: `x := 1.5`
- `NaN != NaN` is true; test with `math.IsNaN(x)`
- Float → int conversion **truncates** toward zero
- Use integers (cents) or `math/big` for money

FLOATS IN PRACTICE

```
x := 0.1 + 0.2
fmt.Println(x == 0.3)            // false
fmt.Println(x)                   // 0.30000000000000004
eq := math.Abs(x-0.3) < 1e-9     // true: use epsilon

fmt.Println(math.Round(2.5))     // 3  (half away from zero)
fmt.Println(math.Floor(-2.5))    // -3
fmt.Println(math.Trunc(-2.7))    // -2
fmt.Println(math.Ceil(2.1))      // 3
fmt.Printf("%.2f\n", 3.14159)    // 3.14

zero := 0.0
fmt.Println(1/zero, -1/zero)     // +Inf -Inf
nan := math.NaN()
fmt.Println(nan == nan, math.IsNaN(nan)) // false true
fmt.Println(math.MaxFloat64)     // 1.7976931348623157e+308

c := complex(3, 4)               // (3+4i)
fmt.Println(real(c), imag(c), cmplx.Abs(c)) // 3 4 5
```

CODE EXPLAINED

- → `0.1 + 0.2` — binary floats can't store 0.1 exactly
- → `math.Abs(a-b) < eps` — the correct equality test
- → `math.Round` — rounds half away from zero; `RoundToEven` also exists
- → `1/zero` — `+Inf`; only integer division by zero panics
- → `complex(re, im)` — built-in complex type; `math/cmplx` for functions

COMING FROM…

- **JS / Python** — same IEEE floats: `0.1 + 0.2 != 0.3`
- **Java** — no `BigDecimal` in stdlib; use `big.Float` / ints
- **C** — `math.NaN()` instead of `NAN` macro

TYPES

TYPE	PRECISION
`float32`	~7 decimal digits
`float64`	~15-16 digits (default)
`complex64`	two float32s
`complex128`	two float64s
`big.Float`	arbitrary precision
`big.Rat`	exact fractions

MATH HELPERS

FUNCTION	RESULT
`math.Inf(1)`	+Inf
`math.IsInf(x, 0)`	either infinity?
`math.Mod(7, 3)`	1 (float remainder)
`math.Pow(2, 10)`	1024
`math.Sqrt(2)`	1.4142135623730951
`math.Hypot(3, 4)`	5

⚠ COMMON MISTAKES

- `if total == 0.3` after arithmetic; it's almost never exactly equal
- Storing money as `float64`; rounding errors accumulate. Use `int64` cents
- `int(2.9)` expecting 3; conversion truncates to 2, use `int(math.Round(x))`

✓ TIPS & BEST PRACTICES

- Format output with `%.2f` or `strconv.FormatFloat(x, 'f', 2, 64)` instead of rounding the value
- Use `big.Rat` when you need exact rational arithmetic
- Prefer `float64` unless you have millions of values and measured memory pressure

QUICK SUMMARY

- `float64` — default float type
- `==` on floats — use an epsilon instead
- `math.Round/Floor/Ceil/Trunc` — rounding
- `+Inf`, `NaN` — results of float division by zero

Booleans & Zero Values

bool / true / false · no truthiness (no int → bool) · zero-value table · nil for pointer/slice/map/chan/func/interface · "make the zero value useful" · var buf bytes.Buffer ready to use

BOOL AND THE ZERO VALUE

`bool` holds `true` or `false`, and Go has **no truthiness**: an `int`, string or pointer can't stand in for a condition. Every type has a **zero value** that a variable gets when declared without an initializer. Good Go types are designed so their zero value is immediately useful, like `sync.Mutex`, `bytes.Buffer` and `strings.Builder`.

NOTES

- Conditions must be `bool`: write `if n != 0`, not `if n`
- No `bool` ↔ `int` conversion; use `if b { n = 1 }`
- `nil` is the zero value for pointers, slices, maps, channels, funcs, interfaces
- Reading a nil map or ranging over a nil slice is fine; writing to a nil map panics

ZERO VALUES IN ACTION

```
var (
    i  int            // 0
    f  float64        // 0
    s  string         // "" (empty, not nil)
    b  bool           // false
    p  *int           // nil
    sl []int          // nil, len 0
    m  map[string]int // nil
    e  error          // nil
)
fmt.Println(len(sl), sl == nil)  // 0 true
sl = append(sl, 1)               // append to nil slice: OK
fmt.Println(m["x"])              // 0: reading nil map OK
// m["x"] = 1                    // panic: nil map write

var buf bytes.Buffer             // ready to use
buf.WriteString("hi")
var mu sync.Mutex                // unlocked, ready to use
mu.Lock(); mu.Unlock()

ok := len(s) == 0 && !b          // true
```

CODE EXPLAINED

→ Every declared variable has a defined value, no garbage memory

→ `string` zero value is `""`, never `nil`

→ `append` on a nil slice allocates as needed

→ Nil map reads return the element zero value; writes panic

→ `bytes.Buffer`, `sync.Mutex` work without constructors

COMING FROM…

- **JS / Python** — no truthy/falsy; write `n != 0`, `s != ""`
- **Java** — no `null` for strings/ints; zero values instead
- **C** — every variable starts initialized

ZERO VALUES

TYPE	ZERO VALUE
`int, float, rune`	`0`
`bool`	`false`
`string`	`""`
`pointer, func`	`nil`
`slice, map, chan`	`nil`
`struct`	all fields zero

LOGICAL OPERATORS

OP	MEANING
`&&`	AND (short-circuit)
`||`	OR (short-circuit)
`!`	NOT
`== !=`	compare bools
`strconv.ParseBool`	"true", "1", "t" → true

⚠ COMMON MISTAKES

- `if count { … }` from C/Python habits: "non-boolean condition in if statement"
- Writing to a map declared with `var m map[K]V`; initialize with `make` or a literal first
- Checking `s == nil` for a string; strings can't be nil, check `s == ""`

✓ TIPS & BEST PRACTICES

- Design structs so the zero value works (no required `Init()` call)
- Return `nil` slices for "no results"; `len()` and `range` handle them
- Name booleans as questions: `isReady`, `hasNext`, `ok`

QUICK SUMMARY

- `bool` — `true` / `false`, no truthiness
- zero values — `0`, `false`, `""`, `nil`
- nil map — read OK, write panics
- useful zero value — `bytes.Buffer`, `sync.Mutex`

Type Conversion

T(v) explicit only · int ↔ float truncation · narrowing overflow · string(rune) vs strconv.Itoa pitfall · []byte(s) / string(b) copy · []rune(s) · slice → array [4]int(s)

EXPLICIT CONVERSIONS ONLY

Go never converts between types implicitly. The conversion syntax is `T(v)`, and it works between numeric types, between strings and byte/rune slices, and between types with the same underlying type. Numeric conversions can **lose data silently** (truncation, wraparound). For string ↔ number conversion you need the `strconv` package (sheet 021), not `T(v)`.

CONVERSION NOTES

- Untyped constants convert automatically when they fit: `var f float64 = 1`
- `[]byte(s)` and `string(b)` **copy** the data
- `string(65)` gives `"A"`, not `"65"` (`go vet` warns)
- Interface → concrete type uses an **assertion** `x.(T)`, not a conversion (sheet 059)

CONVERSION EXAMPLES

```
i := 42
f := float64(i)          // 42.0
n := int(3.99)           // 3: truncates toward zero
neg := int(-3.99)        // -3
// uint8(300)           // compile error: constant overflow
v := 300
b := uint8(v)            // 44: wraps at runtime (300 - 256)

s := strconv.Itoa(65)    // "65"
r := string(rune(65))    // "A"
bs := []byte("héllo")    // [104 195 169 108 108 111]
rs := []rune("héllo")    // [104 233 108 108 111]
back := string(bs)       // "héllo"

arr := [3]int(sl[:3])    // slice → array (1.20)
type Celsius float64
c := Celsius(f)          // same underlying type: OK
```

CODE EXPLAINED

- → `float64(i)` — explicit widening
- → `int(3.99)` — truncation, not rounding
- → `uint8(v)` — keeps the low 8 bits at runtime
- → `string(rune(65))` — code point → character
- → `[3]int(slice)` — converts a slice to an array copy (Go 1.20)

COMING FROM…

- **JS** — no implicit coercion: `"1" + 1` won't compile
- **Python** — `int("5")` is `strconv.Atoi`, not `int()`
- **Java** — no auto-widening between `int` and `int64`

COMMON CONVERSIONS

FROM → TO	HOW
`int → float64`	`float64(n)`
`float64 → int`	`int(math.Round(x))`
`int → string`	`strconv.Itoa(n)`
`string → int`	`strconv.Atoi(s)`
`string ↔ []byte`	`[]byte(s)` / `string(b)`
`string ↔ []rune`	`[]rune(s)` / `string(r)`

LOSSY CASES

EXPRESSION	RESULT
`int(2.9)`	2
`int8(200) (var)`	-56
`uint(-1) (var)`	18446744073709551615
`float32(0.1)`	0.1 (less precise)
`int(1e20) (var)`	implementation-specific

⚠ COMMON MISTAKES

- `string(n)` to print a number; it produces the Unicode character for `n`
- Converting a large `int64` to `int32` without a range check; it wraps silently
- Expecting `int(x)` to round; add `math.Round` first

✓ TIPS & BEST PRACTICES

- Check ranges before narrowing: `if v < 0 || v > math.MaxUint8 { … }`
- Use `strconv` for any string ↔ number work; it reports errors
- Define named types (`type UserID int64`) so mixups need an explicit conversion

QUICK SUMMARY

- `T(v)` — the only conversion syntax
- float → int — truncates
- narrowing — wraps silently
- `strconv` — string ↔ number

Sheet 014

Type Definitions & Aliases

type Celsius float64 · underlying types · named vs unnamed types · type A = B alias · generic aliases (1.24) · assignability rules · methods only on defined types · time.Duration example

DEFINED TYPES & ALIASES

`type Name Underlying` creates a **new, distinct type** that shares the underlying representation but not its methods. You can attach methods to it, and the compiler stops you mixing it with other types. `type A = B` creates an **alias**: another name for the same type, mainly for refactoring.

KEY RULES

- Methods can only be declared on **defined types in your own package**
- Defined types convert to/from their underlying type with `T(v)`
- Untyped constants assign directly: `var d Celsius = 100`
- Generic aliases `type Set[T comparable] = map[T]bool` work since **Go 1.24**

DEFINED TYPE VS ALIAS

```
type Celsius float64              // defined type
type Fahrenheit float64

func (c Celsius) ToF() Fahrenheit {
    return Fahrenheit(c*9/5 + 32)
}

var t Celsius = 100               // untyped constant: OK
fmt.Println(t.ToF())              // 212
var f Fahrenheit = 50
// t = f                          // error: mismatched types
t = Celsius((f - 32) * 5 / 9)     // explicit conversion

type MyInt = int                  // alias: identical to int
var b int = MyInt(5)              // OK: same type

type UserID int64                 // domain type
func Load(id UserID) {}  // Load(OrderID(7)) fails

d := 90 * time.Second             // time.Duration is int64
```

CODE EXPLAINED

→ `type Celsius float64` — new type; gets its own method set

→ Assigning `Fahrenheit` to `Celsius` needs a conversion

→ `type MyInt = int` — alias; no new methods possible

→ `UserID` / `OrderID` — same layout, can't be swapped by accident

→ `time.Duration` — a defined `int64` with methods like `.Seconds()`

COMING FROM…

- **Java** — no `extends`; `type T U` copies layout, not methods
- **TS** — `type A = B` alias ≈ TS type alias
- **C** — `typedef` ≈ alias; defined types are stricter

DEFINED VS ALIAS

ASPECT	DEFINED / ALIAS
Syntax	`type T U` / `type T = U`
New type?	yes / no
Own methods	yes / no
Assign from U	needs conversion / direct

STDLIB DEFINED TYPES

TYPE	UNDERLYING
`time.Duration`	int64
`os.FileMode`	uint32
`http.HandlerFunc`	func(w, r)
`net.IP`	[]byte

⚠ COMMON MISTAKES

- Expecting a defined type to inherit methods: `type MyTime time.Time` has none of `time.Time`'s methods
- Adding methods to a type from another package (`func (s string) X()`); define your own type
- Using aliases everywhere for "readability"; they add no type safety

✓ TIPS & BEST PRACTICES

- Embed the original type in a struct if you need its methods plus your own
- Use defined types for IDs, units and states to catch mixups at compile time
- Reserve aliases for moving a type between packages without breaking callers

QUICK SUMMARY

- `type T U` — new type, can have methods
- `type T = U` — alias, same type
- conversion — `T(v)` between same underlying types
- domain types — compile-time safety for IDs/units

Bytes & Runes

byte = uint8 · rune = int32 · rune literals 'a' '\n' 'é' · unicode.IsLetter / IsDigit / ToUpper · utf8.RuneLen · utf8.RuneError · Unicode 17 (1.27)

BYTE AND RUNE

A `byte` is an alias for `uint8` and represents one byte of raw data. A `rune` is an alias for `int32` and represents one **Unicode code point**. Go strings are UTF-8, so one character can be 1-4 bytes. Rune literals use single quotes, `'a'`, and are just integers. The `unicode` and `unicode/utf8` packages classify and encode runes.

NOTES

- `'a'` is a rune (int32 97); `"a"` is a string
- `utf8.RuneError` (`U+FFFD`) marks invalid UTF-8
- `unicode` tables track **Unicode 17** as of Go 1.27
- Arithmetic works: `'a' + 1 == 'b'`

WORKING WITH RUNES

```
var b byte = 'A'                // 65
r := 'é'                        // rune 233 (U+00E9)
fmt.Printf("%c %d %U\n", r, r, r) // é 233 U+00E9
fmt.Println(string(r + 1))      // ê

fmt.Println(utf8.RuneLen('a'))  // 1
fmt.Println(utf8.RuneLen('é'))  // 2
fmt.Println(utf8.RuneLen('世')) // 3
fmt.Println(utf8.RuneLen('🚀')) // 4

unicode.IsLetter('x')           // true
unicode.IsDigit('7')            // true
unicode.IsSpace('\t')           // true
unicode.IsUpper('Q')            // true
unicode.ToUpper('ß')            // 7838 'ß'

digit := '7' - '0'              // 7 (int32)
esc := []rune{'\n', '\'', '\x41', '\u00e9'}
```

CODE EXPLAINED

- → `%c` / `%U` — print a rune as a character / code point
- → `utf8.RuneLen` — bytes needed to encode a rune
- → `unicode.Is…` — Unicode-aware classification
- → `'7' - '0'` — classic digit-to-number trick
- → `'\x41'`, `'\u00e9'` — hex and Unicode escapes

COMING FROM…

- **Java / C#** — `char` is 16-bit; Go `rune` is a full code point
- **Python** — no char type; `'a'` here is an int
- **C** — `byte` ≈ `unsigned char`

RUNE ESCAPES

LITERAL	MEANING
`'\n' '\t' '\r'`	newline, tab, CR
`'\\' '\''`	backslash, quote
`'\x7f'`	hex byte value
`'\u00e9'`	4-hex code point
`'\U0001F680'`	8-hex code point

UNICODE FUNCTIONS

FUNC	TRUE FOR
`IsLetter`	letters in any script
`IsDigit / IsNumber`	decimal / any numeric
`IsSpace`	space, tab, newline…
`IsPunct`	punctuation
`ToLower / ToTitle`	case mapping

⚠ COMMON MISTAKES

- Treating `len("é")` as 1; it's 2 bytes. Count runes with `utf8.RuneCountInString`
- Using `byte` to hold characters from user input; non-ASCII text breaks
- `string(b)` on a single `byte` above 127 gives a Unicode char, not the raw byte

✓ TIPS & BEST PRACTICES

- Iterate with `for _, r := range s` to get runes, not bytes
- Use `unicode.SimpleFold` or `strings.EqualFold` for case-insensitive comparisons
- Use `golang.org/x/text` for normalization (é vs e + ◌́)

QUICK SUMMARY

- `byte` — `uint8`, raw data
- `rune` — `int32`, one code point
- `'x'` — rune literal; `"x"` — string
- `unicode` / `utf8` — classify and encode

Sheet 016

String Basics

"interpreted" vs raw · immutable byte sequences · len(s) counts bytes · s[i] returns a byte · s[a:b] slicing · + / += concat · == / < comparison · escape sequences

STRINGS ARE BYTE SLICES

A Go `string` is an **immutable sequence of bytes**, conventionally UTF-8. `len(s)` counts bytes, `s[i]` returns a `byte`, and slicing `s[a:b]` shares memory with the original. Strings can't be modified in place; every change builds a new string.

LITERAL FORMS

- `"interpreted"` — escapes like `\n`, `\t`, `\u00e9` work
- `` `raw` `` — backquotes, no escapes, can span lines
- Raw strings are ideal for regex, SQL, JSON and Windows paths
- Zero value is `""`; strings are never `nil`

STRING BASICS

```
s := "Hello, 世界"
fmt.Println(len(s))          // 13 (bytes, not characters)
fmt.Println(s[0])            // 72 (byte 'H')
fmt.Println(s[:5])           // Hello
fmt.Println(s[7:])           // 世界
// s[0] = 'h'                // error: immutable

t := s + "!"                 // concatenation → new string
t += "?"
fmt.Println(t)               // Hello, 世界!?

raw := `C:\temp\new
line two`                    // raw string spans lines
q := "tab:\t quote:\" uni:\u00e9"

fmt.Println("abc" < "abd")   // true (byte-wise compare)
fmt.Println("Go" == "go")    // false (case-sensitive)
```

CODE EXPLAINED

→ `len(s)` — 13 because `世` and `界` are 3 bytes each

→ `s[0]` — a `byte` (uint8), print with `%c` for the char

→ `s[7:]` — byte offsets; slicing mid-character gives garbage

→ `+` / `+=` — allocate a new string each time

→ `<`, `==` — lexicographic byte comparison

COMING FROM…

- **Python** — `len()` counts bytes, not characters
- **Java / JS** — strings are UTF-8 bytes, not UTF-16
- **C** — length is stored; no `\0` terminator

ESCAPE SEQUENCES

ESCAPE	MEANING
`\n \t \r`	newline, tab, CR
`\" \\`	quote, backslash
`\x41`	byte in hex
`\u00e9`	code point (4 hex)
`\U0001F600`	code point (8 hex)

OPERATIONS

EXPR	RESULT
`len("héllo")`	6
`"go" + "pher"`	"gopher"
`s[1:3]`	bytes 1..2
`s == ""`	empty check
`strings.Clone(s)`	independent copy

⚠ COMMON MISTAKES

- Using `len(s)` as a character count; non-ASCII text has more bytes than characters
- Indexing `s[i]` and printing it with `%v`; you get a number. Use `%c` or `string(s[i])` for ASCII
- Building a large string with `+=` in a loop; it's O(n²). Use `strings.Builder`

✓ TIPS & BEST PRACTICES

- Use raw strings for regex patterns so you don't double every backslash
- Check emptiness with `s == ""` or `len(s) == 0` (both idiomatic)
- A small substring keeps a huge parent string alive; `strings.Clone` it if you store it

QUICK SUMMARY

- `string` — immutable UTF-8 bytes
- `len(s)` / `s[i]` — byte count / byte
- `"…"` vs `` `…` `` — escapes vs raw
- `+` — concatenation creates new strings

Strings & UTF-8

UTF-8 AND RUNES

Go source and strings are UTF-8, where one character (rune) takes 1-4 bytes. `for range` over a string **decodes runes** and yields each rune's starting byte index. Converting to `[]rune` gives random access by character at the cost of a copy.

NOTES

- `range` yields `(byteIndex, rune)`, so indexes skip by 1-4
- Invalid bytes decode as `utf8.RuneError` (`U+FFFD`, `�`)
- `[]rune(s)` allocates 4 bytes per character
- Grapheme clusters (e.g. 👍🏽) can span several runes

BYTES VS RUNES

```
s := "héllo"
fmt.Println(len(s))                    // 6 bytes
fmt.Println(utf8.RuneCountInString(s)) // 5 runes

for i, r := range s {
    fmt.Printf("%d:%c ", i, r) // 0:h 1:é 3:l 4:l 5:o
}

r := []rune(s)
fmt.Println(string(r[1]))              // é
fmt.Println(string(r[1:3]))            // él

ch, size := utf8.DecodeRuneInString(s[1:])
fmt.Println(string(ch), size)          // é 2

bad := "a\xffb"
fmt.Println(utf8.ValidString(bad))     // false
fmt.Println(strings.ToValidUTF8(bad, "?")) // a?b
for _, r := range bad { fmt.Print(r, " ") } // 97 65533 98
```

CODE EXPLAINED

→ `RuneCountInString` — character count without allocating

→ `range s` — index `3` follows `1` because `é` is 2 bytes

→ `[]rune(s)` — index by character position

→ `DecodeRuneInString` — decode one rune + its byte size

→ `65533` — `U+FFFD`, the replacement for invalid bytes

COMING FROM…

- **Python** — `for ch in s` ≈ `for _, r := range s`
- **JS** — `[...s]` ≈ `[]rune(s)`
- **Java** — no surrogate pairs; runes are whole code points

UTF8 PACKAGE

FUNCTION	USE
`RuneCountInString(s)`	count characters
`ValidString(s)`	is it valid UTF-8?
`DecodeRuneInString(s)`	first rune + size
`DecodeLastRuneInString`	last rune + size
`RuneLen(r)`	bytes to encode r
`AppendRune(b, r)`	encode into []byte

BYTE VS RUNE LOOPS

LOOP	VISITS
`for i := 0; i < len(s); i++`	each byte
`for i, r := range s`	each rune
`for _, b := range []byte(s)`	each byte (copy)
`for i, r := range []rune(s)`	rune + char index

⚠ COMMON MISTAKES

- Reversing a string byte-by-byte; multi-byte characters get scrambled. Reverse a `[]rune`
- Truncating with `s[:10]` for display; it can cut a character in half
- Assuming the `range` index counts characters; it's a byte offset

✓ TIPS & BEST PRACTICES

- Truncate safely: `string([]rune(s)[:n])` (after checking length)
- Validate external input with `utf8.ValidString` before storing it
- Use `golang.org/x/text/unicode/norm` to compare accented text reliably

QUICK SUMMARY

- `range s` — decodes runes, index is bytes
- `utf8.RuneCountInString` — character count
- `[]rune(s)` — character-indexed copy
- `U+FFFD` — marker for invalid UTF-8

strings Package — Search & Split

SEARCHING & SPLITTING

The `strings` package holds the functions other languages put on a string object. They take the string as the first argument and are all **case-sensitive** unless the name says otherwise. Go 1.24 added iterator versions like `SplitSeq` and `Lines` that avoid building a slice.

NOTES

- Index functions return `-1` when not found
- `Cut` is the idiomatic "split once" (Go 1.18)
- `strings.CutLast` splits at the **last** separator (Go 1.27)
- `Fields` splits on any run of whitespace, dropping empties

SEARCH & SPLIT

```
s := "go is fun, go is fast"
strings.Contains(s, "fun")          // true
strings.ContainsAny(s, "xyz")       // false
strings.Index(s, "is")              // 3
strings.LastIndex(s, "go")          // 11
strings.HasPrefix(s, "go")          // true
strings.Count(s, "go")              // 2
strings.EqualFold("Go", "GO")       // true

k, v, ok := strings.Cut("k=v", "=")       // "k" "v" true
d, f, _ := strings.CutLast("a/b/c.go", "/") // a/b c.go
rest, ok := strings.CutPrefix("v1.2", "v") // "1.2" true

strings.Split("a,b,,c", ",")        // ["a" "b" "" "c"]
strings.SplitN("a,b,c", ",", 2)     // ["a" "b,c"]
strings.Fields("  a  b\tc ")        // ["a" "b" "c"]
for part := range strings.SplitSeq("x,y,z", ",") {
    fmt.Print(part)                 // xyz (no slice built)
}
```

CODE EXPLAINED

- → `Index` / `LastIndex` — byte offset or `-1`
- → `Cut` — before, after, found: replaces Index + slicing
- → `CutLast` — new in 1.27; great for paths and extensions
- → `Split` keeps empty fields; `Fields` drops them
- → `SplitSeq` / `Lines` — iterators (1.24) for `range`

COMING FROM…

- **Python** — `s.find` → `strings.Index`; `in` → `Contains`
- **JS** — `s.split` → `strings.Split`; methods are package funcs
- **Python** — `partition` ≈ `strings.Cut`

SEARCH

FUNCTION	RETURNS
`Contains(s, sub)`	bool
`ContainsRune(s, r)`	bool
`Index(s, sub)`	first byte index / -1
`IndexAny(s, chars)`	first of any char
`HasSuffix(s, suf)`	bool
`EqualFold(a, b)`	case-insensitive ==

SPLIT

FUNCTION	RESULT FOR "A B C"
`Split(s, " ")`	[a b c]
`SplitAfter(s, " ")`	["a " "b " c]
`SplitN(s, " ", 2)`	[a "b c"]
`Fields(s)`	[a b c]
`FieldsFunc(s, f)`	split where f(r) true
`Lines(s)`	iterator of lines (1.24)

⚠ COMMON MISTAKES

- `strings.Split("", ",")` returns `[""]` (one empty string), not an empty slice
- Using `Index` + manual slicing for key/value parsing; `Cut` is shorter and safer
- Calling `strings.Contains(strings.ToLower(a), …)` in hot loops; consider `EqualFold` or precomputing

✓ TIPS & BEST PRACTICES

- Use `strings.Fields` for user input where spacing is inconsistent
- Prefer `SplitSeq` / `FieldsSeq` in loops over large text (no allocation of the full slice)
- For repeated searches over the same text, use `regexp` or `strings.Index` with offsets

QUICK SUMMARY

- `Contains` / `Index` / `HasPrefix` — searching
- `Cut` / `CutPrefix` / `CutLast` — split once
- `Split` / `Fields` — split into a slice
- `SplitSeq` / `Lines` — iterator forms

strings Package — Transform & Join

TRANSFORMING & JOINING

Because strings are immutable, every `strings` transform returns a **new string**. The `Trim` family is the most confusing: `Trim` / `TrimLeft` / `TrimRight` remove any of a **set of characters**, while `TrimPrefix` / `TrimSuffix` remove an exact **substring** once.

NOTES

- `Replace(s, old, new, n)` — `n = -1` means all (same as `ReplaceAll`)
- `NewReplacer` does many replacements in one pass
- `strings.Title` is deprecated; use `golang.org/x/text/cases`
- `ToUpper` / `ToLower` are Unicode-aware

TRANSFORM FUNCTIONS

```
strings.ToUpper("gopher")               // GOPHER
strings.ToLower("HeLLo")                // hello
strings.TrimSpace("  hi \n")            // "hi"
strings.Trim("--hi--", "-")             // "hi"
strings.TrimLeft("xxhixx", "x")         // "hixx"
strings.TrimPrefix("v1.2.3", "v")       // "1.2.3"
strings.TrimSuffix("file.go", ".go")    // "file"
strings.TrimFunc("123abc456", unicode.IsDigit) // "abc"

strings.Replace("aaaa", "a", "b", 2)    // "bbaa"
strings.ReplaceAll("a-b-c", "-", "/")   // "a/b/c"
r := strings.NewReplacer("<", "&lt;", ">", "&gt;")
r.Replace("<b>")                        // "&lt;b&gt;"

strings.Join([]string{"a", "b"}, ", ") // "a, b"
strings.Repeat("ab", 3)                 // "ababab"
strings.Map(func(r rune) rune {         // drop vowels
    if strings.ContainsRune("aeiou", r) { return -1 }
    return r
}, "gopher")                            // "gphr"
```

CODE EXPLAINED

- → `Trim(s, "-")` — strips any `-` chars from both ends
- → `TrimPrefix` — removes the exact prefix once, if present
- → `Replace(…, 2)` — replaces only the first 2 matches
- → `NewReplacer` — reusable, safe for concurrent use
- → `Map` — return `-1` to delete a rune

COMING FROM…

- **Python** — `strip()` → `TrimSpace`; `lstrip(x)` → `TrimLeft`
- **JS** — `replaceAll` → `strings.ReplaceAll`
- **Java** — `String.join` → `strings.Join(slice, sep)`

TRIM FAMILY

FUNCTION	REMOVES
`TrimSpace(s)`	leading/trailing whitespace
`Trim(s, set)`	chars in set, both ends
`TrimLeft / TrimRight`	chars in set, one end
`TrimPrefix(s, p)`	exact prefix once
`TrimSuffix(s, x)`	exact suffix once
`TrimFunc(s, f)`	runes where f is true

CASE HANDLING

NEED	USE
`Upper / lower`	`ToUpper, ToLower`
`Title Case`	`cases.Title(language.English)`
`Compare ignoring case`	`EqualFold`
`Turkish i rules`	`ToUpperSpecial`
`Capitalize first`	`[]rune + unicode.ToUpper`

⚠ COMMON MISTAKES

- `TrimLeft("version", "ver")` → `"sion"`; it trims characters, use `TrimPrefix` for a word
- `TrimRight(name, ".go")` turns `"logo.go"` into `"l"`; use `TrimSuffix`
- Forgetting that strings.Replace returns a value: `strings.ReplaceAll(s, …)` alone does nothing

✓ TIPS & BEST PRACTICES

- Build reusable `strings.NewReplacer` values once at package level
- For HTML escaping use `html.EscapeString`, not a hand-made replacer
- Use `strings.Join` instead of concatenating with separators in a loop

QUICK SUMMARY

- `ToUpper` / `ToLower` — case
- `TrimSpace` / `TrimPrefix` / `TrimSuffix` — trimming
- `Replace` / `ReplaceAll` / `NewReplacer` — substitution
- `Join` / `Repeat` / `Map` — building

strings.Builder & bytes

BUILDING STRINGS EFFICIENTLY

`strings.Builder` accumulates text in a growing byte buffer and produces the final string **without an extra copy**. `bytes.Buffer` is the read/write cousin for `[]byte` data. The `bytes` package mirrors nearly every `strings` function for `[]byte`, so you can process raw data without converting.

NOTES

- Zero values are ready: `var b strings.Builder`
- Never copy a non-empty `Builder` (it panics); pass `*strings.Builder`
- Both implement `io.Writer`, so `fmt.Fprintf(&b, …)` works
- `strings.NewReader(s)` turns a string into an `io.Reader`

BUILDER & BUFFER

```
var b strings.Builder
b.Grow(64)                          // optional: pre-size
for i := 1; i <= 3; i++ {
    fmt.Fprintf(&b, "item%d,", i)  // io.Writer
}
b.WriteString("done")
b.WriteByte('!')
b.WriteRune('✓')
fmt.Println(b.String())  // item1,item2,item3,done!✓

var buf bytes.Buffer
buf.WriteString("hello ")
buf.Write([]byte("world"))
line, _ := buf.ReadString(' ')     // "hello " (consumes it)
fmt.Println(line, buf.String())    // hello  world

data := []byte("  Go,Rust  ")
bytes.TrimSpace(data)              // []byte("Go,Rust")
bytes.Equal([]byte("a"), []byte("a")) // true
r := strings.NewReader("streamed text") // io.Reader
```

CODE EXPLAINED

- → `Grow(n)` — reserve capacity to avoid re-allocations
- → `Fprintf(&b, …)` — formatted writes into the builder
- → `b.String()` — no copy: returns the internal bytes
- → `bytes.Buffer` — supports both writing and reading
- → `bytes.*` — `strings` equivalents for `[]byte`

COMING FROM…

- **Java** — `StringBuilder` ≈ `strings.Builder`
- **Python** — `''.join(parts)` ≈ `strings.Join`
- **C#** — `MemoryStream` ≈ `bytes.Buffer`

BUILDER VS BUFFER

FEATURE	BUILDER / BUFFER
`Result`	`string` / `[]byte` or string
`Read back`	no / yes (Read*)
`String() cost`	free / copies
`Reset`	`Reset()` / `Reset()`, `Truncate`
`Best for`	building text / I/O buffering

CONCATENATION COST

APPROACH	WHEN
`a + b`	a few pieces
`fmt.Sprintf`	formatting needed
`strings.Join`	slice of parts
`strings.Builder`	loops, many parts
`fmt.Appendf`	into existing []byte

⚠ COMMON MISTAKES

- Copying a `strings.Builder` by value after writing: "illegal use of non-zero Builder copied by value"
- Calling `buf.String()` on `bytes.Buffer` repeatedly in a loop; each call copies
- Converting `[]byte` → `string` → `[]byte` just to call a `strings` func; use the `bytes` version

✓ TIPS & BEST PRACTICES

- Call `Grow` when you can estimate the final size
- Use `bytes.Buffer` as an in-memory `io.Reader` / `io.Writer` in tests
- `strings.Builder` is not goroutine-safe; build per goroutine and join

QUICK SUMMARY

- `strings.Builder` — efficient string building
- `bytes.Buffer` — read/write byte buffer
- `bytes` pkg — `strings` API for `[]byte`
- `strings.NewReader` — string as `io.Reader`

strconv

STRING ↔ VALUE CONVERSION

`strconv` converts between strings and numbers or booleans. Parse functions return `(value, error)`: the error is a `*strconv.NumError` whose `Err` is `strconv.ErrSyntax` (not a number) or `strconv.ErrRange` (too big). Format functions never fail. `Append*` variants write into an existing `[]byte`.

ARGUMENTS

- `base` — 2-36, or `0` to detect `0x`, `0o`, `0b` prefixes
- `bitSize` — 8/16/32/64 range check; result is still `int64`
- Float format byte: `'f'` decimal, `'e'` scientific, `'g'` compact
- Precision `-1` means shortest that round-trips

PARSE & FORMAT

```
n, err := strconv.Atoi("42")              // 42 <nil>
_, err = strconv.Atoi("4x")
fmt.Println(err) // …parsing "4x": invalid syntax

i, _ := strconv.ParseInt("-ff", 16, 64)  // -255
h, _ := strconv.ParseInt("0x1F", 0, 64)  // 31 (prefix)
u, _ := strconv.ParseUint("255", 10, 8)  // 255
_, err = strconv.ParseInt("300", 10, 8)  // out of range
f, _ := strconv.ParseFloat("3.14", 64)   // 3.14
b, _ := strconv.ParseBool("t")           // true

strconv.Itoa(-7)                          // "-7"
strconv.FormatInt(255, 2)                 // "11111111"
strconv.FormatFloat(3.14159, 'f', 2, 64) // "3.14"
strconv.FormatFloat(1e21, 'g', -1, 64)   // "1e+21"
strconv.Quote("hi\n")                     // "\"hi\\n\""
strconv.Unquote(`"a\tb"`)                 // "a	b" <nil>
buf := strconv.AppendInt(nil, 42, 10)     // []byte("42")
if errors.Is(err, strconv.ErrRange) { /* too big */ }
```

CODE EXPLAINED

- → `Atoi` — shorthand for `ParseInt(s, 10, 0)` → `int`
- → base `0` — honors `0x`, `0o`, `0b` and `_` separators
- → bitSize `8` — rejects values that don't fit `int8`
- → `FormatFloat(x, 'f', 2, 64)` — fixed 2 decimals
- → `errors.Is(err, strconv.ErrRange)` — classify failures

COMING FROM…

- **Python** — `int(s)` → `strconv.Atoi` + error check
- **JS** — `parseInt` never throws; Go returns an error
- **Java** — `Integer.parseInt` throws; Go returns `err`

PARSE

FUNCTION	RETURNS
`Atoi(s)`	int, error
`ParseInt(s, base, bits)`	int64, error
`ParseUint(s, base, bits)`	uint64, error
`ParseFloat(s, bits)`	float64, error
`ParseBool(s)`	1 t T TRUE true True…

FORMAT

FUNCTION	EXAMPLE
`Itoa(n)`	"42"
`FormatInt(n, 16)`	"2a"
`FormatFloat(f,'e',3,64)`	"3.142e+00"
`FormatBool(b)`	"true"
`Quote(s) / QuoteRune(r)`	Go-quoted literal

⚠ COMMON MISTAKES

- Ignoring the parse error: `n, _ := strconv.Atoi(input)` silently gives 0 on bad input
- Using `fmt.Sprint(n)` in hot paths; `strconv.Itoa` is several times faster
- Passing bitSize 32 and storing straight into `int32` without converting the `int64` result

✓ TIPS & BEST PRACTICES

- Trim input first: `strconv.Atoi(strings.TrimSpace(s))`
- Use `strconv.Append*` with a reusable buffer to avoid allocations
- Use `strconv.Quote` for safely printing untrusted strings in logs

QUICK SUMMARY

- `Atoi` / `Itoa` — int ↔ string
- `ParseInt/Float/Bool` — parse with error
- `FormatInt/Float` — formatting options
- `ErrSyntax` / `ErrRange` — error kinds

Arithmetic Operators

ARITHMETIC OPERATORS

Go's arithmetic operators work on numbers of the **same type**; mixing `int` and `float64` needs a conversion. Integer division truncates toward zero, and `%` takes the sign of the left operand. `++` and `--` are **statements**, not expressions, so `y := x++` doesn't compile.

NOTES

- No power operator: use `math.Pow(x, y)` or `1 << n` for powers of 2
- `+` also concatenates strings
- Integer overflow wraps silently; there is no checked arithmetic
- Int division by a zero **variable** panics; float division gives `±Inf`

ARITHMETIC IN PRACTICE

```
a, b := 17, 5
fmt.Println(a+b, a-b, a*b)    // 22 12 85
fmt.Println(a / b)            // 3   (truncated)
fmt.Println(a % b)            // 2
fmt.Println(-17 / 5)          // -3  (toward zero)
fmt.Println(-17 % 5)          // -2  (sign of dividend)
fmt.Println(17.0 / 5)         // 3.4

x := 10
x++                           // statement only
// y := x++                   // syntax error
fmt.Println(-x, +x)           // -11 11

f := float64(a) / float64(b) // 3.4: convert first
avg := float64(a+b) / 2       // 11

zero := 0
// _ = a / zero               // panic: divide by zero
fmt.Println(math.Pow(2, 10)) // 1024
```

CODE EXPLAINED

- → `17 / 5` — both ints, so the result is an int (3)
- → `-17 % 5` — `-2`, not `3` like Python
- → `17.0 / 5` — untyped constants: float division
- → `x++` — a statement; can't be used inside expressions
- → `float64(a) / float64(b)` — convert before dividing

COMING FROM…

- **Python** — `//` floor vs Go truncation: `-7/2` is `-3`
- **Python / JS** — no `**`; use `math.Pow`
- **C / Java** — `i++` is a statement, not an expression

OPERATORS

OP	NAME	EXAMPLE
`+`	sum / concat	`3 + 4` → 7
`-`	difference	`3 - 4` → -1
`*`	product	`3 * 4` → 12
`/`	quotient	`7 / 2` → 3
`%`	remainder (ints)	`7 % 2` → 1
`++ --`	inc / dec stmt	`i++`

FLOAT HELPERS

NEED	USE
Power	`math.Pow(x, y)`
Float remainder	`math.Mod(7.5, 2)` → 1.5
Floor division	`math.Floor(a / b)`
Absolute value	`math.Abs(x)`
Positive modulo	`((a % n) + n) % n`

⚠ COMMON MISTAKES

- `avg := sum / count` with ints; the fraction is lost. Convert to `float64` first
- Using `%` to wrap negative indexes; `-1 % 5` is `-1`, use `((i % n) + n) % n`
- Writing `x**2`; Go has no exponent operator (`**` is a pointer-deref syntax error)

✓ TIPS & BEST PRACTICES

- Use `x*x` instead of `math.Pow(x, 2)` for small integer powers (faster, exact)
- Use `min(a, b)` / `max(a, b)` builtins (Go 1.21) instead of if-statements
- Guard divisions on user input: `if d == 0 { return errDivZero }`

QUICK SUMMARY

- `/` on ints — truncates toward zero
- `%` — sign follows the dividend
- `++` / `--` — statements only
- `math.Pow` — no `**` operator

Sheet 023

Comparison & Logical Operators

== != < <= > >= · comparable types (struct, array, pointer, chan, interface) · slices/maps/funcs compare only to nil · && || ! short-circuit · no ternary · cmp.Compare · min / max builtins

COMPARISON & LOGIC

Comparison operators return an untyped `bool`. Both sides must be the same type (or one an untyped constant). Only **comparable** types work with ==: numbers, strings, booleans, pointers, channels, interfaces, and arrays or structs made of comparable fields. Slices, maps and funcs can only be compared to `nil`.

NOTES

- `&&` and `||` short-circuit: the right side may not run
- No ternary `? :`; write an `if` / `else`
- `cmp.Compare(a, b)` returns -1, 0 or +1 (Go 1.21)
- Comparing interfaces holding uncomparable values **panics** at runtime

COMPARISONS

```
fmt.Println(3 < 5, "a" >= "b", 2 != 2) // true false false

type Point struct{ X, Y int }
p1, p2 := Point{1, 2}, Point{1, 2}
fmt.Println(p1 == p2)                   // true (field-wise)
fmt.Println([2]int{1, 2} == [2]int{1, 2}) // true

s1, s2 := []int{1}, []int{1}
// s1 == s2                             // compile error
fmt.Println(slices.Equal(s1, s2))       // true
fmt.Println(s1 != nil)                  // true

ok := len(s1) > 0 && s1[0] == 1  // short-circuit
fmt.Println(!ok)                        // false

fmt.Println(cmp.Compare(2, 5))          // -1
fmt.Println(min(3, 1, 2), max(4.5, 2)) // 1 4.5

label := "odd"                          // no ternary:
if n%2 == 0 { label = "even" }
```

CODE EXPLAINED

→ Struct and array `==` compare every field/element

→ `slices.Equal` / `maps.Equal` / `bytes.Equal` for the rest

→ `len(s) > 0 && s[0]…` — right side skipped when empty

→ `cmp.Compare` — three-way compare for sort funcs

→ `min` / `max` — variadic builtins since Go 1.21

COMING FROM…

- **JS** — no `===`; `==` never coerces types
- **Python** — `==` on structs compares fields; slices need `slices.Equal`
- **C / Java** — no `?:` ternary operator

COMPARABLE?

TYPE	==
`numbers, string, bool`	✔
`pointer, chan`	✔ (identity)
`array, struct`	✔ if elements are
`interface`	✔ (may panic)
`slice, map, func`	only == `nil`

OPERATORS

OP	MEANING
`== !=`	equal / not equal
`< <= > >=`	ordered types only
`&&`	AND, short-circuit
`||`	OR, short-circuit
`!`	NOT

EQUALITY HELPERS

FUNC	FOR
`slices.Equal`	slices
`maps.Equal`	maps
`bytes.Equal`	[]byte
`reflect.DeepEqual`	anything (slow)
`strings.EqualFold`	case-insensitive

⚠ COMMON MISTAKES

- Using a slice or map as a map key; keys must be comparable (use an array or string key)
- `reflect.DeepEqual` in production hot paths; it's slow and treats nil vs empty slices as different
- Comparing floats with `==` after arithmetic (see sheet 011)

✓ TIPS & BEST PRACTICES

- Order cheap checks first in `&&` chains to skip expensive ones
- Make structs comparable (no slice fields) if you want them as map keys
- Use `cmp.Or(a, b, c)` to pick the first non-zero value

QUICK SUMMARY

- `== != < >` — same-type comparisons
- structs/arrays — comparable field-by-field
- slices/maps/funcs — only `== nil`
- `&&` / `||` — short-circuit; no ternary

Bitwise Operators

BITWISE OPERATORS

Bitwise operators work on integers bit by bit. Go adds **&^ (AND NOT, bit clear)**, and uses ^x as unary NOT instead of ~. Right shift is **arithmetic** for signed integers (sign bit copied) and logical for unsigned. Shifting by a negative count panics.

NOTES

- `^` is XOR when binary, NOT when unary
- `x &^ y` clears in `x` every bit set in `y`
- Shift counts may be any integer type (since Go 1.13)
- `math/bits` has popcount, rotate, leading/trailing zeros

BITS IN ACTION

```
a, b := 0b1100, 0b1010          // 12, 10
fmt.Printf("%04b\n", a&b)       // 1000  AND
fmt.Printf("%04b\n", a|b)       // 1110  OR
fmt.Printf("%04b\n", a^b)       // 0110  XOR
fmt.Printf("%04b\n", a&^b)      // 0100  AND NOT
fmt.Println(^0)                 // -1    NOT
fmt.Println(1<<4, 64>>2)        // 16 16
fmt.Println(-16 >> 2)           // -4 (sign kept)

const (
    FlagA uint8 = 1 << iota     // 0001
    FlagB                       // 0010
    FlagC                       // 0100
)
f := FlagA | FlagC              // set
f &^= FlagA                     // clear
on := f&FlagC != 0              // test → true
f ^= FlagB                      // toggle

bits.OnesCount(255)             // 8
bits.LeadingZeros64(1)          // 63
bits.RotateLeft8(0b1000_0001, 1) // 3
```

CODE EXPLAINED

- → `%04b` — binary, padded to 4 digits
- → `a &^ b` — keep bits of `a` not in `b`
- → `-16 >> 2` — arithmetic shift keeps the sign
- → set `|=`, clear `&^=`, test `& != 0`, toggle `^=`
- → `math/bits` — fast intrinsics for bit tricks

COMING FROM…

- **C / Java** — `^x` is NOT (not `~x`)
- **C** — `&^` (bit clear) is Go-only
- **Java** — no `>>>`; use unsigned types instead

TRUTH TABLE

A	B	&	|	^	&^
0	0	0	0	0	0
0	1	0	1	1	0
1	0	0	1	1	1
1	1	1	1	0	0

FLAG IDIOMS

GOAL	CODE
Set bit	`f |= mask`
Clear bit	`f &^= mask`
Toggle bit	`f ^= mask`
Test bit	`f&mask != 0`
Lowest set bit	`x & -x`

⚠ COMMON MISTAKES

- Writing `~x` for NOT (C habit); Go uses `^x`
- Shifting past the type width: `uint8(1) << 8` is `0` at runtime and a compile error for constants
- Shifting a signed value expecting zeros on the left; convert to unsigned first

✓ TIPS & BEST PRACTICES

- Use typed constants (`type Perm uint8`) so flags don't mix with other ints
- Give the flag type a `String()` method for readable logs
- Use `bits.Len(x)` to find the number of bits needed to represent `x`

QUICK SUMMARY

- `& | ^` — AND, OR, XOR
- `&^` — bit clear (AND NOT)
- `^x` — bitwise NOT
- `<< >>` — shifts; signed `>>` keeps the sign

Sheet 025

Assignment & Precedence

= vs := · += -= &^= <<= · tuple assignment a, b = b, a · := needs ≥ 1 new variable · 5-level precedence table · <- receive operator · evaluation order

ASSIGNMENT & PRECEDENCE

= assigns to existing variables and := declares new ones. Compound operators like += and &^= combine an operation with assignment. In a **tuple assignment**, all right-hand values are evaluated before any assignment happens, which makes swaps a one-liner. Go has only **five precedence levels** for binary operators.

NOTES

- Assignment is a statement, not an expression: no `if x = f()`
- `:=` reuses existing names in the same scope if at least one is new
- Unary operators (`!` , `-` , `^` , `*` , `&` , `<-`) bind tightest
- Operators of equal precedence group left to right

ASSIGNMENT FORMS

```
x := 10           // declare + assign
x = 20            // assign
x += 5            // 25
x -= 3            // 22
x *= 2            // 44
x /= 4            // 11
x %= 4            // 3
x <<= 2           // 12
x &^= 4           // 8

a, b := 1, 2
a, b = b, a       // swap → 2 1
i, s := 0, []int{10, 20}
i, s[i] = 1, 9    // index evaluated first: s[0] = 9

v, err := strconv.Atoi("7")   // declare both
w, err := strconv.Atoi("8")   // OK: w is new, err reused

fmt.Println(1 + 2*3)          // 7
fmt.Println(1<<2 + 1)         // 5: << binds tighter than +
```

CODE EXPLAINED

- → `x op= y` — same as `x = x op y` (x evaluated once)
- → `a, b = b, a` — swap without a temp variable
- → `i, s[i] = …` — operands on the left are evaluated first
- → `w, err :=` — legal because `w` is new
- → `1<<2 + 1` — `<<` is level 5, `+` is level 4

COMING FROM…

- **Python** — `a, b = b, a` works the same
- **C** — `x = y = 0` chaining isn't allowed
- **JS** — assignment isn't an expression: no `if (x = f())`

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
5	* / % << >> & &^
4	+ - | ^
3	== != < <= > >=
2	&&
1	||

COMPOUND OPS

OP	MEANING
+= -= *= /=	arithmetic
%=	remainder
&= |= ^= &^=	bitwise
<<= >>=	shifts
++ --	±1 (statements)

⚠ COMMON MISTAKES

- Assuming C precedence: Go's `<<` binds tighter than `+`, so `a + b<<1` is `a + (b<<1)`
- `if x := f(); x = 5 {` typos; Go rejects assignment in conditions
- `:=` inside a block meant to update an outer variable creates a shadow

✓ TIPS & BEST PRACTICES

- Add parentheses when mixing shifts and arithmetic; clarity beats memorizing levels
- `gofmt` hints precedence through spacing: `1<<2 + 1`
- Use tuple assignment for swaps and Fibonacci-style updates: `a, b = b, a+b`

QUICK SUMMARY

- `=` assign / `:=` declare
- `+=` , `&^=` , `<<=` — compound assignment
- `a, b = b, a` — tuple assignment
- 5 levels — `* / << &` > `+ - | ^` > compare > `&&` > `||`

Pointers

T type · &x address-of · p dereference · new(T) / new(expr) (1.26) · nil-pointer deref panic · no pointer arithmetic · auto-deref p.Field · returning &local is safe (escapes to heap)

POINTERS

A pointer holds the address of a value. &x takes an address, *p reads or writes through it, and *T is the pointer type. Go has **no pointer arithmetic** and is garbage collected, so returning a pointer to a local variable is safe: the compiler moves it to the heap if needed (escape analysis).

NOTES

- Zero value is `nil`; dereferencing `nil` panics
- `p.Field` auto-dereferences struct pointers (no `->`)
- `new(T)` allocates a zeroed `T`; Go 1.26 also allows `new(expr)`
- Use pointers to mutate a caller's value or avoid copying big structs

POINTER BASICS

```
x := 10
p := &x               // p is *int
fmt.Println(*p)       // 10
*p = 20               // write through the pointer
fmt.Println(x)        // 20

func double(n *int) { *n *= 2 }
double(&x)            // x is now 40

type User struct{ Name string }
u := &User{Name: "Ann"}  // pointer to composite literal
u.Name = "Bo"            // same as (*u).Name

q := new(int)         // *int → 0
r := new(42)          // *int → 42 (Go 1.26)
s := new(time.Now()) // *time.Time (1.26)

var np *User
// fmt.Println(np.Name) // panic: nil pointer dereference
if np != nil { fmt.Println(np.Name) }

func newCounter() *int { c := 0; return &c } // safe
```

CODE EXPLAINED

- → `&x` — address of `x`; `*p` — value at `p`
- → `double(&x)` — pass a pointer to let the callee modify `x`
- → `u.Name` — automatic dereference for struct fields
- → `new(expr)` — allocate and initialize in one step (1.26)
- → `&c` escaping — the GC keeps `c` alive

COMING FROM…

- **C** — no pointer arithmetic; returning `&local` is safe
- **Java / Python** — explicit pointers decide sharing vs copying
- **C++** — `p.Field` auto-derefs; no `->`

SYNTAX

FORM	MEANING
`*T`	pointer-to-T type
`&x`	address of x
`*p`	dereference
`&T{…}`	pointer to new composite
`new(T)`	pointer to zero T

VALUE OR POINTER?

SITUATION	USE
`Callee must modify`	pointer
`Large struct (≥ ~64B)`	pointer
`Small, immutable`	value
`Maps, slices, chans`	value (already refs)
`Optional field`	pointer (nil = unset)

⚠ COMMON MISTAKES

- Dereferencing a pointer that may be `nil` (e.g. a failed lookup result) without checking
- Taking pointers to maps or slices to "pass by reference"; they already share data
- Returning `&s[i]` and then appending to `s`; the pointer may refer to the old backing array

✓ TIPS & BEST PRACTICES

- Prefer values by default; reach for pointers for mutation or large structs
- Use `new(expr)` (1.26) for optional fields: `Port: new(8080)`
- Run `go build -gcflags=-m` to see which variables escape to the heap

QUICK SUMMARY

- `&x` / `*p` — address-of / dereference
- `*T` — pointer type; zero value `nil`
- `new(T)` / `new(expr)` — allocate
- no pointer arithmetic; escaping is safe

if / else

IF / ELSE

Go's `if` needs no parentheses but **always requires braces**. It can start with a short **init statement** whose variables are scoped to the `if` and its `else` branches, the idiom behind `if err := f(); err != nil`. Go code favors **early returns** (guard clauses) over deep `else` nesting.

NOTES

- Condition must be a `bool`; no truthy values
- `else` must be on the same line as the closing `}`
- Init-statement variables are visible in every branch, not after
- There is no ternary operator

IF FORMS

```
if n > 0 {
    fmt.Println("positive")
} else if n < 0 {
    fmt.Println("negative")
} else {
    fmt.Println("zero")
}

if v, ok := cache[key]; ok {      // comma-ok + init stmt
    return v
}

if err := save(doc); err != nil { // err scoped to this if
    return fmt.Errorf("save: %w", err)
}

func process(u *User) error {     // guard clauses
    if u == nil { return errNoUser }
    if !u.Active { return errInactive }
    return send(u)                // happy path unindented
}
```

CODE EXPLAINED

- → `else if` — chain as many as needed
- → `if v, ok := m[k]; ok` — lookup and test in one line
- → `if err := …; err != nil` — keeps `err` out of the outer scope
- → Guard clauses — handle failures first, then the main logic
- → gofmt keeps `} else {` on one line

COMING FROM…

- **Python** — braces required; no parentheses around conditions
- **JS / Java** — `if x := f(); x > 0` scopes `x` to the if
- **All** — no ternary; use if/else

COMMON CONDITIONS

PATTERN	MEANING
`if err != nil`	error check
`if v, ok := m[k]; ok`	map key exists
`if s, ok := x.(T); ok`	type assertion succeeded
`if len(s) == 0`	empty slice/string
`if p != nil && p.OK`	nil-safe field check

TERNARY ALTERNATIVES

NEED	GO
`x = c ? a : b`	`x := b; if c { x = a }`
`default if empty`	`cmp.Or(s, "default")`
`smaller of two`	`min(a, b)`
`bool → int`	`if b { n = 1 }`

⚠ COMMON MISTAKES

- Putting `else` on a new line after `}`: syntax error (automatic semicolon)
- Using a variable from the init statement after the `if` ends; it's out of scope
- Nesting the happy path inside `if err == nil { … }` instead of returning early

✓ TIPS & BEST PRACTICES

- Keep the success path at the left margin; return early on errors
- Use the init statement to limit the scope of temporary variables
- Drop the `else` after a branch that ends in `return` (golint `indent-error-flow`)

QUICK SUMMARY

- `if cond { }` — braces required, no parens
- `if x := f(); x > 0` — init statement
- `else if` / `else` — on the `}` line
- guard clauses — return early

switch

SWITCH

A `switch` evaluates cases **top to bottom** and runs the first match, then stops: there is **no implicit fallthrough** and no `break` needed. Cases can list several values, and a **tagless** `switch` (no expression) is a clean replacement for long `if–else` chains. Use the explicit `fallthrough` keyword to continue into the next case.

NOTES

- Case values needn't be constants: any expression works
- `default` can appear anywhere; it runs if nothing matches
- `break` exits the switch (not an enclosing loop)
- Type switches `switch v := x.(type)` are on sheet 059

SWITCH FORMS

```
switch day {
case "Sat", "Sun":                  // multiple values
    fmt.Println("weekend")
case "Fri":
    fmt.Println("almost")
    fallthrough                     // run next body too
case "Mon", "Tue", "Wed", "Thu":
    fmt.Println("workday")
default:
    fmt.Println("unknown")
}

switch {                            // tagless: like if-else
case score >= 90:
    grade = "A"
case score >= 80:
    grade = "B"
default:
    grade = "C"
}

switch os := runtime.GOOS; os {     // init statement
case "darwin", "linux":
    fmt.Println("unix-like:", os)
}
```

CODE EXPLAINED

- → `case "Sat", "Sun"` — matches either value
- → `fallthrough` — unconditionally enters the next case body
- → `switch {` — each case is a boolean expression
- → `switch x := …; x {` — init statement, scoped to switch
- → Cases are checked in order; first match wins

COMING FROM…

- **C / Java / JS** — no fallthrough and no `break` needed
- **Python** — like `match`, but cases can be any expression
- **Rust** — `switch {}` ≈ `match` with guards

SWITCH VS C/JAVA

BEHAVIOR	GO
Fallthrough	off; opt in with `fallthrough`
break needed	no
Case values	any expression
Multiple values	`case 1, 2, 3:`
No tag	`switch { case cond: }`

EXITING A LOOP

INSIDE SWITCH	EFFECT
break	leaves switch only
break Loop	leaves labeled loop
continue	next loop iteration
return	leaves function

⚠ COMMON MISTAKES

- Adding `break` to every case out of habit; harmless but noisy
- Using `break` inside a `switch` in a `for` loop expecting to exit the loop; use a label
- Expecting `fallthrough` to re-check the next case condition; it doesn't

✓ TIPS & BEST PRACTICES

- Prefer a tagless `switch` over 3+ `else if` branches
- List enum cases explicitly; the `exhaustive` linter flags missing ones
- Put the most common case first for readability (no performance difference)

QUICK SUMMARY

- `switch x { case a, b: }` — value switch
- no fallthrough — first match only
- `fallthrough` — explicit continue
- `switch {` — clean if-else chains

for Loops

three-clause for i := 0; i < n; i++ · while-style for cond · infinite for { } · per-iteration loop variables (1.22) · multiple vars i, j := 0, n-1 · for range 10 (1.22)

FOR: GO'S ONLY LOOP

Go has a single loop keyword, `for`, used in three shapes: the classic **three-clause** loop, a **condition-only** loop (Go's `while`), and an **infinite** loop. Since **Go 1.22** each iteration gets a **fresh copy** of the loop variable, which fixed the classic closure-capture bug. `for range n` counts from 0 to n-1.

NOTES

- No parentheses; braces required
- Any clause may be omitted: `for ; i < n; {`
- No `while` or `do-while` keywords
- Loop variables are per-iteration from Go 1.22 (module `go` ≥ 1.22)

LOOP SHAPES

```
for i := 0; i < 3; i++ {          // classic
    fmt.Print(i, " ")             // 0 1 2
}

n := 1
for n < 100 {                     // while-style
    n *= 2
}                                 // n == 128

for {                             // infinite
    line, err := r.ReadString('\n')
    if err != nil { break }
    process(line)
}

for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
    s[i], s[j] = s[j], s[i]       // reverse in place
}

for i := range 5 { fmt.Print(i) } // 01234 (Go 1.22)
for range 3 { fmt.Print("hi ") }  // hi hi hi
```

CODE EXPLAINED

- → `for init; cond; post` — the C-style form
- → `for cond` — runs while `cond` is true
- → `for {}` — exit with `break` or `return`
- → `i, j := 0, n-1` — parallel assignment in the clauses
- → `range 5` — integers 0-4, no variable needed with `for range`

COMING FROM…

- **C / Java** — no `while` / `do`; `for cond {}` instead
- **Python** — `for i := range 5` ≈ `for i in range(5)`
- **JS** — loop vars are per-iteration (since 1.22), like `let`

LOOP FORMS

FORM	EQUIVALENT
`for i := 0; i < n; i++`	C/Java for
`for cond`	while (cond)
`for`	while (true)
`for i := range n`	0 … n-1 (1.22)
`for k, v := range x`	for-each (sheet 030)

DO-WHILE IN GO

PATTERN	CODE
Run ≥ once	`for { …; if !cond { break } }`
Retry loop	`for attempt := range 3 { … }`
Poll	`for !done() { time.Sleep(d) }`

⚠ COMMON MISTAKES

- Modifying the slice length inside a three-clause loop over it; recompute bounds carefully
- Relying on per-iteration variables in a module whose `go.mod` says `go 1.21` or older
- Busy-waiting with `for !done {}`; use channels, `sync.WaitGroup` or a `Ticker`

✓ TIPS & BEST PRACTICES

- Use `for range n` for simple repetition
- Name nested loop labels (`outer:`) when you need to `break` out of both
- Prefer `for range` over index loops unless you need the index arithmetic

QUICK SUMMARY

- `for init; cond; post` — classic
- `for cond` — while
- `for {}` — forever
- `for range n` — count (1.22)

for range

FOR RANGE

`for range` iterates over arrays, slices, strings, maps, channels, integers and **iterator functions**. It yields one or two values depending on the type. The value variable is a **copy** of the element, so assigning to it doesn't change the collection; use the index to modify in place.

NOTES

- Map iteration order is **randomized** on purpose
- Strings range over **runes**, with byte indexes
- The range expression is evaluated **once** before the loop
- Ranging over a `nil` slice or map runs zero times

RANGING OVER EVERYTHING

```
nums := []int{10, 20, 30}
for i, v := range nums { fmt.Print(i, ":", v, " ") }
for i := range nums { nums[i] *= 2 }   // modify in place
for _, v := range nums { v++ }  // no effect: v is a copy

ages := map[string]int{"ann": 30, "bo": 25}
for name, age := range ages {          // random order
    fmt.Println(name, age)
}

for i, r := range "hé!" { fmt.Print(i, string(r)) } //0h1é3!

ch := make(chan int)
go func() { ch <- 1; ch <- 2; close(ch) }()
for v := range ch { fmt.Print(v) }     // 12, ends on close

for i := range 3 { fmt.Print(i) }      // 012 (Go 1.22)

for i, v := range slices.Backward(nums) { // iterator (1.23)
    fmt.Print(i, v, " ")                  // 2 60 1 40 0 20
}
```

CODE EXPLAINED

- → `i, v` — index and copy of element
- → `for i := range s` — index only; write `s[i]` to modify
- → maps — `key, value`; order differs between runs
- → channels — one value; loop ends when channel is closed
- → functions — any `iter.Seq` / `iter.Seq2` (sheet 054)

COMING FROM…

- **Python** — `for i, v := range s` ≈ `enumerate(s)`
- **JS** — like `for…of`, but yields index + value
- **Java** — enhanced for; `v` is a copy, not a reference

WHAT RANGE YIELDS

OVER	1ST, 2ND VALUE
`array / slice`	index, element copy
`string`	byte index, rune
`map`	key, value
`channel`	element (one value)
`integer n`	0 … n-1
`iter.Seq2[K,V]`	K, V

SKIPPING VALUES

WRITE	GETS
`for i := range s`	index only
`for _, v := range s`	value only
`for range s`	neither (just repeat)
`for k := range m`	keys only

⚠ COMMON MISTAKES

- Taking `&v` in the loop to store pointers; each points to a copy. Use `&s[i]`
- Depending on map order in tests or output; sort the keys (sheet 050)
- Ranging over a channel nobody closes; the loop blocks forever

✓ TIPS & BEST PRACTICES

- Use `slices.Sorted(maps.Keys(m))` for deterministic map iteration
- For large structs, range by index to avoid copying each element
- Deleting map entries during `range` is safe; adding may or may not be visited

QUICK SUMMARY

- `for i, v := range s` — index + copy
- maps — random order
- strings — runes; channels — until closed
- `range n`, `range iter` — ints & iterators

Sheet 031

Labels, break, continue & goto

break / continue · labeled break outer · continue outer · breaking out of select / switch inside loops · goto rules · return from nested loops

BREAK, CONTINUE, LABELS & GOTO

`break` exits the innermost `for`, `switch` or `select`; `continue` skips to the next loop iteration. A **label** before a loop lets you break or continue an **outer** loop, which is the clean way out of nested loops or a `select` inside a `for`. `goto` exists but is rarely used.

NOTES

- Labels are identifiers followed by `:` on the statement
- Unused labels are compile errors
- `goto` can't jump over variable declarations or into blocks
- Inside `select`, plain `break` exits only the `select`

LOOP CONTROL

```
for i := range 10 {
    if i%2 == 0 { continue }    // skip evens
    if i > 7 { break }          // stop at 9
    fmt.Print(i, " ")           // 1 3 5 7
}

outer:
for _, row := range grid {
    for _, cell := range row {
        if cell == target { break outer } // exits both
        if cell < 0 { continue outer } // next row
    }
}

loop:
for {
    select {
    case msg := <-msgs:
        handle(msg)
    case <-ctx.Done():
        break loop  // plain break only exits select
    }
}
```

CODE EXPLAINED

- → `continue` — jumps to the post statement / next element
- → `break outer` — terminates the labeled loop
- → `continue outer` — next iteration of the outer loop
- → `break loop` — required to leave a `for` / `select` combo
- → `return` is often clearer than labels inside a helper func

COMING FROM…

- **Java** — same labeled `break outer` / `continue outer`
- **Python** — no labels; Go doesn't need flag variables
- **C** — `goto` exists but is restricted

STATEMENT TARGETS

STATEMENT	APPLIES TO
`break`	for, switch, select
`break L`	labeled for/switch/select
`continue`	for only
`continue L`	labeled for
`goto L`	label in same function

ALTERNATIVES

INSTEAD OF	CONSIDER
`deep labels`	extract a func + `return`
`flag variables`	`break outer`
`goto cleanup`	`defer`
`search loop`	`slices.IndexFunc`

⚠ COMMON MISTAKES

- `break` inside `switch` or `select` within a loop, expecting to leave the loop
- Declaring a label and never using it: "label L defined and not used"
- Using `goto` for cleanup paths; `defer` handles that reliably

✓ TIPS & BEST PRACTICES

- Name labels after what they loop over: `rows:`, `retry:`, `loop:`
- If you need two labels, the code probably wants a helper function
- Use `slices.ContainsFunc` / `IndexFunc` to replace simple search loops

QUICK SUMMARY

- `break` / `continue` — innermost loop
- `break L` / `continue L` — labeled outer loop
- for + select — needs a label to exit
- `goto` — legal, rarely needed

defer

DEFER

`defer` schedules a function call to run **when the surrounding function returns**, whether normally or by panic. Deferred calls run in **LIFO** order. The call's **arguments are evaluated immediately** at the `defer` statement, but the call itself runs later. It's Go's tool for cleanup: closing files, unlocking mutexes, ending spans.

NOTES

- Defers run at **function** exit, not block or loop-iteration exit
- Deferred closures can read and modify **named results**
- `os.Exit` skips deferred calls; `panic` runs them
- Defer cost is near zero since Go 1.14 (open-coded defers)

DEFER PATTERNS

```
func readConfig(path string) (cfg Config, err error) {
    f, err := os.Open(path)
    if err != nil { return cfg, err }
    defer f.Close()                  // runs on every return

    defer func() {                   // sees named result
        if err != nil { err = fmt.Errorf("cfg: %w", err) }
    }()
    return parse(f)
}

for i := range 3 { defer fmt.Print(i) } // prints 210 (LIFO)

x := 1
defer fmt.Println("x =", x)  // x = 1 (args evaluated now)
x = 2

for _, name := range files {         // defer in a loop: wrap
    func() { f := open(name); defer f.Close(); use(f) }()
}
```

CODE EXPLAINED

- → `defer f.Close()` — after the error check, never before
- → `defer mu.Unlock()` — unlock even on early return/panic
- → `defer func() { … }()` — closure sees the final `err`
- → `defer fmt.Println("x =", x)` — captures `x` = 1 immediately
- → Wrap loop bodies in a func so each defer runs per iteration

COMING FROM…

- **Python** — `defer` ≈ `with` / `finally` for cleanup
- **Java** — ≈ try-with-resources, but any call works
- **C#** — ≈ `using` / `finally`

COMMON USES

RESOURCE	DEFER
file / conn	`defer f.Close()`
mutex	`defer mu.Unlock()`
HTTP response	`defer resp.Body.Close()`
timer / ticker	`defer t.Stop()`
context	`defer cancel()`
recover	`defer func(){ recover() }()`

EVALUATION TIMING

PART	WHEN
function value	at defer
arguments	at defer
call	at function return
closure body vars	at return (latest)

⚠ COMMON MISTAKES

- `defer f.Close()` before checking `err`; `f` is nil and `Close` panics
- Deferring inside a long loop; files stay open until the function ends
- Ignoring the `Close` error on files you wrote; write errors may surface there

✓ TIPS & BEST PRACTICES

- Pair acquire + `defer` release on adjacent lines so they're reviewed together
- For writes, capture the close error: `defer func() { err = errors.Join(err, f.Close()) }()`
- Use a deferred closure to add context to every returned error

QUICK SUMMARY

- `defer call()` — runs at function return
- LIFO — last deferred runs first
- args — evaluated immediately
- closures — can modify named results

Errors as Values

ERRORS ARE VALUES

Go has no exceptions for ordinary failures. A function that can fail returns an `error` as its **last result**, and the caller checks it immediately with `if err != nil`. `error` is a one-method interface, so any type with `Error() string` is an error. A `nil` error means success.

CONVENTIONS

- Error strings are **lowercase**, no trailing punctuation
- On error, other results are usually zero values; don't use them
- Return errors up the stack, adding context as you go
- Handle each error **once**: log it or return it, not both

RETURNING & CHECKING

```
type error interface { Error() string } // built in

func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}

q, err := divide(10, 0)
if err != nil {
    fmt.Println("error:", err)    // error: division by zero
    return
}
fmt.Println(q)

func loadUser(id int) (*User, error) {
    row, err := db.Query(id)
    if err != nil {
        return nil, fmt.Errorf("load user %d: %w", id, err)
    }
    return parseUser(row)         // forward both results
}
```

CODE EXPLAINED

- → `error` — built-in interface with one method
- → `(T, error)` — error is always the last result
- → `errors.New` — a fixed message
- → `fmt.Errorf("…: %w", err)` — add context, keep the cause
- → `return parseUser(row)` — forward both results

COMING FROM…

- **Python / Java** — no `try` / `catch`; errors are return values
- **JS** — no rejected promises; check `err` after every call
- **Rust** — like `Result<T, E>`, without the `?` operator

CREATING ERRORS

WAY	USE
`errors.New("msg")`	fixed message / sentinel
`fmt.Errorf("…%v", x)`	formatted message
`fmt.Errorf("…: %w", err)`	wrap a cause
`errors.Join(e1, e2)`	combine several
`&MyErr{…}`	custom type with fields

STDLIB ERRORS

ERROR	MEANING
`io.EOF`	end of input (not a failure)
`fs.ErrNotExist`	missing file
`context.Canceled`	ctx was canceled
`sql.ErrNoRows`	query found nothing
`http.ErrServerClosed`	normal shutdown

⚠ COMMON MISTAKES

- `result, _ := f()` dropping errors; failures turn into silent wrong data
- Capitalized or punctuated messages: `"Failed."` reads badly when wrapped (`load: Failed.`)
- Logging an error and also returning it, so it's reported several times

✓ TIPS & BEST PRACTICES

- Use the `errcheck` linter (in golangci-lint) to find ignored errors
- Wrap with the operation, not "error": `fmt.Errorf("open config: %w", err)`
- At the top (main/handler), log once with full context

QUICK SUMMARY

- `error` — interface with `Error() string`
- `(T, error)` — error last
- `if err != nil` — check immediately
- `nil` — success

Wrapping & Inspecting Errors

WRAPPING & INSPECTING

`fmt.Errorf` with the `%w` verb **wraps** an error: the message gains context while the original stays reachable. `errors.Is` walks the chain looking for a specific **value** (sentinel), and `errors.As` looks for a specific **type**. Go 1.26 added the generic `errors.AsType[T]`, which returns the match directly.

NOTES

- `%v` formats the cause but **does not** wrap it
- Multiple `%w` verbs in one `Errorf` wrap several errors (1.20)
- `errors.Join` combines errors; `Is` / `As` search all of them
- Never compare wrapped errors with `==`; use `errors.Is`

IS, AS & ASTYPE

```
_, err := os.Open("missing.txt")
err = fmt.Errorf("load config: %w", err)
fmt.Println(err) // load config: open missing.txt: no such…

errors.Is(err, fs.ErrNotExist)           // true
err == fs.ErrNotExist                    // false (wrapped)

var pe *fs.PathError
if errors.As(err, &pe) {                 // find by type
    fmt.Println(pe.Path)                 // missing.txt
}
if pe, ok := errors.AsType[*fs.PathError](err); ok { // 1.26
    fmt.Println(pe.Op)                   // open
}

inner := errors.Unwrap(err)              // one level down

e := errors.Join(errA, errB)             // both kept
errors.Is(e, errB)                       // true
w := fmt.Errorf("both: %w, %w", errA, errB) // multi-wrap
```

CODE EXPLAINED

- `%w` — wrap; the chain is searchable
- `errors.Is(err, target)` — any error in the chain `==` target?
- `errors.As(err, &ptr)` — first error assignable to `ptr`'s type
- `errors.AsType[T]` — generic form, no pre-declared var (1.26)
- `errors.Join` — tree of errors; message joins with newlines

COMING FROM…

- **Java** — `%w` wrapping ≈ exception `cause`
- **Python** — `raise X from err` ≈ `fmt.Errorf("…: %w", err)`
- **Python** — `except ValueError` ≈ `errors.As` / `AsType`

ERRORS PACKAGE

FUNC	PURPOSE
`Is(err, target)`	match a sentinel value
`As(err, &target)`	match a type, fill target
`AsType[T](err)`	generic As → (T, bool)
`Unwrap(err)`	one level down
`Join(errs...)`	combine (nil-safe)
`New(msg)`	simple error

%W VS %V

VERB	EFFECT
`%w`	wraps: Is/As can see cause
`%v`	text only: cause hidden
`%w twice`	wraps both (1.20+)
`vet`	flags %w of *E pointer (1.27)

⚠ COMMON MISTAKES

- Wrapping with `%v` and then expecting `errors.Is` to find the cause
- `errors.As(err, pe)` without `&`: As needs a pointer to the target variable
- Wrapping internal errors into public APIs, making them part of your contract

✓ TIPS & BEST PRACTICES

- Prefer `errors.AsType` on Go 1.26+; it reads better than declare-then-As
- Use `%v` deliberately at API boundaries to hide implementation details
- In tests, assert with `errors.Is` rather than comparing message strings

QUICK SUMMARY

- `%w` — wrap with context
- `errors.Is` — match by value
- `errors.As` / `AsType` — match by type
- `errors.Join` — combine errors

Custom & Sentinel Errors

SENTINELS & CUSTOM TYPES

A **sentinel error** is a package-level variable such as `io.EOF` that callers compare against with `errors.Is`. A **custom error type** is a struct implementing `Error()` that carries extra fields (status codes, paths) for callers to read with `errors.As`. Add an `Unwrap` method if your type wraps another error.

GUIDELINES

- Name sentinels `ErrXxx`; name types `XxxError`
- Return custom types as `*T` (pointer receiver) consistently
- An `Is(target error) bool` method customizes matching
- Exported sentinels and types become part of your API

DEFINING ERRORS

```go
var ErrNotFound = errors.New("not found")   // sentinel

type ValidationError struct {               // custom type
    Field string
    Err   error
}
func (e *ValidationError) Error() string {
    return e.Field + ": " + e.Err.Error()
}
func (e *ValidationError) Unwrap() error { return e.Err }

func find(id int) (*Item, error) {
    if id < 0 {
        return nil, &ValidationError{"id", ErrNotFound}
    }
    return nil, ErrNotFound
}

_, err := find(-1)
errors.Is(err, ErrNotFound)             // true (via Unwrap)
if ve, ok := errors.AsType[*ValidationError](err); ok {
    fmt.Println(ve.Field)                   // id
}
```

CODE EXPLAINED

→ `var ErrNotFound` — compare with `errors.Is`

→ `*ValidationError` — carries structured data

→ `Unwrap()` — makes the inner error visible to `Is` / `As`

→ `&ValidationError{…}` — return the pointer, matching the receiver

→ `AsType[*ValidationError]` — extract the fields

COMING FROM…

- **Java** — custom error types replace exception subclasses
- **Python** — sentinel `ErrX` ≈ a module-level exception value
- **Rust** — custom error structs, matched with `errors.As`

SENTINEL VS TYPE

ASPECT	SENTINEL / TYPE
Check with	`errors.Is` / `errors.As`
Extra data	none / fields
Example	`io.EOF` / `*fs.PathError`
Coupling	low / higher

THE TYPED-NIL TRAP

CODE	RESULT
`var p *MyErr = nil`	nil pointer
`var err error = p`	err != nil !
`return p (as error)`	caller sees non-nil
`fix`	`return nil` literally

⚠ COMMON MISTAKES

- Returning a nil `*MyErr` as `error`: the interface holds a type, so `err != nil` is true
- Defining `Error()` on the value receiver but returning `&T{}` (or vice versa) inconsistently
- Matching errors by `err.Error() == "not found"`; messages change, use `errors.Is`

✓ TIPS & BEST PRACTICES

- Keep sentinels for conditions callers act on (`ErrNotFound`), not every failure
- Expose behavior via small interfaces (`interface{ Timeout() bool }`) instead of types
- Always end functions with `return nil` on success, never a typed nil variable

QUICK SUMMARY

- `var ErrX = errors.New(…)` — sentinel
- `type XError struct` + `Error()` — custom type
- `Unwrap()` — expose the cause
- typed nil — never return a nil `*T` as `error`

panic & recover

PANIC & RECOVER

`panic` stops normal execution, runs deferred calls up the stack, and crashes the program with a stack trace. It's for **programmer errors and impossible states**, not expected failures. `recover()` called **inside a deferred function** stops the panic and returns its value, letting a server keep running after one bad request.

NOTES

- `recover()` returns `nil` when not panicking or not called from a defer
- A panic in another goroutine can't be recovered from `main`
- Runtime panics: nil map write, index out of range, nil deref, bad assertion
- `Must…` helpers (`regexp.MustCompile`) panic for init-time failures

PANIC AND RECOVERY

```
func mustPositive(n int) int {
    if n <= 0 { panic(fmt.Sprintf("bad n: %d", n)) }
    return n
}

func safeCall(f func()) (err error) {
    defer func() {
        if r := recover(); r != nil {      // stop the panic
            err = fmt.Errorf("recovered: %v", r)
        }
    }()
    f()
    return nil
}

err := safeCall(func() { var m map[string]int; m["x"] = 1 })
fmt.Println(err) // recovered: assignment to entry in nil map

var re = regexp.MustCompile(`^\d+$`) // panics at init if bad
```

CODE EXPLAINED

- → `panic(v)` — any value; usually a string or error
- → `recover()` — only works directly inside a deferred func
- → named result `err` — the deferred func converts panic → error
- → Runtime errors are panics with a `runtime.Error` value
- → `MustCompile` — fail fast on a constant that must be valid

COMING FROM…

- **Java / Python** — `panic` ≈ unchecked exception; rarely caught
- **Python** — `recover` ≈ a top-level `except` in a deferred func
- **Rust** — `panic!` ≈ Go `panic`

COMMON RUNTIME PANICS

CAUSE	MESSAGE
`nil map write`	assignment to entry in nil map
`s[10] on len 3`	index out of range [10]
`nil pointer`	invalid memory address
`x.(T) wrong type`	interface conversion
`close closed chan`	close of closed channel

PANIC VS ERROR

SITUATION	USE
`File not found`	error
`Bad user input`	error
`Broken invariant`	panic
`Init-time config`	Must… panic
`Library API`	error (never panic)

⚠ COMMON MISTAKES

- Using `panic` / `recover` as try/catch for normal control flow
- Calling `recover()` in a helper called by the deferred func; it must be the deferred func itself
- Starting goroutines without recovery in servers; one panic kills the whole process

✓ TIPS & BEST PRACTICES

- Recover at boundaries only: HTTP middleware, worker loops (`net/http` already recovers per request)
- Log the stack with `debug.Stack()` when you recover
- Re-panic (`panic(r)`) values you don't know how to handle

QUICK SUMMARY

- `panic(v)` — unwinds, runs defers, crashes
- `recover()` — only in a deferred func
- errors for expected failures; panic for bugs
- `Must…` — panic on invalid constants

Error Handling Patterns

add context at each layer · handle once (log or return) · errors at API boundaries · errgroup · error from deferred Close · opaque vs inspectable errors · slog with an err attribute

HANDLING ERRORS WELL

Good Go error handling adds **context at each layer**, handles each error **exactly once**, and decides at API boundaries what callers may inspect. Concurrent work collects errors with `errgroup` or `errors.Join`, and errors from deferred `Close` calls on written files must not be lost.

PRINCIPLES

- Wrap: `fmt.Errorf("verb object: %w", err)` at each layer
- Decide once: return it **or** log it
- Opaque by default; expose sentinels/types deliberately
- Treat `io.EOF` and `context.Canceled` as normal outcomes

PATTERNS

```
func (s *Store) Save(u User) (err error) {
    f, err := os.Create(s.path(u.ID))
    if err != nil {
        return fmt.Errorf("save user %d: %w", u.ID, err)
    }
    defer func() { err = errors.Join(err, f.Close()) }()
    return json.NewEncoder(f).Encode(u)
}

g, ctx := errgroup.WithContext(ctx)      // x/sync/errgroup
for _, url := range urls {
    g.Go(func() error { return fetch(ctx, url) })
}
if err := g.Wait(); err != nil {         // first error wins
    slog.Error("fetch failed", "err", err)
}

var errs []error                         // collect all
for _, r := range rows {
    if e := check(r); e != nil { errs = append(errs, e) }
}
return errors.Join(errs...)              // nil if none
```

CODE EXPLAINED

→ `errors.Join(err, f.Close())` — keep both write and close errors

→ `errgroup.WithContext` — cancels siblings on first error

→ `g.Wait()` — returns the first non-nil error

→ `slog.Error(…, "err", err)` — log once at the top

→ `errors.Join(errs...)` — validation reports every problem

COMING FROM…

- **Java** — no stack traces by default; add context when wrapping
- **Python** — `except: log; raise` is the "handle twice" anti-pattern
- **JS** — `Promise.allSettled` ≈ `errors.Join` of results

WHERE TO HANDLE

LAYER	ACTION
`low-level func`	return with context
`service`	wrap / translate
`HTTP handler`	map to status + log
`main`	log + `os.Exit(1)`
`goroutine`	send on chan / errgroup

EXPECTED "ERRORS"

VALUE	TREAT AS
`io.EOF`	end of stream
`sql.ErrNoRows`	not found (404)
`context.Canceled`	client went away
`http.ErrServerClosed`	clean shutdown

⚠ COMMON MISTAKES

- Logging at every layer, producing five log lines for one failure
- Swallowing errors in goroutines (`go f()` where `f` returns an error)
- Returning raw driver errors from a public package, leaking implementation details

✓ TIPS & BEST PRACTICES

- Translate errors at boundaries: DB `ErrNoRows` → your `ErrNotFound`
- Use `errgroup.SetLimit(n)` to bound concurrency while collecting errors
- Attach errors to logs as attributes: `slog.Any("err", err)`

QUICK SUMMARY

- wrap with context — each layer
- handle once — log **or** return
- `errgroup` — concurrent errors
- `errors.Join` — keep close/validation errors

Sheet 038

Defining & Calling Functions

func name(a, b int) int · shared-type params · pass by value · multiple returns (int, error) · named results · naked return · no overloading / no default args

FUNCTIONS

Functions are declared with `func`, parameters list the name **before** the type, and the result type comes last. Go functions can **return multiple values**, which is how `(result, error)` works. Arguments are always **passed by value** (copied); pass a pointer when the callee must modify the caller's data.

RULES

- Consecutive params of one type share it: `func f(a, b int)`
- No overloading and no default or named arguments
- Named results are zero-initialized and documented in signatures
- A function with results must end in `return` (or `panic`)

DECLARING & CALLING

```
func add(a, b int) int { return a + b }

func divmod(a, b int) (int, int) {      // multiple results
    return a / b, a % b
}

func split(sum int) (x, y int) {        // named results
    x = sum * 4 / 9
    y = sum - x
    return                              // naked return: x, y
}

func rename(u *User, name string) { u.Name = name } // mutate

sum := add(2, 3)                        // 5
q, r := divmod(17, 5)                   // 3 2
_, r = divmod(9, 4)                     // ignore quotient
x, y := split(17)                       // 7 10
rename(&user, "Ann")
fmt.Println(add(divmod(7, 2)))          // 4: results feed args
```

CODE EXPLAINED

- → `(a, b int) int` — shared param type, single result
- → `(int, int)` — two unnamed results
- → `(x, y int)` — named results act as local variables
- → naked `return` — returns the named results
- → `add(divmod(7, 2))` — multi-result call passed straight through

COMING FROM…

- **Python** — no default/keyword args; use options structs
- **Java** — no overloading; different names instead
- **Python / JS** — multiple returns, not tuples/arrays

SIGNATURE FORMS

FORM	EXAMPLE
no result	`func log(msg string)`
one result	`func len() int`
multiple	`func f() (int, error)`
named	`func f() (n int, err error)`
variadic	`func f(xs ...int)`

MISSING FEATURES

OTHER LANGUAGES	GO INSTEAD
overloading	different names: `ParseInt`, `ParseFloat`
default args	options struct / functional options
keyword args	struct literal with field names
exceptions	error result

⚠ COMMON MISTAKES

- Naked returns in long functions; readers can't see what's returned
- Passing a large struct by value then mutating it, expecting the caller to see changes
- Returning many unnamed results of the same type: `(int, int, int)` is easy to misorder

✓ TIPS & BEST PRACTICES

- Name results when they document meaning: `(width, height int)`
- Return a struct once you have more than three results
- Accept an options struct instead of long parameter lists

QUICK SUMMARY

- `func name(p T) R` — declaration
- multiple results — `(T, error)`
- named results + naked `return`
- pass by value — use pointers to mutate

Variadic Functions

func f(nums ...int) · nums is []int · spread f(s...) · ...any like fmt.Println · must be the last param · aliasing the caller's slice · variadic vs slice param

VARIADIC FUNCTIONS

A final parameter written `...T` accepts **zero or more** arguments of type T, which arrive in the function as a `[]T`. Callers can pass individual values or **spread** an existing slice with `s...`. When you spread, the function receives the caller's slice itself, not a copy.

NOTES

- Only the **last** parameter can be variadic
- With no arguments the parameter is a `nil` slice
- `...any` accepts mixed types (like `fmt.Println`)
- You can't mix loose args and a spread: `f(1, s...)` is invalid

VARIADIC EXAMPLES

```
func sum(nums ...int) int {
    total := 0
    for _, n := range nums { total += n }
    return total
}

sum()                       // 0 (nums is nil)
sum(1, 2, 3)                // 6
xs := []int{4, 5}
sum(xs...)                  // 9: spread a slice

func logf(level string, format string, args ...any) {
    fmt.Printf("["+level+"] "+format+"\n", args...) // forward
}
logf("INFO", "%s has %d items", "cart", 3)

all := append([]int{1}, xs...)   // append is variadic too

func zero(s ...int) { s[0] = 0 }
zero(xs...)                 // xs is now [0 5]: same array!
```

CODE EXPLAINED

- → `nums ...int` — `nums` is `[]int` inside the function
- → `sum()` — valid; loop over a nil slice does nothing
- → `xs...` — pass the slice as the variadic param
- → `args...` — forward variadic args to another variadic func
- → `zero(xs...)` — spread slices are shared, not copied

COMING FROM…

- **Python** — `...int` ≈ `*args`; `f(s...)` ≈ `f(*s)`
- **JS** — `...rest` and spread, same idea
- **Java** — `int... nums` is nearly identical

STDLIB VARIADICS

FUNCTION	SIGNATURE
`fmt.Println`	`(a ...any)`
`append`	`(s []T, elems ...T)`
`errors.Join`	`(errs ...error)`
`slices.Concat`	`(slices ...S)`
`min / max`	`(x T, y ...T)`

CALL FORMS

CALL	NUMS RECEIVES
`f()`	nil
`f(1)`	[]int{1}
`f(1, 2, 3)`	[]int{1, 2, 3}
`f(s...)`	s itself
`f(1, s...)`	compile error

⚠ COMMON MISTAKES

- Forgetting `...` when forwarding: `fmt.Printf(format, args)` prints `[a b]` as one value
- Mutating a variadic slice and surprising a caller who spread their slice
- Using `...any` where a typed `...T` would catch mistakes at compile time

✓ TIPS & BEST PRACTICES

- Require at least one value with `func max(first int, rest ...int)`
- Copy with `slices.Clone(s)` before mutating a variadic argument
- `go vet` checks `...any` printf-style wrappers if the name ends in `f`

QUICK SUMMARY

- `...T` — last param, arrives as `[]T`
- `f(s...)` — spread a slice
- spread — shares the caller's array
- `args...` — forward to another variadic

Function Values & Types

FUNCTIONS AS VALUES

Functions are **first-class values**: you can assign them to variables, pass them as arguments, return them, and store them in maps or struct fields. A **function type** like `func(int) bool` describes a signature. Methods can be used as values too, either bound to a receiver (`t.Method`) or as an expression taking the receiver as its first argument (`T.Method`).

NOTES

- Zero value of a func type is `nil`; calling it panics
- Funcs are only comparable to `nil`
- Anonymous funcs (`func(){}`) can be called immediately
- Defined func types can have methods (`http.HandlerFunc`)

FIRST-CLASS FUNCTIONS

```
type Predicate func(int) bool

func filter(xs []int, keep Predicate) []int {
    var out []int
    for _, x := range xs { if keep(x) { out = append(out, x) } }
    return out
}

isEven := func(n int) bool { return n%2 == 0 }
filter([]int{1, 2, 3, 4}, isEven)           // [2 4]

ops := map[string]func(a, b int) int{
    "+": func(a, b int) int { return a + b },
    "*": func(a, b int) int { return a * b },
}
ops["*"](3, 4)                              // 12

slices.SortFunc(users, func(a, b User) int {
    return cmp.Compare(a.Age, b.Age)
})
greet := user.Greet         // method value, bound to user
greet()
f := (*User).Greet          // method expression: f(&user)
```

CODE EXPLAINED

- → `type Predicate func(int) bool` — named function type
- → `keep(x)` — call a function received as a value
- → `map[string]func…` — dispatch table
- → `slices.SortFunc` — callback returns -1/0/+1
- → `user.Greet` vs `(*User).Greet` — bound vs unbound

COMING FROM…

- **JS / Python** — first-class functions work the same
- **Java** — no functional interfaces; `func(int) bool` is the type
- **C** — function values, without pointer syntax

FUNCTION TYPE USES

PATTERN	EXAMPLE
callback	`slices.IndexFunc(s, f)`
strategy	`type Hash func([]byte) uint64`
handler	`http.HandlerFunc`
factory	`func newX() func() int`
hook	`OnClose func()` field

METHOD VALUES

FORM	TYPE
`u.Greet`	`func()`
`(*User).Greet`	`func(*User)`
`User.Name`	`func(User) string`
`strings.ToUpper`	`func(string) string`

⚠ COMMON MISTAKES

- Calling an optional hook field without a nil check: `if s.OnClose != nil { s.OnClose() }`
- Comparing two funcs with `==`; only `nil` comparison compiles
- Using `func(…)` params where a one-method interface would be clearer and mockable

✓ TIPS & BEST PRACTICES

- Give a function type a method to make it satisfy an interface (`HandlerFunc` pattern)
- Pass `strings.TrimSpace` etc. directly; no wrapper lambda needed
- Prefer `slices` / `maps` helpers that take funcs over hand-written loops

QUICK SUMMARY

- funcs — values you can pass and return
- `type F func(T) R` — function types
- `func(…) {…}` — anonymous functions
- `t.M` / `T.M` — method value / expression

Closures

capture by reference · counter / generator pattern · loop-variable capture (fixed in 1.22) · closures in goroutines · memoization · sync.OnceValue · deferred closures

CLOSURES

A **closure** is a function literal that **captures variables** from its surrounding scope by reference: it sees and can modify their current values, and they live as long as the closure does. Closures power generators, middleware, deferred cleanups and callbacks. Since **Go 1.22**, loop variables are per-iteration, so capturing them in a loop is safe.

NOTES

- Captured variables are shared, not copied
- Each call to the outer function creates new captured state
- Closures that escape move captured vars to the heap
- `sync.OnceValue(f)` wraps a closure to run it once (1.21)

CAPTURING STATE

```
func counter() func() int {
    n := 0                          // captured
    return func() int { n++; return n }
}
c1, c2 := counter(), counter()
c1(); c1()                          // 1, 2
fmt.Println(c1(), c2())             // 3 1: separate state

x := 1
show := func() { fmt.Println(x) }
x = 2
show()                              // 2: sees the latest value

var funcs []func()
for i := range 3 {
    funcs = append(funcs, func() { fmt.Print(i) })
}
for _, f := range funcs { f() } // 012 (Go 1.22+)

config := sync.OnceValue(func() Config { return load() })
cfg := config()                     // load() runs only once
```

CODE EXPLAINED

- → `n` — lives on after `counter` returns
- → `c1`, `c2` — each has its own `n`
- → `show()` — reads `x` at call time, not creation
- → loop capture — each iteration's `i` is distinct (1.22)
- → `sync.OnceValue` — lazy, thread-safe initialization

COMING FROM…

- **JS** — closures capture variables, like JS
- **Python** — no `nonlocal` needed to modify captured vars
- **Java** — captured vars needn't be effectively final

CLOSURE USES

USE	EXAMPLE
generator	`next := counter()`
middleware	`func(h Handler) Handler`
deferred cleanup	`defer func(){…}()`
goroutine body	`go func(){…}()`
memoization	cache map captured

LOOP CAPTURE BY VERSION

GO.MOD GO	BEHAVIOR
≤ 1.21	one shared `i`: prints 333
≥ 1.22	per-iteration `i`: prints 012
old fix	`i := i` inside loop
vet	`loopclosure` check

⚠ COMMON MISTAKES

- Expecting a closure to snapshot a variable; it reads whatever the variable holds when called
- Goroutines capturing a shared variable they all write: a data race. Pass values as args or lock
- Keeping a closure around that captured a huge slice, preventing it from being freed

✓ TIPS & BEST PRACTICES

- Remove old `i := i` loop copies once your module targets Go 1.22+ (`go fix` can help)
- Use closures for small adapters; switch to a struct with methods when state grows
- Pass values as parameters (`go func(v int){…}(v)`) when intent should be explicit

QUICK SUMMARY

- closure — func literal + captured vars
- capture is by reference
- per-iteration loop vars — Go 1.22
- `sync.OnceValue` — run-once closure

Scope & Shadowing

SCOPE & SHADOWING

Go is **lexically scoped by blocks**: each `{ }`, plus the implicit blocks of `if`, `for` and `switch` statements, starts a new scope. Declaring a name that already exists in an outer scope **shadows** it. The usual source of bugs is `:=` creating a new `err` or variable inside a block when you meant to assign the outer one.

SCOPE LEVELS

- **Universe** — builtins: `int`, `len`, `nil`, `true`, `error` …
- **Package** — top-level names, visible in every file of the package
- **File** — imports are file-scoped
- **Function / block** — params, locals, init statements

SHADOWING BUGS

```
var cfg *Config                    // package level

func setup() error {
    cfg, err := load()             // BUG: new local cfg
    _ = cfg                        // package cfg still nil
    return err
}

func setupFixed() error {
    var err error
    cfg, err = load()              // assigns package cfg
    return err
}

x := 1
if x := 2; x > 1 {                 // shadows outer x
    fmt.Println(x)                 // 2
}
fmt.Println(x)                     // 1

len := 3                           // shadows builtin len!
// n := len("abc")                 // error: len is int
```

CODE EXPLAINED

- → `cfg, err := load()` — declares a new `cfg` in the function
- → `var err error` + `=` — reuse the package-level `cfg`
- → `if x := 2` — `x` lives only inside the `if`
- → `len := 3` — legal, but breaks `len()` in that scope
- → Shadowing isn't an error; the compiler allows it

COMING FROM…

- **Python** — block scope; variables in `if` don't leak out
- **JS** — like `let` / `const`, no hoisting
- **Java** — shadowing is allowed (and a common bug)

VISIBILITY

DECLARED	VISIBLE IN
`builtin`	everywhere
`package-level`	whole package (all files)
`Exported (Caps)`	other packages too
`import`	that file only
`inside block`	that block + nested

FINDING SHADOWS

TOOL	HOW
`vet shadow`	`go run golang.org/x/tools/go/analysis/passes/shadow/cmd/shadow ./...`
`golangci-lint`	`govet: enable: [shadow]`
`gopls`	highlights declarations
`predeclared lint`	flags `len`, `new` shadowing

⚠ COMMON MISTAKES

- `err` shadowed inside `if` / `for`, so the outer `err` checked later is always nil
- Naming a variable `new`, `len`, `copy`, `string` or `error`
- Package-level mutable variables used as hidden globals across files

✓ TIPS & BEST PRACTICES

- Declare `var err error` up front when assigning to existing variables
- Keep scopes tiny with `if x := …; cond` init statements
- Enable the `shadow` and `predeclared` linters in CI

QUICK SUMMARY

- block scope — every `{}` and if/for/switch
- `:=` in a block — may shadow
- imports — file scope
- builtins — can be (but shouldn't be) shadowed

Package Initialization

PACKAGE INITIALIZATION

Before `main` runs, Go initializes every imported package **once**, dependencies first. Within a package, package-level variables are initialized in **dependency order** (not file order), then every `init()` function runs in the order the files are presented to the compiler. A blank import `import _ "pkg"` runs a package's init purely for its side effects.

NOTES

- A file can have **multiple** `init()` functions
- `init` takes no args, returns nothing, and can't be called
- Imported packages finish initializing before the importer
- Initialization runs in a single goroutine

INIT ORDER

```
package app

import (                        // side-effect imports
    _ "image/png"               // registers PNG decoder
    _ "github.com/lib/pq"       // registers "postgres" driver
)

var a = b + 1           // initialized second (needs b)
var b = f()             // initialized first
func f() int { return 41 }

func init() {           // runs after all package vars
    fmt.Println("init 1:", a)   // init 1: 42
}
func init() {           // multiple inits are allowed
    registry["json"] = jsonCodec{}
}
var loadCfg = sync.OnceValue(readConfig) // lazy alternative
```

CODE EXPLAINED

- → `a` depends on `b`, so `b` initializes first
- → `init()` — after every package-level var is set
- → two `init` funcs — run in source order
- → `_ "image/png"` — import only for its `init` registration
- → `sync.OnceValue` — defer expensive setup until first use

COMING FROM…

- **Java** — `init()` ≈ static initializer blocks
- **Python** — ≈ module-level code run on import
- **JS** — side-effect imports ≈ `import 'polyfill'`

FULL ORDER

STEP	WHAT
1	imported packages (recursively)
2	package-level vars, by dependency
3	`init()` funcs, file order
4	`main.main()`

SIDE-EFFECT IMPORTS

IMPORT	REGISTERS
`_ "image/jpeg"`	image.Decode format
`_ "time/tzdata"`	embedded time zones
`_ "embed"`	enables //go:embed string
`_ "modernc.org/sqlite"`	sql driver

⚠ COMMON MISTAKES

- Doing network or file I/O in `init`; failures can't be returned and tests become slow
- Relying on `init` order across files; it depends on file naming
- Mutable global registries filled in `init` that tests can't reset

✓ TIPS & BEST PRACTICES

- Prefer explicit `New…` constructors called from `main` over `init` side effects
- Use `sync.OnceValue` / `OnceFunc` for lazy, error-aware setup
- Keep `init` for cheap registrations and constant tables

QUICK SUMMARY

- deps → package vars → `init()` → `main`
- vars — dependency order
- `init()` — automatic, multiple allowed
- `import _` — side effects only

Sheet 044

Functional Options & Function Patterns

type Option func(Server) · WithTimeout(d) Option · NewServer(addr, opts…) · config-struct alternative · decorator / middleware wrapping · recursion (no tail-call optimization) · callback hooks

FUNCTIONAL OPTIONS

Go has no default or named arguments, so configurable constructors use the **functional options** pattern: each option is a function that modifies a config struct, and the constructor takes them as a variadic list. Callers pass only what they need, and new options can be added without breaking anyone. A plain **config struct** is the simpler alternative.

RELATED PATTERNS

- **Decorator** — wrap a func/handler to add behavior
- **Callback hooks** — func fields called at lifecycle points
- **Recursion** works normally; Go has **no tail-call optimization**
- Deep recursion grows the goroutine stack (up to 1 GB)

OPTIONS & DECORATORS

```
type Server struct{ addr string; timeout time.Duration }
type Option func(*Server)

func WithTimeout(d time.Duration) Option {
    return func(s *Server) { s.timeout = d }
}

func NewServer(addr string, opts ...Option) *Server {
    s := &Server{addr: addr, timeout: 30 * time.Second}
    for _, opt := range opts { opt(s) }   // apply in order
    return s
}
srv := NewServer(":8080", WithTimeout(5*time.Second))

func timed(f func()) func() {             // decorator
    return func() { t := time.Now(); f(); log.Println(time.Since(t)) }
}

func fact(n int) int {                    // recursion
    if n <= 1 { return 1 }
    return n * fact(n-1)
}
```

CODE EXPLAINED

- → `type Option func(*Server)` — each option mutates the config
- → defaults set first, then options override them
- → `opts ...Option` — zero or more options
- → `timed(f)` — returns a wrapped function
- → `fact` — base case first; each call adds a stack frame

COMING FROM…

- **Python** — options ≈ keyword args with defaults
- **Java** — functional options ≈ the builder pattern
- **Python** — decorators ≈ funcs returning wrapped funcs

OPTIONS VS CONFIG STRUCT

ASPECT	OPTIONS / STRUCT
Call site	`New(a, WithX(1))` / `New(Config{X: 1})`
Defaults	in constructor / zero values
Validation	per option / once
Boilerplate	more / less

SEEN IN

LIBRARY	STYLE
`grpc.Dial`	functional options
`zap.New`	functional options
`http.Server`	config struct
`slog.HandlerOptions`	config struct

⚠ COMMON MISTAKES

- Options that can't report errors; return `func(*S) error` if validation is needed
- Deep recursion on untrusted input (e.g. nested JSON) without a depth limit
- Exporting the config fields and also offering options; pick one style

✓ TIPS & BEST PRACTICES

- Start with a config struct; switch to options when defaults get complex
- Name options `WithXxx` so they read well at the call site
- Convert hot recursive functions to loops with an explicit stack

QUICK SUMMARY

- `type Option func(*T)` — option type
- `New(req, opts ...Option)` — constructor
- decorators — funcs returning wrapped funcs
- recursion — fine, but no TCO

Arrays

[5]int (length is part of the type) · [...]int{1, 2, 3} · value semantics (copy on assign) · comparable with == · [3][3]int · arrays as map keys · when to prefer slices

ARRAYS

An array has a **fixed length that is part of its type**: `[3]int` and `[4]int` are different types. Arrays are **values**: assigning or passing one copies every element. Because of that they're rarely used directly; slices (built on arrays) are the everyday collection. Arrays shine as fixed-size keys, buffers and lookup tables.

NOTES

- `[...]T{…}` lets the compiler count the elements
- Arrays of comparable elements support `==`, so they work as map keys
- Out-of-range **constant** indexes are compile errors
- `len(arr)` is a compile-time constant

ARRAY BASICS

```
var a [3]int                    // [0 0 0]
b := [3]string{"x", "y", "z"}
c := [...]int{10, 20, 30, 40}   // len 4, counted by compiler
d := [5]int{1: 7, 3: 9}         // [0 7 0 9 0] indexed literal

a[0] = 5
fmt.Println(len(c), c[len(c)-1]) // 4 40

e := c                          // full copy
e[0] = 99
fmt.Println(c[0], e[0])         // 10 99

fmt.Println(b == [3]string{"x", "y", "z"}) // true

var grid [3][3]int              // 2D array
grid[1][2] = 1

seen := map[[2]int]bool{}       // array as map key
seen[[2]int{0, 1}] = true

s := c[1:3]                     // slice of the array: [20 30]
```

CODE EXPLAINED

- → `[...]int{…}` — length inferred (4)
- → `{1: 7, 3: 9}` — index: value literals; others zero
- → `e := c` — copies all elements (value semantics)
- → `==` — element-wise comparison
- → `c[1:3]` — a slice that shares the array's memory

COMING FROM…

- **Python / JS** — arrays are fixed size; slices are the "list"
- **Java** — arrays are values: assignment copies
- **C** — arrays don't decay to pointers

ARRAY VS SLICE

ASPECT	ARRAY / SLICE
Type	[N]T / []T
Length	fixed, in type / dynamic
Assign/pass	copies / shares
Comparable	yes / no (only nil)
Map key	yes / no

GOOD USES

USE	EXAMPLE
fixed buffer	var buf [512]byte
hash digests	sha256.Sum256 → [32]byte
coordinates	[2]int{x, y}
lookup table	[256]bool for ASCII
composite key	map[[2]string]int

⚠ COMMON MISTAKES

- Passing a large array to a function and wondering why changes don't stick (copy); pass `*[N]T` or a slice
- Mixing `[3]int` and `[]int` in a signature; they're unrelated types
- Ranging `for _, v := range bigArray` copies the whole array first

✓ TIPS & BEST PRACTICES

- Use arrays when the size is part of the meaning (RGB, IPv4, digest)
- Convert slice → array with `[4]byte(s)` (Go 1.20) after a length check
- Use `arr[:]` to pass an array to slice-based APIs

QUICK SUMMARY

- `[N]T` — fixed length in the type
- value semantics — copies on assign
- comparable — map keys
- `arr[:]` — view as a slice

Slices — Basics

SLICES

A slice is a **dynamically sized view** into an array and Go's main list type. It has a **length** (elements you can index) and a **capacity** (room before reallocating). `append` adds elements and returns the possibly new slice, so you must **always assign its result**. `make([]T, len, cap)` pre-allocates.

NOTES

- Zero value is `nil`: `len` 0, `append` and `range` work
- Indexing beyond `len` panics even if `cap` is larger
- `s[low:high]` excludes `high`; omitted bounds default to 0 and `len`
- Slices aren't comparable; use `slices.Equal`

SLICE BASICS

```
s := []int{1, 2, 3}              // literal
var empty []string               // nil slice
buf := make([]byte, 0, 1024)     // len 0, cap 1024
grid := make([]int, 5)           // [0 0 0 0 0]

s = append(s, 4)                 // [1 2 3 4]
s = append(s, 5, 6)              // variadic
s = append(s, []int{7, 8}...)    // append another slice

fmt.Println(len(s), cap(s))      // 8 8 (cap may vary)
fmt.Println(s[2:5])              // [3 4 5]
fmt.Println(s[:2], s[6:])        // [1 2] [7 8]
last := s[len(s)-1]              // 8

for i, v := range s { _ = i + v }

matrix := make([][]int, 3)       // 2D: slice of slices
for i := range matrix { matrix[i] = make([]int, 4) }

fmt.Println(empty == nil, len(empty)) // true 0
```

CODE EXPLAINED

- → `make([]T, len, cap)` — pre-size to avoid growth copies
- → `s = append(s, …)` — always reassign
- → `append(s, other...)` — concatenate slices
- → `s[2:5]` — elements 2, 3, 4
- → 2D — allocate each row separately

COMING FROM…

- **Python** — slice ≈ `list`; `append` ≈ `.append` (reassign!)
- **JS** — slice ≈ Array; `s[a:b]` ≈ `.slice(a, b)` but shared
- **Java** — slice ≈ `ArrayList`

CREATING

FORM	LEN / CAP
`[]int{1, 2}`	2 / 2
`make([]int, 3)`	3 / 3
`make([]int, 0, 10)`	0 / 10
`var s []int`	0 / 0 (nil)
`arr[1:3]`	2 / len(arr)-1

EVERYDAY OPERATIONS

TASK	CODE
push	`s = append(s, x)`
pop	`x, s = s[len(s)-1], s[:len(s)-1]`
first n	`s[:n]`
copy	`slices.Clone(s)`
clear (keep cap)	`s = s[:0]`

⚠ COMMON MISTAKES

- `append(s, x)` without assigning; the new element may be lost
- `make([]int, 10)` then `append`, expecting 1 element; you get 11 (10 zeros + x)
- Returning `[]T{}` vs `nil` inconsistently; JSON encodes them as `[]` vs `null`

✓ TIPS & BEST PRACTICES

- When the final size is known, use `make([]T, 0, n)` + `append`
- Prefer returning `nil` for "no results" unless JSON shape matters
- Use the `slices` package for search, sort and modification (sheet 048)

QUICK SUMMARY

- `[]T` — dynamic view over an array
- `len` / `cap` — size / room
- `s = append(s, x)` — always reassign
- `make([]T, n, c)` — pre-allocate

Slices — Internals & Gotchas

header (ptr, len, cap) · shared backing array · append reallocation & growth · full slice expr s[a:b:c] · copy(dst, src) · slices.Clone · sub-slice memory retention · passing slices to funcs

HOW SLICES WORK

A slice value is a small **header**: a pointer to a backing array, a length and a capacity. Slicing creates a new header over the **same array**, so writes through one slice are visible through the other. `append` writes in place while capacity remains, and only **allocates a new array** when it runs out, after which the slices no longer share.

NOTES

- Header = `{ptr, len, cap}` (24 bytes on 64-bit)
- Growth roughly doubles small slices, ~1.25× for large ones
- `s[a:b:c]` — full slice expression sets cap to `c - a`
- `copy(dst, src)` copies `min(len(dst), len(src))` elements

SHARING & GROWTH

```
base := []int{1, 2, 3, 4, 5}
a := base[1:3]                  // [2 3], cap 4
a[0] = 99                       // base is now [1 99 3 4 5]

a = append(a, 100)              // fits in cap: overwrites base[3]!
fmt.Println(base)               // [1 99 3 100 5]

b := base[1:3:3]                // full slice expr: cap 2
b = append(b, 7)                // exceeds cap → new array
b[0] = -1                       // base unaffected

dst := make([]int, 2)
n := copy(dst, base)            // 2, dst = [1 99]
c := slices.Clone(base)         // independent copy

big := loadMB()                 // 1 MB
head := big[:10]                // keeps the whole MB alive
head = slices.Clone(big[:10]) // keeps only 10 elements

func addOne(s []int) { s = append(s, 1) } // caller won't see it
```

CODE EXPLAINED

- → `a := base[1:3]` — same backing array
- → `append` within cap — overwrites `base[3]`
- → `[1:3:3]` — limit cap so append must copy
- → `slices.Clone` — break the sharing
- → `addOne` — append changes the local header only

COMING FROM…

- **Python** — `s[1:3]` copies in Python; in Go it shares memory
- **JS** — `.slice()` copies; Go needs `slices.Clone`
- **Rust** — slice ≈ `&mut [T]` view

SLICE EXPRESSIONS

EXPR	LEN / CAP
`s[i:j]`	j-i / cap-i
`s[i:j:k]`	j-i / k-i
`s[:0]`	0 / cap (reuse)
`s[len(s):]`	0 / cap-len

COPYING

WAY	NOTE
`copy(dst, src)`	dst must be sized
`slices.Clone(s)`	nil stays nil
`append([]T(nil), s...)`	older idiom
`slices.Concat(a, b)`	new combined slice

⚠ COMMON MISTAKES

- Appending to a sub-slice and corrupting the parent's later elements
- Expecting `func f(s []int) { s = append(s, x) }` to grow the caller's slice; return it instead
- Holding a tiny sub-slice of a huge buffer; the whole buffer can't be GC'd

✓ TIPS & BEST PRACTICES

- Return the slice from functions that append: `s = add(s, x)`
- Use `s[a:b:b]` when handing out sub-slices that others may append to
- Reuse buffers with `buf = buf[:0]` in hot loops

QUICK SUMMARY

- header — ptr, len, cap
- sub-slices — share memory
- append past cap — new array
- `slices.Clone` / `copy` — independent data

slices Package

THE SLICES PACKAGE

`slices` (Go 1.21) provides generic helpers for any slice type: searching, inserting, deleting, comparing and more. Functions that change length **return the new slice**, which you must assign. Since Go 1.22, `Delete` and `Compact` also **zero the removed tail** so the GC can free referenced objects.

NOTES

- `…Func` variants take a callback instead of `==`
- `Insert`, `Delete`, `Compact`, `Grow` return the updated slice
- Sorting and binary search are on sheet 052
- Iterator helpers: `All`, `Values`, `Backward`, `Collect`, `Chunk` (1.23)

COMMON HELPERS

```
s := []int{3, 1, 4, 1, 5, 9}
slices.Contains(s, 4)                    // true
slices.Index(s, 1)                       // 1
slices.IndexFunc(s, func(n int) bool { return n > 4 }) // 4
slices.Max(s), slices.Min(s)             // 9 1

s = slices.Insert(s, 1, 7, 8)            // [3 7 8 1 4 1 5 9]
s = slices.Delete(s, 1, 3)               // [3 1 4 1 5 9]
s = slices.DeleteFunc(s, func(n int) bool { return n == 1 }) // [3 4 5 9]
s = slices.Replace(s, 0, 1, 30)          // [30 4 5 9]

slices.Reverse(s)                        // [9 5 4 30] in place
slices.Equal(s, []int{9, 5, 4, 30})      // true
d := slices.Compact([]int{1, 1, 2, 2, 1}) // [1 2 1]
all := slices.Concat([]int{1}, []int{2, 3}) // [1 2 3]
slices.Repeat([]string{"a"}, 3)          // [a a a]

for c := range slices.Chunk(all, 2) { fmt.Print(c) } // [1 2][3]
evens := slices.Collect(filterSeq(slices.Values(all))) // iter → slice
```

CODE EXPLAINED

- → `IndexFunc` / `ContainsFunc` — predicate versions
- → `Insert(s, i, v…)` — shift and insert at index `i`
- → `Delete(s, i, j)` — remove `s[i:j]`
- → `Compact` — collapse **consecutive** duplicates
- → `Chunk` / `Values` / `Collect` — iterator bridges

COMING FROM…

- **Python** — `x in s` → `slices.Contains`; `s.insert` → `Insert`
- **JS** — `indexOf` → `slices.Index`; `reverse` → `Reverse`
- **Java** — ≈ `Collections` / `List` helpers

QUERY

FUNC	RETURNS
`Contains / ContainsFunc`	bool
`Index / IndexFunc`	int or -1
`Max / Min (+Func)`	element (panics if empty)
`Equal / EqualFunc`	bool
`Compare`	-1 / 0 / +1

MODIFY

FUNC	EFFECT
`Insert(s, i, v…)`	insert at i
`Delete(s, i, j)`	remove range
`DeleteFunc(s, f)`	remove matches
`Compact(s)`	dedupe neighbors
`Grow(s, n) / Clip(s)`	add / trim capacity

⚠ COMMON MISTAKES

- `slices.Delete(s, i, i+1)` without reassigning `s`; the length doesn't change
- Using `Compact` to remove all duplicates; sort first (`slices.Sort` + `Compact`)
- `slices.Max` on an empty slice panics; check `len` first

✓ TIPS & BEST PRACTICES

- Dedupe: `slices.Sort(s); s = slices.Compact(s)`
- Use `slices.ContainsFunc` instead of hand-written `found` loops
- Use `slices.Grow(s, n)` before a known number of appends

QUICK SUMMARY

- `Contains` / `Index` — search
- `Insert` / `Delete` — reassign result
- `Compact` / `Reverse` / `Equal` — utilities
- `Collect` / `Values` / `Chunk` — iterators

Maps — Basics

map[string]int{} · make(map[K]V, hint) · m[k] = v · comma-ok v, ok := m[k] · delete(m, k) · clear(m) · random iteration order · writing to a nil map panics

MAPS

A map is a built-in hash table from keys to values: `map[K]V`. Keys must be **comparable** (no slices, maps or funcs). Reading a missing key returns the **zero value**, so use the **comma-ok** form to tell "missing" from "zero". Maps are reference-like: passing one to a function shares the same data.

NOTES

- Zero value `nil` map: reads OK, writes **panic**
- Iteration order is randomized every run
- `clear(m)` empties a map (Go 1.21)
- Maps aren't safe for concurrent writes; you'll get a fatal error

MAP BASICS

```
ages := map[string]int{"ann": 31, "bo": 25}  // literal
scores := make(map[string]int, 100)          // size hint

ages["cy"] = 40                  // insert / update
fmt.Println(ages["ann"])         // 31
fmt.Println(ages["zed"])         // 0 (missing → zero value)

age, ok := ages["zed"]           // comma-ok: 0 false

delete(ages, "bo")               // no-op if missing
fmt.Println(len(ages))           // 2

for name, age := range ages {    // random order
    fmt.Println(name, age)
}

counts := map[string]int{}
for _, w := range words { counts[w]++ }  // zero value helps

clear(ages)                      // empty, keeps allocation
var m map[string]int             // nil
// m["x"] = 1                    // panic: assignment to nil map
```

CODE EXPLAINED

- → `make(map[K]V, n)` — hint avoids rehashing
- → missing key — returns `0`, `""`, `nil` …
- → `v, ok := m[k]` — `ok` is false if absent
- → `counts[w]++` — works because missing reads as 0
- → `clear(m)` — remove all entries (1.21)

COMING FROM…

- **Python** — map ≈ `dict`; missing key returns zero, no `KeyError`
- **JS** — map ≈ `Map` /object; `v, ok := m[k]` ≈ `has` + `get`
- **Java** — ≈ `HashMap`; iteration order is random

OPERATIONS

OP	SYNTAX
create	`make(map[K]V)` / `map[K]V{}`
set	`m[k] = v`
get	`v := m[k]`
exists	`_, ok := m[k]`
remove	`delete(m, k)`

VALID KEY TYPES

KEY	OK?
`string, int, bool`	✔
`struct (comparable)`	✔
`[N]T array`	✔
`pointer, chan`	✔ (identity)
`[]T, map, func`	✖

⚠ COMMON MISTAKES

- Declaring `var m map[K]V` and writing to it without `make`
- Treating a zero value as "present"; use comma-ok when zero is a valid value
- Writing from several goroutines without a mutex: `fatal error: concurrent map writes`

✓ TIPS & BEST PRACTICES

- Use struct keys for composite lookups: `map[Point]bool`
- Use `map[K][]V` + `append` for grouping; missing keys start as nil slices
- Protect shared maps with `sync.Mutex` / `RWMutex` (or `sync.Map` for special cases)

QUICK SUMMARY

- `map[K]V` — hash table, comparable keys
- `v, ok := m[k]` — presence check
- `delete` / `clear` — remove
- nil map — write panics; random order

Maps — Sets & Patterns

MAP PATTERNS & SETS

The `maps` package adds cloning, comparison and **iterators** like `maps.Keys`. Go has no set type; the idiom is `map[T]struct{}` or `map[T]bool`. Iteration is random, so sort keys for stable output with `slices.Sorted(maps.Keys(m))`.

NOTES

- `maps.Keys` / `maps.Values` return **iterators**, not slices (1.23)
- `slices.Collect(maps.Keys(m))` builds a slice
- `struct{}{}` uses no memory; `bool` reads more naturally
- `sync.Map` suits write-once/read-many caches across goroutines

PATTERNS

```
m := map[string]int{"b": 2, "a": 1, "c": 3}
for _, k := range slices.Sorted(maps.Keys(m)) {
    fmt.Print(k, m[k], " ")              // a1 b2 c3
}
vals := slices.Collect(maps.Values(m))   // [2 1 3] any order

seen := map[string]struct{}{}            // set
seen["go"] = struct{}{}
_, has := seen["go"]                     // true

byLen := map[int][]string{}              // grouping
for _, w := range []string{"a", "bb", "cc"} {
    byLen[len(w)] = append(byLen[len(w)], w)
}                                        // {1:[a] 2:[bb cc]}

type key struct{ x, y int }              // composite key
grid := map[key]rune{{0, 0}: '#'}

c := maps.Clone(m)                       // shallow copy
maps.DeleteFunc(c, func(k string, v int) bool { return v > 1 })
maps.Copy(c, map[string]int{"z": 26})    // merge into c
```

CODE EXPLAINED

- → `slices.Sorted(maps.Keys(m))` — deterministic order
- → `map[T]struct{}` — set with zero-size values
- → `append(byLen[n], w)` — nil slice for new keys works
- → `map[key]rune{{0, 0}: '#'}` — struct key literal
- → `Clone` / `Equal` / `DeleteFunc` / `Copy` — helpers

COMING FROM…

- **Python** — no `set`; use `map[T]struct{}`
- **Python** — `defaultdict(list)` ≈ `append(m[k], v)`
- **Java** — `HashSet` ≈ `map[T]bool`

MAPS PACKAGE

FUNC	DOES
`Keys / Values`	iterators (1.23)
`All`	iter.Seq2[K, V]
`Collect(seq)`	map from iterator
`Clone / Copy`	copy / merge
`DeleteFunc`	remove matches

SET OPERATIONS

OP	CODE
`add`	`s[x] = struct{}{}`
`has`	`_, ok := s[x]`
`remove`	`delete(s, x)`
`union`	`maps.Copy(a, b)`
`size`	`len(s)`

⚠ COMMON MISTAKES

- Expecting `maps.Keys` to return `[]K`; wrap it in `slices.Collect` or `slices.Sorted`
- Assuming `maps.Clone` deep-copies; nested slices/maps are shared
- Using `sync.Map` as a general concurrent map; a mutex + map is usually faster and typed

✓ TIPS & BEST PRACTICES

- Sort keys before printing or hashing maps to get stable output and tests
- Use `map[T]bool` when `if set[x]` readability beats the tiny memory savings
- Consider `golang.org/x/exp/maps`-free code: the stdlib now covers most needs

QUICK SUMMARY

- `maps.Keys/Values/All` — iterators
- `slices.Sorted(maps.Keys(m))` — ordered keys
- `map[T]struct{}` — set
- `map[K][]V` — grouping

Structs

type User struct { … } · named-field literals · anonymous structs · nested structs · field tags json:"name" · == comparison · &User{} · field-selector keys in literals (1.27)

STRUCTS

A struct groups named fields into one type and is Go's main way to model data. Struct literals should use **field names** so adding fields later doesn't break code; omitted fields get zero values. Structs are values (copied on assign) and comparable if all fields are. **Tags** attach metadata used by encoders like `json`.

NOTES

- Exported fields start with a capital; only they are seen by `json`, etc.
- `&T{…}` creates a pointer directly
- Anonymous structs are handy for one-off data and tests
- Go 1.27: literal keys can be field selectors of embedded fields

DEFINING & USING STRUCTS

```
type Address struct{ City, Zip string }

type User struct {
    ID      int       `json:"id"`
    Name    string    `json:"name"`
    Email   string    `json:"email,omitempty"`
    Address           // embedded (sheet 056)
    tags    []string  // unexported
}

u := User{ID: 1, Name: "Ann"}       // named fields; rest zero
p := &User{Name: "Bo"}              // pointer
p.Name = "Bob"                      // auto-deref
u2 := u                             // copy
u2.Name = "Cy"                      // u unchanged

pt := struct{ X, Y int }{1, 2}      // anonymous struct
fmt.Printf("%+v\n", pt)             // {X:1 Y:2}

tests := []struct{ in, want string }{ {"a", "A"}, {"b", "B"} }
```

CODE EXPLAINED

- → `` `json:"id"` `` — struct tag read via reflection
- → `Address` — embedded field; its fields are promoted
- → `User{ID: 1, Name: "Ann"}` — keyed literal
- → `u2 := u` — independent copy (slices inside still shared)
- → `[]struct{…}` — table-driven test data

COMING FROM…

- **Java** — no classes; struct + methods, no constructors
- **Python** — ≈ `@dataclass`; fields are fixed
- **C** — struct tags add metadata (JSON names)

LITERAL FORMS

FORM	NOTES
`T{A: 1, B: 2}`	keyed (preferred)
`T{1, 2}`	positional; all fields
`T{}`	all zero
`&T{…}`	pointer to new value
`struct{…}{…}`	anonymous type

COMMON TAGS

TAG	USED BY
`json:"name,omitempty"`	encoding/json
`xml:"name,attr"`	encoding/xml
`db:"user_id"`	sqlx
`yaml:"name"`	yaml libs
`validate:"required"`	validator

⚠ COMMON MISTAKES

- Positional literals for structs from other packages; they break when fields are added (vet warns)
- Lowercase fields that should be serialized; `json` silently ignores them
- Typos in tags (`json:name`): no compile error. `go vet` checks tag syntax

✓ TIPS & BEST PRACTICES

- Use `%+v` to print structs with field names while debugging
- Order fields large → small to reduce padding in hot structs (`fieldalignment` linter)
- Use anonymous structs for table-driven tests and one-off JSON shapes

QUICK SUMMARY

- `type T struct{…}` — named fields
- `T{Field: v}` — keyed literal
- tags — metadata for encoders
- values — copied; comparable if fields are

Sorting & Searching

slices.Sort · slices.SortFunc · SortStableFunc · cmp.Compare · multi-key sort with cmp.Or · slices.BinarySearch / BinarySearchFunc · slices.IsSorted · legacy sort.Slice

SORTING & SEARCHING

`slices.Sort` sorts any ordered type in place, and `slices.SortFunc` takes a comparison function returning **negative, zero or positive**, usually built with `cmp.Compare`. Neither is stable; use `SortStableFunc` to keep equal elements in order. `slices.BinarySearch` finds items in sorted slices in O(log n).

NOTES

- `cmp.Compare(a, b)` → -1 / 0 / +1 for ordered types
- `cmp.Or` chains comparisons for multi-key sorts
- Reverse order: swap arguments, `cmp.Compare(b, a)`
- The older `sort` package (`sort.Slice`) still works but is slower

SORT & SEARCH

```
nums := []int{5, 2, 8, 1}
slices.Sort(nums)                          // [1 2 5 8]
i, found := slices.BinarySearch(nums, 5) // 2 true
i, found = slices.BinarySearch(nums, 4)  // 2 false (insert pos)

type P struct{ Name string; Age int }
people := []P{{"Bo", 30}, {"Ann", 25}, {"Cy", 30}}

slices.SortFunc(people, func(a, b P) int {
    return cmp.Compare(a.Age, b.Age)      // by age asc
})
slices.SortStableFunc(people, func(a, b P) int {
    return cmp.Or(
        cmp.Compare(b.Age, a.Age),        // age desc, then
        strings.Compare(a.Name, b.Name),  // name asc
    )
})
j, ok := slices.BinarySearchFunc(people, 25, func(p P, t int) int {
    return cmp.Compare(p.Age, t)
})
slices.IsSorted(nums)                     // true
```

CODE EXPLAINED

- → `BinarySearch` — index + found; index is the insert point
- → `cmp.Compare(a.Age, b.Age)` — ascending
- → `cmp.Or(…)` — first non-zero comparison wins
- → `SortStableFunc` — equal keys keep their order
- → `sort.Slice` — legacy, `less` func on indexes

COMING FROM…

- **Python** — `key=` → `cmp.Compare` func returning -1/0/1
- **Java** — `Comparator` ≈ `slices.SortFunc` callback
- **JS** — `sort((a,b)=>a-b)` ≈ `SortFunc`, but typed

SORT FUNCTIONS

FUNC	NOTES
`slices.Sort(s)`	ordered types, in place
`slices.SortFunc(s, cmp)`	custom compare
`slices.SortStableFunc`	stable
`slices.Sorted(seq)`	iterator → sorted slice
`slices.IsSorted(s)`	check

SEARCH FUNCTIONS

FUNC	REQUIRES
`slices.Index(s, v)`	any slice, O(n)
`slices.BinarySearch`	sorted, O(log n)
`slices.BinarySearchFunc`	sorted by key
`sort.Search(n, f)`	generic predicate search

⚠ COMMON MISTAKES

- Returning `a.Age < b.Age` (bool) from a `SortFunc` comparator; it must return an `int`
- Binary-searching an unsorted slice; results are meaningless
- Subtracting for comparisons (`a - b`) on large ints; overflow flips the sign. Use `cmp.Compare`

✓ TIPS & BEST PRACTICES

- Sort a copy when the caller's order matters: `slices.Sorted(slices.Values(s))`
- Use `strings.Compare` / `cmp.Compare` + `cmp.Or` for readable multi-key sorts
- Keep data sorted on insert with `BinarySearch` + `slices.Insert` for small sets

QUICK SUMMARY

- `slices.Sort` / `SortFunc` — sort in place
- `cmp.Compare` + `cmp.Or` — comparators
- `SortStableFunc` — stable order
- `BinarySearch` — sorted lookup

Stacks, Queues & Heaps

STACKS, QUEUES & HEAPS

Go has no built-in stack or queue types; **slices** cover both. `container/heap` turns any type implementing `heap.Interface` into a priority queue, and `container/list` provides a doubly linked list. A buffered **channel** is a ready-made concurrent FIFO queue with a fixed capacity.

NOTES

- Stack push/pop on a slice end is O(1) amortized
- Queue `s = s[1:]` is O(1) but keeps the old array until reallocated
- `heap.Interface` = `sort.Interface` + `Push` + `Pop`
- `container/list` is rarely faster than a slice in practice

STACK, QUEUE, HEAP

```
stack := []int{}
stack = append(stack, 1, 2, 3)            // push
top := stack[len(stack)-1]                // peek: 3
stack = stack[:len(stack)-1]              // pop

queue := []string{"a", "b"}
queue = append(queue, "c")                // enqueue
front := queue[0]; queue = queue[1:]      // dequeue "a"

type MinHeap []int
func (h MinHeap) Len() int           { return len(h) }
func (h MinHeap) Less(i, j int) bool { return h[i] < h[j] }
func (h MinHeap) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }
func (h *MinHeap) Push(x any)        { *h = append(*h, x.(int)) }
func (h *MinHeap) Pop() any {
    old := *h; x := old[len(old)-1]; *h = old[:len(old)-1]; return x
}
h := &MinHeap{5, 2, 8}
heap.Init(h)
heap.Push(h, 1)
fmt.Println(heap.Pop(h), heap.Pop(h))     // 1 2
```

CODE EXPLAINED

- → stack — append to push, reslice to pop
- → queue — `queue[1:]` drops the front
- → `Less` — `<` gives a min-heap; `>` a max-heap
- → `Push` / `Pop` — pointer receivers; called by `heap`
- → always use `heap.Push/Pop`, not `h.Push` directly

COMING FROM…

- **Python** — `heapq` ≈ `container/heap`
- **Java** — `PriorityQueue` ≈ heap; `ArrayDeque` ≈ slice
- **Python** — `deque` ≈ slice or ring buffer

CHOOSING

NEED	USE
LIFO stack	slice
FIFO queue	slice or ring buffer
concurrent queue	buffered chan
priority queue	container/heap
O(1) middle insert	container/list

HEAP FUNCTIONS

FUNC	COST
`heap.Init(h)`	O(n)
`heap.Push(h, x)`	O(log n)
`heap.Pop(h)`	O(log n), min first
`heap.Fix(h, i)`	after changing h[i]
`heap.Remove(h, i)`	O(log n)

⚠ COMMON MISTAKES

- Calling `h.Pop()` directly instead of `heap.Pop(h)`; the heap order breaks
- Popping from an empty slice: `index out of range [-1]`; check `len` first
- Long-running `queue = queue[1:]` loops that never shrink memory; copy down periodically

✓ TIPS & BEST PRACTICES

- For a max-heap, flip `Less` to `h[i] > h[j]`
- Store an `index` field in heap items to support `heap.Fix` updates
- Use a buffered channel as a semaphore or bounded work queue between goroutines

QUICK SUMMARY

- slice — stack and simple queue
- `container/heap` — priority queue
- `container/list` — linked list
- buffered channel — concurrent FIFO

Iterators

ITERATORS (RANGE-OVER-FUNC)

Since **Go 1.23**, `for range` accepts **iterator functions**. An `iter.Seq[V]` is a `func(yield func(V) bool)`: it calls `yield` for each value and must **stop when `yield` returns false** (the loop hit `break`). `iter.Seq2[K, V]` yields pairs. The `slices` and `maps` packages produce and consume these iterators.

NOTES

- Iterators are lazy: values are produced on demand
- Ignoring a `false` from `yield` causes a runtime panic
- `iter.Pull` converts a push iterator into `next()` / `stop()`
- Name iterator methods `All`, `Keys`, `Values`, `Backward`

WRITING & USING ITERATORS

```
func Countdown(n int) iter.Seq[int] {
    return func(yield func(int) bool) {
        for i := n; i > 0; i-- {
            if !yield(i) { return }   // stop on break
        }
    }
}
for v := range Countdown(5) {
    if v < 3 { break }
    fmt.Print(v, " ")                 // 5 4 3
}

func (s *Set[T]) All() iter.Seq[T] {  // method iterator
    return func(yield func(T) bool) {
        for k := range s.m { if !yield(k) { return } }
    }
}

for i, v := range slices.All([]string{"a", "b"}) { _, _ = i, v }
evens := slices.Collect(Filter(slices.Values(nums), isEven))
next, stop := iter.Pull(Countdown(3))
defer stop()
v, ok := next()                       // 3 true
```

CODE EXPLAINED

- → `iter.Seq[int]` — a function taking `yield`
- → `if !yield(i) { return }` — honor early `break`
- → methods returning iterators — `for x := range s.All()`
- → `slices.Values` → iterator → `slices.Collect` → slice
- → `iter.Pull` — step manually; always call `stop`

COMING FROM…

- **Python** — `yield` generators ≈ `iter.Seq` with `yield` func
- **JS** — `function*` generators, push-style
- **Java** — `Iterator` ≈ `iter.Pull`

TYPES

TYPE	SIGNATURE
`iter.Seq[V]`	`func(yield func(V) bool)`
`iter.Seq2[K,V]`	`func(yield func(K, V) bool)`
`iter.Pull(seq)`	`next() (V, bool), stop()`
`iter.Pull2(seq)`	pairs version

STDLIB ITERATORS

SOURCE	YIELDS
`slices.All / Values`	index+value / value
`slices.Backward`	reverse order
`maps.Keys / All`	keys / pairs
`strings.Lines / SplitSeq`	substrings (1.24)
`bytes.Lines`	[]byte lines

⚠ COMMON MISTAKES

- Not checking `yield`'s result: `panic: range function continued iteration after exit`
- Returning a `[]T` built eagerly when callers only need the first few items
- Forgetting `defer stop()` with `iter.Pull`, leaking its goroutine-like state

✓ TIPS & BEST PRACTICES

- Write small adapters (`Filter`, `Map`) once and compose them
- Expose `All()` methods on your collections instead of exporting internals
- Use `slices.Collect` / `maps.Collect` at the end of an iterator pipeline

QUICK SUMMARY

- `iter.Seq` / `Seq2` — iterator funcs
- `yield` false — stop immediately
- `for x := range seq` — consume (1.23)
- `slices.Collect` / `iter.Pull` — bridge

Methods & Receivers

METHODS

A method is a function with a **receiver**: `func (u User) Name() string`. A **value receiver** gets a copy; a **pointer receiver** `(u *User)` can modify the original and avoids copying. Go automatically takes the address or dereferences when you call a method, so `u.SetName(…)` works on an addressable value even with a pointer receiver.

CHOOSING A RECEIVER

- Pointer receiver if the method **mutates**, or the struct is large
- Pointer receiver if the type holds a `sync.Mutex` (never copy it)
- Be **consistent**: if any method needs `*T`, use `*T` for all
- Receiver names are short (`u` , `s`), never `this` or `self`

VALUE VS POINTER RECEIVERS

```
type Counter struct{ n int }

func (c Counter) Value() int { return c.n }  // value receiver
func (c *Counter) Inc()      { c.n++ }       // pointer receiver

var c Counter
c.Inc()                  // Go rewrites to (&c).Inc()
c.Inc()
fmt.Println(c.Value())   // 2

p := &Counter{}
p.Inc()                  // pointer → pointer method
fmt.Println(p.Value())   // 1: (*p).Value() automatically

func (c Counter) Broken() { c.n = 100 } // modifies a copy only

type Celsius float64                    // any defined type
func (t Celsius) String() string { return fmt.Sprintf("%.1f°C", t) }
fmt.Println(Celsius(21.5))              // 21.5°C

// Counter{}.Inc()       // error: can't take address of literal
```

CODE EXPLAINED

- → `(c Counter)` — receives a copy
- → `(c *Counter)` — operates on the original
- → `c.Inc()` — auto `&c` because `c` is addressable
- → `Broken` — changes vanish when it returns
- → methods work on any defined type, not just structs

COMING FROM…

- **Java / Python** — no `this` / `self`; receiver is named
- **C++** — pointer receiver ≈ non-const method
- **Python** — methods declared outside the struct

METHOD SETS

TYPE	CALLABLE METHODS
`T (addressable var)`	T and *T methods
`T (map elem, literal)`	T methods only
`*T`	T and *T methods
`interface satisfaction by T`	T methods only
`… by *T`	T and *T methods

RECEIVER RULES

RULE	DETAIL
`Same package`	methods only on local types
`Base type`	not a pointer or interface
`Name`	1-2 letters, consistent
`Mixed receivers`	allowed, but avoid

⚠ COMMON MISTAKES

- A setter with a value receiver; the change is applied to a copy and lost
- Calling a pointer method on a map element: `m[k].Inc()` doesn't compile (not addressable)
- Copying a struct that contains a `sync.Mutex` via a value receiver (`go vet` copylocks)

✓ TIPS & BEST PRACTICES

- Default to pointer receivers for structs with behavior; value receivers for small immutable types
- Implement `String() string` for readable `fmt` output
- Store `*T` in maps (`map[K]*T`) when you need to call mutating methods on entries

QUICK SUMMARY

- `func (r T) M()` — value receiver
- `func (r *T) M()` — pointer receiver, can mutate
- auto `&` / `*` on calls
- consistency — pick one receiver kind per type

Sheet 056

Embedding & Composition

embedded fields · promoted fields & methods · shadowing promoted names · embedding pointers · embedding interfaces in structs · composition over inheritance · not "is-a"

EMBEDDING

Go has no inheritance. Instead, a struct can **embed** another type by listing it without a field name. The embedded type's fields and methods are **promoted**, so they can be used as if declared on the outer struct. This is composition: the outer type *has* the inner one, and the inner methods still receive the inner value as their receiver.

NOTES

- The embedded field's name is its type name: `e.Logger`
- Outer fields/methods **shadow** promoted ones with the same name
- Promoted methods count toward interface satisfaction
- Embedding an interface in a struct delegates to whatever value is set

COMPOSITION BY EMBEDDING

```
type Logger struct{ prefix string }
func (l *Logger) Log(msg string) { fmt.Println(l.prefix + msg) }

type Service struct {
    *Logger                     // embedded pointer
    name string
}

s := Service{Logger: &Logger{"[svc] "}, name: "api"}
s.Log("started")                // promoted: [svc] started
s.Logger.Log("explicit")        // still reachable by type name

type ReadWriter struct{ io.Reader; io.Writer } // interfaces

type Admin struct {
    User                        // promotes User fields + methods
    Level int
}
a := Admin{User: User{Name: "Ann"}, Level: 9}
fmt.Println(a.Name)             // Ann (promoted field)
func (a Admin) Greet() string { return "Hi " + a.User.Greet() } // shadows
```

CODE EXPLAINED

- → `*Logger` — embedded pointer; set it or calls panic
- → `s.Log` — promoted from `*Logger`
- → `s.Logger` — the embedded field, named by its type
- → embedding `io.Reader` + `io.Writer` — satisfies `io.ReadWriter`
- → `Admin.Greet` — outer method shadows `User.Greet`

COMING FROM…

- **Java** — no `extends`; embedding is composition
- **Python** — no MRO; outer field simply shadows inner
- **Java** — no polymorphism via embedding; use interfaces

EMBEDDING VS INHERITANCE

CONCEPT	GO
is-a subtype	no: `Admin` is not a `User`
reuse methods	promotion
override	shadow with same name
call super	`a.User.Greet()`
polymorphism	interfaces

PROMOTION RULES

CASE	RESULT
outer defines name	outer wins
two embeds, same name	ambiguous if used
embedded T, method on *T	promoted if outer is addressable
unexported embedded	fields still promoted

⚠ COMMON MISTAKES

- Passing an `Admin` where a `User` is expected; it doesn't compile, pass `a.User`
- Expecting the inner method to call the outer's "override"; there's no virtual dispatch
- Leaving an embedded pointer or interface nil and calling a promoted method: nil dereference

✓ TIPS & BEST PRACTICES

- Embed to reuse behavior; use an interface when callers need substitutability
- Embed `sync.Mutex` only in unexported types (it exposes `Lock` / `Unlock` publicly)
- Wrap an interface by embedding it and overriding one method (decorator pattern)

QUICK SUMMARY

- embedding — field with no name
- promotion — inner fields/methods usable on outer
- shadowing — outer wins
- composition, not inheritance

Sheet 057

Encapsulation & Constructors

exported vs unexported (package-level privacy) · NewThing(…) Thing · getters without Get prefix · zero-value-ready types · internal/ packages · validation in constructors

ENCAPSULATION & CONSTRUCTORS

Go's only access control is **package-level**: capitalized identifiers are exported, everything else is package-private. There are no language constructors; by convention `NewThing(…)` returns an initialized `*Thing`. Types with a **useful zero value** may need no constructor at all.

CONVENTIONS

- `New` alone when the package has one main type: `list.New()`
- Getters drop `Get`: `u.Name()`, setters keep `Set`: `u.SetName()`
- `internal/` packages can only be imported by their parent tree
- Unexported fields + exported methods = invariants enforced

CONTROLLED CONSTRUCTION

```
type Account struct {
    owner   string        // unexported: package-only
    balance int64
}

func New(owner string) (*Account, error) {   // constructor
    if owner == "" { return nil, errors.New("owner required") }
    return &Account{owner: owner}, nil
}

func (a *Account) Owner() string  { return a.owner }   // getter
func (a *Account) Deposit(n int64) error {
    if n < 0 { return ErrNegative }
    a.balance += n
    return nil
}

acct, err := account.New("ann")   // from another package
// acct.balance = 1e9   // compile error: unexported field
```

CODE EXPLAINED

→ lowercase `owner`, `balance` — inaccessible outside the package

→ `New` — validates and returns `(*T, error)`

→ `Owner()` — getter without a `Get` prefix

→ `Deposit` — the only way to change the balance

→ `acct.balance` — rejected at compile time

COMING FROM…

- **Java** — no `private` / `public`; case decides visibility
- **Python** — `_name` is a convention; Go enforces it
- **Java** — `NewX()` replaces constructors

VISIBILITY

IDENTIFIER	VISIBLE
`Account, New`	everywhere
`owner, helper()`	same package
`internal/x`	parent tree only
`_test.go same pkg`	sees unexported
`pkg_test (external)`	exported only

CONSTRUCTOR NAMING

SITUATION	NAME
`main type of pkg`	`New`
`several types`	`NewReader`, `NewWriter`
`can fail`	`(*T, error)`
`must not fail`	`MustNew` (panics)
`zero value works`	no constructor

⚠ COMMON MISTAKES

- Exporting struct fields "for convenience" and losing the ability to enforce invariants
- Constructors that return a value `T` when methods have pointer receivers
- Naming getters `GetName()`; Go style is `Name()`

✓ TIPS & BEST PRACTICES

- Make the zero value usable where possible (`var b bytes.Buffer` style)
- Return concrete types from constructors; accept interfaces in parameters
- Use `internal/` for code shared inside your module but not public

QUICK SUMMARY

- Capitalized — exported; lowercase — package-private
- `NewX(…) (*X, error)` — constructor convention
- getters — `Name()`, not `GetName()`
- `internal/` — module-private packages

Interfaces

INTERFACES

An interface is a set of method signatures. A type satisfies an interface **implicitly**, simply by having those methods; there's no `implements` keyword. That keeps packages decoupled: you can define a small interface where you **use** it, and existing types fit automatically. An interface value holds a **(type, value)** pair.

DESIGN RULES

- Keep interfaces small; one-method interfaces end in `-er`
- "Accept interfaces, return structs"
- Define interfaces in the **consumer** package
- Interfaces can embed other interfaces

IMPLICIT SATISFACTION

```
type Shape interface {
    Area() float64
    Perimeter() float64
}

type Rect struct{ W, H float64 }
func (r Rect) Area() float64      { return r.W * r.H }
func (r Rect) Perimeter() float64 { return 2 * (r.W + r.H) }

type Circle struct{ R float64 }
func (c Circle) Area() float64      { return math.Pi * c.R * c.R }
func (c Circle) Perimeter() float64 { return 2 * math.Pi * c.R }

func total(shapes ...Shape) (sum float64) {
    for _, s := range shapes { sum += s.Area() }
    return
}
total(Rect{2, 3}, Circle{1})          // 9.14…

var _ Shape = Rect{}                  // compile-time check
var _ io.Writer = (*bytes.Buffer)(nil)

type ReadCloser interface { io.Reader; io.Closer } // embedding
```

CODE EXPLAINED

- → `Shape` — any type with both methods qualifies
- → `Rect`, `Circle` — no declaration of intent needed
- → `total(...Shape)` — polymorphism via interfaces
- → `var _ Shape = Rect{}` — fails to compile if methods drift
- → `io.Reader; io.Closer` — compose interfaces

COMING FROM…

- **Java / C#** — no `implements`; satisfaction is implicit
- **Python** — duck typing, but checked at compile time
- **TS** — like structural typing for interfaces

FAMOUS INTERFACES

INTERFACE	METHOD
`io.Reader`	`Read(p []byte) (int, error)`
`io.Writer`	`Write(p []byte) (int, error)`
`fmt.Stringer`	`String() string`
`error`	`Error() string`
`sort.Interface`	`Len, Less, Swap`

INTERFACE VALUE

STATE	== NIL?
`var s Shape`	yes (type & value nil)
`s = Rect{}`	no
`s = (*Rect)(nil)`	no! type is set
`s = nil`	yes

⚠ COMMON MISTAKES

- Declaring big interfaces up front "just in case" (interface pollution)
- Returning an interface from constructors, hiding useful concrete methods
- Assigning a nil pointer to an interface and then checking `iface == nil` (sheet 060)

✓ TIPS & BEST PRACTICES

- Add `var _ I = (*T)(nil)` next to types that must satisfy `I`
- Write the interface after you have two implementations or a test double
- Accept `io.Reader` instead of `*os.File` to make functions testable

QUICK SUMMARY

- interface — set of methods
- implicit satisfaction — no `implements`
- small, consumer-defined interfaces
- value = (dynamic type, value)

Sheet 059

any, Type Assertions & Type Switches

any = interface{} · v := x.(T) (panics) · v, ok := x.(T) · switch v := x.(type) · multiple types per case · asserting to an interface · assertions vs reflect

ANY & TYPE ASSERTIONS

any is an alias for `interface{}`, the interface with no methods, which **every type satisfies**. To get a concrete value back out you use a **type assertion** `x.(T)`, which panics on mismatch unless you use the two-result **comma-ok** form. A **type switch** checks several types at once.

NOTES

- Asserting to an interface type checks for those methods
- In a type switch with one type per case, `v` has that type
- With several types in a case, `v` keeps the interface type
- Prefer generics (sheet 062) over `any` when types are known

ASSERTIONS & TYPE SWITCHES

```
var x any = "hello"

s := x.(string)              // "hello"
n, ok := x.(int)             // 0 false: no panic
// n := x.(int)              // panic: interface conversion

func describe(v any) string {
    switch t := v.(type) {
    case nil:
        return "nil"
    case int, int64:
        return fmt.Sprintf("integer %v", t) // t is any here
    case string:
        return "string of len " + strconv.Itoa(len(t))
    case error:
        return "error: " + t.Error()
    case fmt.Stringer:
        return t.String()
    default:
        return fmt.Sprintf("other %T", t)
    }
}

if w, ok := r.(io.WriterTo); ok { w.WriteTo(dst) } // optional capability
```

CODE EXPLAINED

- → `x.(string)` — returns the string or panics
- → `v, ok := x.(T)` — safe form
- → `switch t := v.(type)` — `t` is typed per case
- → `case int, int64` — `t` stays `any`
- → asserting to `io.WriterTo` — detect optional methods

COMING FROM…

- **Java** — `x.(T)` ≈ cast; comma-ok avoids `ClassCastException`
- **Python** — `isinstance` ≈ type switch
- **TS** — `any` ≈ `unknown`, needs narrowing

FORMS

EXPRESSION	ON MISMATCH
`x.(T)`	panic
`v, ok := x.(T)`	zero value, false
`switch x.(type)`	falls to default
`errors.AsType[T]`	for errors (1.26)

WHEN TO USE ANY

CASE	BETTER?
`JSON of unknown shape`	`map[string]any` OK
`printf-style args`	`...any` OK
`generic container`	use type params
`plugin values`	define an interface

⚠ COMMON MISTAKES

- Single-result assertion on untrusted input; one unexpected type crashes the program
- Type-switching on many concrete types where an interface method would be cleaner
- Using `errors` type switches instead of `errors.As`, which misses wrapped errors

✓ TIPS & BEST PRACTICES

- Use `%T` to discover a value's dynamic type while debugging
- Check for optional interfaces (`io.WriterTo`, `http.Flusher`) with comma-ok
- Keep `any` at the edges (decoding); convert to concrete types quickly

QUICK SUMMARY

- `any` — `interface{}`, holds anything
- `x.(T)` — assert, panics on mismatch
- `v, ok := x.(T)` — safe assert
- `switch v := x.(type)` — type switch

Interface Gotchas

nil interface vs interface holding a nil pointer · pointer vs value method sets satisfying interfaces · comparing interfaces (panic on uncomparable) · interface pollution · define interfaces at the consumer · return concrete types

INTERFACE GOTCHAS

Most interface surprises come from two facts: an interface value is a **(type, value) pair**, and a type's **method set** depends on whether you hold a T or a *T. An interface holding a nil pointer is **not nil**, and a value T doesn't satisfy an interface whose methods have pointer receivers.

REMEMBER

- `iface == nil` only when both type and value are nil
- `T` has only value-receiver methods; `*T` has both
- Comparing interfaces with uncomparable dynamic types panics
- Interfaces belong to the consumer, not the implementer

THE CLASSIC TRAPS

```
type MyErr struct{}
func (*MyErr) Error() string { return "boom" }

func run() error {
    var e *MyErr = nil
    return e                    // returns non-nil error!
}
fmt.Println(run() == nil)       // false
func runFixed() error { return nil } // return literal nil

type Speaker interface{ Speak() }
type Dog struct{}
func (d *Dog) Speak() {}        // pointer receiver
// var s Speaker = Dog{}        // error: Speak has pointer receiver
var s Speaker = &Dog{}          // OK

var a, b any = []int{1}, []int{1}
// a == b                       // panic: comparing uncomparable
k := map[any]int{}
// k[[]int{1}] = 1              // panic: hash of unhashable type

func Save(w io.Writer) {}       // accept the small interface
```

CODE EXPLAINED

- → `return e` — interface gets type `*MyErr`, value nil
- → `run() == nil` — false because the type part is set
- → `Dog{}` lacks `Speak` (pointer method) in its method set
- → `&Dog{}` — `*Dog` has the method
- → uncomparable dynamic types panic at runtime, not compile time

COMING FROM…

- **Java** — no `null` check on interfaces; typed nil isn't nil
- **C#** — interface holding nil ≠ null reference
- **Python** — no equivalent trap; learn this one

NIL INTERFACE STATES

VALUE ASSIGNED	IFACE == NIL
nothing	true
nil (literal)	true
(*T)(nil)	false
T{} zero struct	false

SATISFIES INTERFACE?

METHODS ON	T / *T SATISFIES
value receiver	✔ / ✔
pointer receiver	✖ / ✔
mixed	only *T fully

⚠ COMMON MISTAKES

- Returning a typed nil pointer through an `error` result
- Putting values (not pointers) in an `[]Speaker` when methods use pointer receivers
- Defining an interface next to its only implementation "for mocking" in the producer package

✓ TIPS & BEST PRACTICES

- Return `nil` explicitly on success paths, never a nil-valued variable of concrete type
- Use `reflect.ValueOf(x).IsNil()` only as a last resort; fix the source instead
- `nilness` and `nilerr` linters catch many of these traps

QUICK SUMMARY

- interface nil — needs nil type **and** value
- typed nil pointer — non-nil interface
- pointer methods — only `*T` satisfies
- uncomparable dynamic types — `==` panics

Standard Library Interfaces

STANDARD LIBRARY INTERFACES

A handful of tiny interfaces glue the standard library together. Implement them on your types and they plug into `fmt`, `io.Copy`, `json`, `sort`, `net/http` and more for free. Recognizing them is one of the fastest ways to read idiomatic Go.

NOTES

- `io.Reader` / `io.Writer` connect files, sockets, buffers, hashes
- `String()` is used by `%v` and `%s`; `Error()` wins over it
- `MarshalJSON` / `UnmarshalJSON` customize JSON for a type

IMPLEMENTING STDLIB INTERFACES

```
type Level int
const (Debug Level = iota; Info; Warn)

var names = [...]string{"DEBUG", "INFO", "WARN"}
func (l Level) String() string { return names[l] } // fmt.Stringer
fmt.Println(Warn)                                  // WARN

func (l Level) MarshalText() ([]byte, error) { // TextMarshaler
    return []byte(strings.ToLower(l.String())), nil
}
json.Marshal(map[string]Level{"lvl": Info}) // {"lvl":"info"}

type Upper struct{ w io.Writer }            // io.Writer wrapper
func (u Upper) Write(p []byte) (int, error) {
    return u.w.Write(bytes.ToUpper(p))
}
fmt.Fprintln(Upper{os.Stdout}, "shout")    // SHOUT

type Hello struct{}                         // http.Handler
func (Hello) ServeHTTP(w http.ResponseWriter, _ *http.Request) {
    io.WriteString(w, "hi")
}
```

CODE EXPLAINED

- → `String()` — `fmt` prints `WARN`, not `2`
- → `MarshalText` — JSON writes the text form
- → `Upper` — any `io.Writer` can be decorated
- → `Fprintln(Upper{…})` — fmt accepts any writer
- → `ServeHTTP` — makes `Hello` an HTTP handler

COMING FROM…

- **Python** — `String()` ≈ `__str__`; `Error()` ≈ exception str
- **Java** — ≈ `toString`, `Comparable`, `Closeable`
- **Java** — `io.Reader` ≈ `InputStream`

CORE INTERFACES

INTERFACE	PACKAGE
`Reader / Writer / Closer`	io
`ReaderAt / Seeker`	io
`Stringer / Formatter`	fmt
`Marshaler / Unmarshaler`	encoding/json
`TextMarshaler`	encoding

MORE

INTERFACE	PURPOSE
`sort.Interface`	legacy sorting
`heap.Interface`	priority queue
`http.Handler`	serve requests
`context.Context`	cancel + deadlines
`hash.Hash`	io.Writer + Sum

⚠ COMMON MISTAKES

- `String()` that calls `fmt.Sprintf("%v", t)` on itself: infinite recursion. Convert to the underlying type first
- `Write` returning `n < len(p)` without an error; callers treat that as a failure
- Pointer-receiver `String()` used on values; `fmt` won't find it for `T`

✓ TIPS & BEST PRACTICES

- Use `io.Copy`, `io.MultiWriter`, `io.TeeReader` instead of manual loops
- Prefer `MarshalText` over `MarshalJSON` for simple scalar types
- Check `var _ fmt.Stringer = Level(0)` to lock in the implementation

QUICK SUMMARY

- `io.Reader` / `io.Writer` — streaming glue
- `fmt.Stringer` — custom printing
- `json.Marshaler` / `TextMarshaler` — encoding
- `http.Handler` — web servers

Generics — Basics

GENERICS

Since **Go 1.18**, functions and types can take **type parameters** in square brackets: `func Map[T, U any](…)`. Each type parameter has a **constraint** (an interface) that limits which types are allowed. Callers rarely write type arguments because the compiler **infers** them from the arguments.

NOTES

- `any` allows every type; `comparable` allows `==` / map keys
- `var zero T` gives a type parameter's zero value
- Generic types must be instantiated: `Stack[int]`
- Write generic code when the logic is identical across types

GENERIC FUNCTIONS & TYPES

```
func Map[T, U any](xs []T, f func(T) U) []U {
    out := make([]U, 0, len(xs))
    for _, x := range xs { out = append(out, f(x)) }
    return out
}
lens := Map([]string{"go", "rust"}, strLen) // [2 4]; T, U inferred

func Index[T comparable](xs []T, v T) int {
    for i, x := range xs { if x == v { return i } }
    return -1
}

type Stack[T any] struct{ items []T }
func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 { return zero, false }
    v := s.items[len(s.items)-1]; s.items = s.items[:len(s.items)-1]
    return v, true
}
var st Stack[string]                    // instantiate
st.Push("a")
```

CODE EXPLAINED

- → `[T, U any]` — two type parameters
- → type inference — no need to write `Map[string, int]`
- → `comparable` — required for `x == v`
- → `Stack[T any]` — generic struct type
- → `var zero T` — return a zero value of any T

COMING FROM…

- **Java** — `[T any]` ≈ `<T>`; no type erasure
- **TS** — generics look alike; constraints are interfaces
- **Python** — ≈ `TypeVar`, but enforced

SYNTAX

FORM	EXAMPLE
generic func	`func F[T any](x T)`
generic type	`type Box[T any] struct{ v T }`
method	`func (b Box[T]) Get() T`
explicit instantiation	`F[int](3)`
multiple params	`[K comparable, V any]`

USE GENERICS?

SITUATION	CHOICE
same logic, many types	generics
behavior differs per type	interface
containers / algorithms	generics
only calling methods	interface

⚠ COMMON MISTAKES

- Making everything generic; if a plain interface works, it's simpler
- `func F[T any](a, b T) bool { return a == b }` — `any` doesn't allow `==`; use `comparable`
- Expecting inference from return types alone; `F[int]()` must be explicit

✓ TIPS & BEST PRACTICES

- Check `slices`, `maps`, `cmp` first; they cover most generic helpers
- Name type params `T`, `K`, `V`, `E` (element), `S ~ []E` (slice)
- Benchmark hot paths; generic code is fast but not always identical to hand-written

QUICK SUMMARY

- `[T constraint]` — type parameters
- `any` / `comparable` — basic constraints
- inference — callers skip type args
- `type X[T any]` — generic types

Generics — Constraints

CONSTRAINTS

A constraint is an interface that can also list **type sets**: `int | float64` allows exactly those types, and `~int` allows any type whose **underlying type** is `int`. `cmp.Ordered` covers every type that supports <.

NOTES

- Constraint interfaces with type sets can't be used as normal variable types
- `~` matters: without it, `type Celsius float64` wouldn't match `float64`
- Inline constraints are allowed: `[T ~int | ~int64]`

DEFINING CONSTRAINTS

```
type Number interface {
    ~int | ~int32 | ~int64 | ~float32 | ~float64
}

func Sum[T Number](xs []T) T {
    var total T
    for _, x := range xs { total += x }  // + allowed for all
    return total
}

type Celsius float64
Sum([]Celsius{20.5, 21})                // 41.5 thanks to ~

func Largest[T cmp.Ordered](xs ...T) T { // ints, floats, strings
    m := xs[0]
    for _, x := range xs[1:] { m = max(m, x) }
    return m
}


func Keys[M ~map[K]V, K comparable, V any](m M) []K {
    return slices.Collect(maps.Keys(m))
}
```

CODE EXPLAINED

- → `Number` — union of underlying types
- → `total += x` — every member supports `+`
- → `~` — accepts defined types like `Celsius`
- → `cmp.Ordered` — `< <= > >=` available
- → `M ~map[K]V` — accept named map types too

COMING FROM…

- **Java** — `cmp.Ordered` ≈ `<T extends Comparable>`
- **Rust** — constraints ≈ trait bounds
- **C++** — ≈ concepts; `~int` ≈ underlying type match

BUILT-IN CONSTRAINTS

CONSTRAINT	ALLOWS
`any`	all types
`comparable`	==, !=, map keys
`cmp.Ordered`	< etc.: ints, floats, strings
`x/exp/constraints.Integer`	all integer types
`x/exp/constraints.Float`	float32, float64

TYPE SET SYNTAX

SYNTAX	MEANING
`int | string`	exactly these
`~int`	underlying type int
`interface{ ~int; String() string }`	both
`[S ~[]E, E any]`	any slice type

⚠ COMMON MISTAKES

- Forgetting `~`, so named types (`time.Duration`) are rejected
- Using a type-set constraint as a variable type: `var n Number` doesn't compile
- Expecting `T Number` to allow mixing `int` and `float64` in one call; `T` is one type per call

✓ TIPS & BEST PRACTICES

- Use the `S ~[]E` pattern so functions accept named slice types and return them unchanged
- Import `cmp.Ordered` rather than redefining your own ordered constraint
- Keep constraints minimal: ask only for the operations you use

QUICK SUMMARY

- constraint — interface + optional type set
- `A | B` — union; `~T` — underlying type
- `cmp.Ordered` — comparison operators
- `S ~[]E` — accept named slices

Sheet 064

Generic Methods & Advanced Generics

methods on generic types func (s Stack[T]) Push(v T) · generic methods with own type params (1.27) · limits (not in interfaces, can't satisfy interface methods) · generic type aliases · rand.N[Int] · inference on func assignment (1.27) · self-referential constraints

GENERIC METHODS & MORE

Methods on generic types use the type's parameters: `func (s *Stack[T]) Push(v T)`. Since **Go 1.27**, methods can also declare **their own type parameters**, like `math/rand/v2`'s `(*Rand).N[Int]`. Generic methods still can't appear in interfaces or satisfy interface methods. Go 1.24 added **generic type aliases**, and 1.27 extended inference to function assignment.

VERSION NOTES

- 1.18 — type params on funcs and types
- 1.24 — generic aliases: `type Set[T comparable] = map[T]struct{}`
- 1.27 — generic methods; inference generalized to func conversions
- Interface methods still can't have type params

ADVANCED GENERICS

```
type List[T any] struct{ items []T }
func (l *List[T]) Add(v T) { l.items = append(l.items, v) }

// Go 1.27: method with its own type parameter
func (l *List[T]) MapTo[U any](f func(T) U) *List[U] {
    out := &List[U]{}
    for _, v := range l.items { out.Add(f(v)) }
    return out
}
nums := &List[int]{items: []int{1, 2}}
strs := nums.MapTo(strconv.Itoa)       // *List[string]

r := rand.New(rand.NewPCG(1, 2))
d := r.N(10 * time.Second)             // generic method (1.27)

type Set[T comparable] = map[T]struct{} // generic alias (1.24)

var idx func([]string, string) int = slices.Index // inferred
f := (func([]int, int) int)(slices.Index) // conversion (1.27)

type Node[T any] interface {           // self-referential
    Children() []T
}
func Walk[T Node[T]](n T) { for _, c := range n.Children() { Walk(c) } }
```

CODE EXPLAINED

- → `(*List[T]).Add` — method using the type's `T`
- → `MapTo[U any]` — method-level type parameter (1.27)
- → `r.N(…)` — `rand.N` as a generic method on `*Rand`
- → `type Set[T] = …` — alias with type params (1.24)
- → `T Node[T]` — constraint referring to `T` itself

COMING FROM…

- **Java / C#** — generic methods arrived in Go 1.27
- **C++** — like member templates, but not allowed in interfaces
- **TS** — generic aliases ≈ generic `type` aliases

GENERIC METHODS LIMITS

ALLOWED?	CASE
✔	methods on generic types
✔	methods with own type params (1.27)
✖	type params in interface methods
✖	generic method satisfying interface

PATTERNS

PATTERN	EXAMPLE
container	`List[T]`, `Cache[K, V]`
result type	`Result[T]{Val T; Err error}`
func adapter	`Map`, `Filter`, `Reduce`
typed ID	`ID[T any] int64`

⚠ COMMON MISTAKES

- Adding a generic method to an interface; it's still not allowed in 1.27
- Using generic methods on a module whose `go.mod` says `go 1.26` or earlier
- Deep generic abstractions that hurt readability more than they remove duplication

✓ TIPS & BEST PRACTICES

- Prefer top-level generic functions when a method doesn't need the receiver's state
- Use generic aliases to rename types across packages without breaking callers
- Run `go fix` to apply modernizers after upgrading Go versions

QUICK SUMMARY

- `func (x *T[P]) M()` — method on generic type
- `func (x *T) M[U any]()` — generic method (1.27)
- `type A[T] = …` — generic alias (1.24)
- no type params in interface methods

io.Reader & io.Writer

STREAMS: READER & WRITER

`io.Reader` and `io.Writer` are the two most important interfaces in Go. Files, network connections, HTTP bodies, compressors, hashes and buffers all implement them, so one function written against `io.Reader` works with every source. The `io` package supplies helpers to copy, combine, limit and split streams.

THE READ CONTRACT

- `Read(p)` fills **up to** `len(p)` bytes and returns `n`
- Process `n > 0` bytes **before** checking `err`
- `io.EOF` signals the normal end of the stream
- `Write` must return a non-nil error if `n < len(p)`

COMPOSING STREAMS

```
r := strings.NewReader("hello, reader")
buf := make([]byte, 5)
for {
    n, err := r.Read(buf)
    fmt.Print(string(buf[:n]), "|")   // hello|, rea|der|
    if err == io.EOF { break }
    if err != nil { return err }
}

n, err := io.Copy(os.Stdout, resp.Body)    // stream, no full load
data, err := io.ReadAll(io.LimitReader(r, 1<<20)) // cap at 1 MB

h := sha256.New()
tee := io.TeeReader(file, h)               // hash while reading
io.Copy(io.Discard, tee)
fmt.Printf("%x\n", h.Sum(nil))

var log bytes.Buffer
w := io.MultiWriter(os.Stdout, &log)       // write to both
fmt.Fprintln(w, "saved")

pr, pw := io.Pipe()                        // sync in-memory pipe
go func() { json.NewEncoder(pw).Encode(v); pw.Close() }()
http.Post(url, "application/json", pr)
```

CODE EXPLAINED

- → `buf[:n]` — use only the bytes actually read
- → `io.Copy` — streams with a 32 KB buffer
- → `LimitReader` — protect against huge inputs
- → `TeeReader` — copy everything read into a writer
- → `io.Pipe` — connect a writer goroutine to a reader

COMING FROM…

- **Java** — `io.Reader` ≈ `InputStream`, `Writer` ≈ `OutputStream`
- **Python** — ≈ file-like objects with `read` / `write`
- **Node** — ≈ streams, but synchronous calls

IO HELPERS

FUNC	PURPOSE
`Copy(dst, src)`	stream everything
`CopyN(dst, src, n)`	exactly n bytes
`ReadAll(r)`	read into []byte
`ReadFull(r, buf)`	fill buf or error
`WriteString(w, s)`	write a string

ADAPTERS

FUNC	BUILDS
`LimitReader`	stops after n
`TeeReader`	read + copy
`MultiReader`	concatenate readers
`MultiWriter`	fan-out writes
`Discard`	writer that drops data

⚠ COMMON MISTAKES

- Checking `err` before using `n`; the last chunk often arrives together with `io.EOF`
- Treating `io.EOF` as a failure and reporting it to users
- `io.ReadAll` on untrusted input without a limit (memory exhaustion)

✓ TIPS & BEST PRACTICES

- Accept `io.Reader` / `io.Writer` params so tests can pass `strings.Reader` / `bytes.Buffer`
- Use `io.ReadFull` when you need an exact number of bytes (headers, frames)
- Wrap with `bufio` for many small reads/writes

QUICK SUMMARY

- `io.Reader` — `Read(p) (n, err)`
- `io.Writer` — `Write(p) (n, err)`
- `io.EOF` — normal end
- `io.Copy` / `TeeReader` / `MultiWriter` — compose

Reading Files

READING FILES

`os.ReadFile` loads a whole file into memory in one call, ideal for small files. For large files or line-by-line processing, `os.Open` returns an `*os.File` (an `io.Reader`), which you wrap in a `bufio.Scanner` or `bufio.Reader`. Always `defer f.Close()` **after** checking the open error.

NOTES

- `bufio.Scanner` strips the trailing `\n` (and `\r\n`)
- Scanner's default max line is **64 KB**; raise it with `Buffer`
- Check `scanner.Err()` after the loop
- `errors.Is(err, fs.ErrNotExist)` detects a missing file

READING PATTERNS

```
data, err := os.ReadFile("config.json")   // whole file
if errors.Is(err, fs.ErrNotExist) { return defaults, nil }

f, err := os.Open("access.log")
if err != nil { return err }
defer f.Close()

sc := bufio.NewScanner(f)
sc.Buffer(make([]byte, 0, 64*1024), 1024*1024) // allow 1 MB lines
for sc.Scan() {
    line := sc.Text()                  // no trailing newline
    if strings.HasPrefix(line, "#") { continue }
    process(line)
}
if err := sc.Err(); err != nil { return err }

sc = bufio.NewScanner(strings.NewReader("a b  c"))
sc.Split(bufio.ScanWords)              // word by word

br := bufio.NewReader(os.Stdin)        // interactive input
name, _ := br.ReadString('\n')         // includes the '\n'
name = strings.TrimSpace(name)
```

CODE EXPLAINED

- → `os.ReadFile` — returns `[]byte`; convert with `string(data)`
- → `defer f.Close()` — only after the error check
- → `sc.Scan()` — false at EOF or on error
- → `sc.Buffer` — needed for lines over 64 KB
- → `ReadString('\n')` — keeps the delimiter; trim it

COMING FROM…

- **Python** — `open().read()` → `os.ReadFile`
- **Python** — `for line in f` → `bufio.Scanner`
- **Java** — `Files.readAllBytes` → `os.ReadFile`

CHOOSING A READER

NEED	USE
small file at once	os.ReadFile
lines	bufio.Scanner
words / runes	sc.Split(ScanWords)
very long lines	bufio.Reader.ReadLine
binary chunks	f.Read(buf) / io.ReadFull

OPENING FILES

FUNC	MODE
os.Open(name)	read-only
os.Create(name)	write, truncate
os.OpenFile(n, flag, perm)	custom
os.Stdin	standard input
os.DirFS(dir)	read-only fs.FS

⚠ COMMON MISTAKES

- `bufio.Scanner: token too long` on big lines; call `sc.Buffer` with a larger max
- Forgetting `sc.Err()`, so read errors look like a normal end of file
- Reading multi-GB files with `os.ReadFile`; stream them instead

✓ TIPS & BEST PRACTICES

- Open with `os.Open` + `defer Close` in a small helper so the file closes per call
- Pass `io.Reader` to parsing functions instead of file names
- Use `os.Root` (1.24) to confine reads to a directory when paths come from users

QUICK SUMMARY

- `os.ReadFile` — whole file
- `os.Open` + `defer f.Close()` — streaming
- `bufio.Scanner` — line by line
- `fs.ErrNotExist` — missing file check

Writing Files

os.WriteFile(name, data, 0o644) · os.Create · os.OpenFile with O_APPEND|O_CREATE|O_WRONLY · bufio.Writer + Flush · fmt.Fprintf(f, …) · f.Sync · check the Close error · atomic write via temp + os.Rename

WRITING FILES

`os.WriteFile` writes a whole byte slice in one call, creating or truncating the file. For streaming output, `os.Create` or `os.OpenFile` returns an `*os.File`; wrap it in `bufio.Writer` for many small writes and remember to **`Flush`**. The error from **`Close` matters for writes**, since buffered data can fail to land on disk.

PERMISSIONS & FLAGS

- `0o644` — owner rw, others r (typical files)
- `0o755` — dirs and executables; `0o600` — secrets
- Permissions are filtered by the process `umask`
- Combine flags with `|`: `os.O_APPEND|os.O_CREATE|os.O_WRONLY`

WRITING PATTERNS

```
err := os.WriteFile("out.txt", []byte("hello\n"), 0o644)

f, err := os.OpenFile("app.log",
    os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0o644)   // append
if err != nil { return err }
defer f.Close()
fmt.Fprintf(f, "%s started\n", time.Now().Format(time.RFC3339))

func writeReport(path string, rows []Row) (err error) {
    f, err := os.Create(path)                      // truncate
    if err != nil { return err }
    defer func() { err = errors.Join(err, f.Close()) }()
    w := bufio.NewWriter(f)
    for _, r := range rows { fmt.Fprintln(w, r) }
    return w.Flush()                               // don't forget
}

tmp, _ := os.CreateTemp(dir, "cfg-*.tmp")          // atomic write
tmp.Write(data); tmp.Sync(); tmp.Close()
os.Rename(tmp.Name(), filepath.Join(dir, "cfg.json"))
```

CODE EXPLAINED

- → `os.WriteFile` — create/truncate + write + close
- → `O_APPEND|O_CREATE|O_WRONLY` — append, creating if needed
- → `errors.Join(err, f.Close())` — keep the close error
- → `w.Flush()` — push buffered data to the file
- → temp + `Sync` + `Rename` — readers never see half a file

COMING FROM…

- **Python** — `open(p, 'a')` → `O_APPEND|O_CREATE|O_WRONLY`
- **Java** — `BufferedWriter` → `bufio.Writer` + `Flush`
- **Node** — `fs.writeFileSync` → `os.WriteFile`

OPEN FLAGS

FLAG	MEANING
`O_RDONLY / O_WRONLY / O_RDWR`	access mode
`O_CREATE`	create if missing
`O_TRUNC`	empty existing file
`O_APPEND`	write at end
`O_EXCL`	fail if exists (with CREATE)

WRITE METHODS

CALL	WRITES
`f.Write(b)`	[]byte
`f.WriteString(s)`	string
`fmt.Fprintf(f, …)`	formatted
`w.Flush()`	buffered → file
`f.Sync()`	fsync to disk

⚠ COMMON MISTAKES

- Forgetting `bufio.Writer.Flush()`: the end of the file is silently missing
- Ignoring `Close()` errors on written files (full disk, NFS errors)
- Using `0644` permissions for files containing secrets; use `0o600`

✓ TIPS & BEST PRACTICES

- Write atomically with temp file + `os.Rename` (same directory/filesystem)
- Use `os.MkdirAll(dir, 0o755)` before writing into nested paths
- Return `w.Flush()` as the last statement so its error isn't lost

QUICK SUMMARY

- `os.WriteFile` — one-shot write
- `os.Create` / `os.OpenFile` — streaming
- `bufio.Writer` + `Flush` — fast small writes
- check `Close` — write errors surface there

Paths & the Filesystem

PATHS & DIRECTORIES

Use `path/filepath` for **operating-system paths** (it uses \ on Windows) and `path` only for slash-separated paths like URLs. The `os` package handles directory creation, listing, stat and removal. **Go 1.24's `os.Root`** opens a directory and blocks any access that escapes it, which prevents path-traversal bugs.

NOTES

- `filepath.Join` cleans `..` and duplicate separators
- `os.ReadDir` returns entries **sorted by name**
- `WalkDir` is faster than the older `Walk` (no extra `Stat`)
- `os.UserHomeDir()`, `os.UserConfigDir()` for per-user paths

FILESYSTEM OPERATIONS

```
p := filepath.Join("data", "2026", "report.csv")  // data/2026/report.csv
filepath.Base(p)               // report.csv
filepath.Dir(p)                // data/2026
filepath.Ext(p)                // .csv
abs, _ := filepath.Abs(p)      // /home/me/app/data/2026/report.csv
rel, _ := filepath.Rel("data", p) // 2026/report.csv

info, err := os.Stat(p)
if errors.Is(err, fs.ErrNotExist) { /* missing */ }
fmt.Println(info.Size(), info.IsDir(), info.ModTime())

os.MkdirAll("out/logs", 0o755)
entries, _ := os.ReadDir(".")
for _, e := range entries { fmt.Println(e.Name(), e.IsDir()) }

filepath.WalkDir(".", func(path string, d fs.DirEntry, err error) error {
    if err != nil { return err }
    if d.IsDir() && d.Name() == ".git" { return filepath.SkipDir }
    if filepath.Ext(path) == ".go" { fmt.Println(path) }
    return nil
})
matches, _ := filepath.Glob("*.json")
root, _ := os.OpenRoot("uploads")  // 1.24: can't escape dir
f, err := root.Open("../etc/passwd") // error: escapes root
```

CODE EXPLAINED

- → `filepath.Join` — OS separator, cleaned result
- → `os.Stat` + `fs.ErrNotExist` — existence check
- → `os.MkdirAll` — like `mkdir -p`
- → `filepath.SkipDir` — prune a directory in `WalkDir`
- → `os.OpenRoot` — safe access to user-supplied paths

COMING FROM…

- **Python** — `os.path.join` → `filepath.Join`
- **Python** — `os.walk` → `filepath.WalkDir`
- **Java** — `Paths.get` / `Files.walk` equivalents

FILEPATH

FUNC	RESULT
`Join(a, b)`	a/b
`Split(p)`	dir, file
`Ext(p)`	.csv
`Clean("a//b/../c")`	a/c
`Match("*.go", n)`	glob match

OS DIR & FILE OPS

FUNC	LIKE
`MkdirAll(p, perm)`	mkdir -p
`RemoveAll(p)`	rm -rf
`Rename(a, b)`	mv
`MkdirTemp("", "x")`	mktemp -d
`Getwd() / Chdir()`	pwd / cd

⚠ COMMON MISTAKES

- Building paths with `+ "/" +`; breaks on Windows and doesn't clean `..`
- Joining user input to a base dir and trusting it; `../` escapes. Use `os.Root` or check with `filepath.IsLocal`
- Using `path.Join` for file paths (always `/`)

✓ TIPS & BEST PRACTICES

- Use `filepath.IsLocal(p)` (1.20) to validate relative paths from users
- Prefer `os.ReadDir` / `WalkDir` over the older `ioutil` / `Walk` APIs
- Use `t.TempDir()` in tests; it's cleaned up automatically

QUICK SUMMARY

- `path/filepath` — OS paths
- `os.Stat` / `ReadDir` / `MkdirAll` — fs ops
- `filepath.WalkDir` — recursive walk
- `os.Root` — confined access (1.24)

embed & io/fs

EMBED & IO/FS

The `//go:embed` directive compiles files **into the binary** at build time, so templates, static web assets and SQL migrations ship inside a single executable. Embedded directories become an `embed.FS`, which implements the `io/fs.FS` interface. Code written against `fs.FS` works equally with embedded files, real directories (`os.DirFS`) and test fakes (`fstest.MapFS`).

RULES

- The directive goes directly above a **package-level** var
- Patterns are relative to the source file's directory; no `..`
- Files starting with `.` or `_` are skipped unless you use `all:`
- Requires `import "embed"` (blank import for string/[]byte vars)

EMBEDDING FILES

```
//go:embed version.txt
var version string                // file contents as string

//go:embed logo.png
var logo []byte

//go:embed templates/*.html static
var assets embed.FS               // directory tree

//go:embed all:public
var public embed.FS               // include dot/underscore files

page, _ := assets.ReadFile("templates/index.html")
tmpl := template.Must(template.ParseFS(assets, "templates/*.html"))

sub, _ := fs.Sub(assets, "static")              // strip prefix
http.Handle("/static/", http.StripPrefix("/static/",
    http.FileServerFS(sub)))                    // 1.22

var testFS = fstest.MapFS{"a.txt": {Data: []byte("hi")}}
```

CODE EXPLAINED

- → `string` / `[]byte` — single file only
- → `embed.FS` — one or more files and directories
- → `all:` — keep hidden and `_` files
- → `fs.Sub` — re-root an FS at a subdirectory
- → `fstest.MapFS` — in-memory FS for tests

COMING FROM…

- **Java** — `//go:embed` ≈ classpath resources
- **Python** — ≈ `importlib.resources`
- **Rust** — ≈ `include_str!` / `include_bytes!`

FS FUNCTIONS

FUNC	DOES
`fs.ReadFile(fsys, name)`	read a file
`fs.ReadDir(fsys, dir)`	list entries
`fs.WalkDir(fsys, root, fn)`	walk tree
`fs.Glob(fsys, pattern)`	match names
`fs.Sub(fsys, dir)`	subtree FS

FS IMPLEMENTATIONS

SOURCE	TYPE
`embedded files`	`embed.FS`
`real directory`	`os.DirFS(dir)`
`zip archive`	`*zip.Reader`
`tests`	`fstest.MapFS`
`restricted dir`	`os.Root.FS()` (1.24)

⚠ COMMON MISTAKES

- Putting `//go:embed` above a local variable; it only works at package level
- Writing `// go:embed` with a space; it becomes a plain comment
- Using OS-style `\` separators with `embed.FS`; paths are always `/`

✓ TIPS & BEST PRACTICES

- Embed migrations and templates to deploy a single binary
- Accept `fs.FS` in functions so tests can use `fstest.MapFS`
- Use `go list -f '{{.EmbedFiles}}'` to see what got embedded

QUICK SUMMARY

- `//go:embed` — files into the binary
- `string` / `[]byte` / `embed.FS` targets
- `fs.FS` — common filesystem interface
- `fs.Sub` / `http.FileServerFS` — serving

JSON

JSON

`encoding/json` maps JSON to Go values using **struct tags** to rename fields and control omission. Only **exported** fields are encoded. Go 1.24 added `omitzero`, and **Go 1.27** ships `encoding/json/v2` and `jsontext`: the v1 package now runs on the v2 engine, v2 rejects invalid UTF-8 and duplicate keys by default, and unmarshaling is much faster.

NOTES

- `omitempty` skips `0`, `""`, `false`, nil, empty slices/maps
- `omitzero` (1.24) skips zero values, incl. `time.Time{}` structs
- Unknown JSON fields are ignored unless you call `DisallowUnknownFields`
- Numbers decode into `float64` when the target is `any`

ENCODE & DECODE

```
type User struct {
    ID      int       `json:"id"`
    Name    string    `json:"name"`
    Email   string    `json:"email,omitempty"`
    Created time.Time `json:"created,omitzero"`
    Secret  string    `json:"-"`                 // never encoded
}

b, err := json.Marshal(User{ID: 1, Name: "Ann"})
fmt.Println(string(b))            // {"id":1,"name":"Ann"}

var u User
err = json.Unmarshal([]byte(`{"id":2,"name":"Bo"}`), &u)

dec := json.NewDecoder(r.Body)    // streams from a Reader
dec.DisallowUnknownFields()
if err := dec.Decode(&u); err != nil { /* 400 */ }
json.NewEncoder(w).Encode(u)      // to a Writer (+ newline)

var anyv map[string]any           // unknown shape
json.Unmarshal(data, &anyv)       // numbers → float64
type Env struct{ Type string; Data json.RawMessage } // defer decoding
```

CODE EXPLAINED

- → `json:"name"` — key name in JSON
- → `omitempty` / `omitzero` — drop empty values
- → `json:"-"` — exclude a field
- → `Unmarshal(data, &u)` — needs a pointer
- → `RawMessage` — keep raw bytes, decode later by type

COMING FROM…

- **Python** — `json.dumps` → `json.Marshal` (struct tags rename)
- **JS** — `JSON.parse` → `Unmarshal` into a typed struct
- **Java** — Jackson annotations ≈ struct tags

GO ↔ JSON TYPES

GO	JSON
`bool`	true / false
`int, float64`	number
`string`	string
`[]T / array`	array (nil slice → null)
`map[string]T, struct`	object

JSON/V2 (1.27)

ITEM	BEHAVIOR
`json/v2.Marshal`	new API, options-based
`jsontext`	low-level tokens/values
`v1 encoding/json`	now v2-backed
`invalid UTF-8 / dup keys`	rejected by default in v2

⚠ COMMON MISTAKES

- Lowercase struct fields: they're silently left out of the JSON
- `json.Unmarshal(data, u)` without `&`: returns an `InvalidUnmarshalError`
- Decoding large numbers into `any`; IDs lose precision as `float64`. Use `dec.UseNumber()` or typed fields

✓ TIPS & BEST PRACTICES

- Use `Decoder` for HTTP bodies and files; `Unmarshal` for in-memory bytes
- Implement `MarshalJSON` / `UnmarshalJSON` or `TextMarshaler` for custom formats
- Test against `GOEXPERIMENT=nojsonv2` when upgrading to 1.27 if output must stay byte-identical

QUICK SUMMARY

- `Marshal` / `Unmarshal` — bytes ↔ values
- `NewEncoder` / `NewDecoder` — streams
- tags — `name`, `omitempty`, `omitzero`, `-`
- `encoding/json/v2` — new engine (1.27)

CSV, XML & Other Encodings

CSV, XML & Other Encodings

The `encoding/*` packages convert between Go values and common formats. `encoding/csv` reads and writes records as `[]string`. `encoding/xml` works like `json` with struct tags. `base64` and `hex` turn bytes into text, `binary` handles fixed-size binary layouts, and `gob` is Go-to-Go serialization.

Notes

- CSV reader handles quoted fields and embedded commas
- `r.Comma = ';'` or `'\t'` changes the delimiter
- `csv.Writer` buffers: call `Flush` then check `Error()`
- `base64.URLEncoding` is safe in URLs and filenames

Encoding Round-Trips

```
r := csv.NewReader(strings.NewReader("name,age\n\"Lee, Ann\",31\n"))
records, _ := r.ReadAll()          // [[name age] [Lee, Ann 31]]
for _, rec := range records[1:] { fmt.Println(rec[0]) } // Lee, Ann

w := csv.NewWriter(os.Stdout)
w.Write([]string{"id", "note"})
w.Write([]string{"1", `says "hi"`}) // 1,"says ""hi"""
w.Flush()
if err := w.Error(); err != nil { return err }

type Book struct {
    XMLName xml.Name `xml:"book"`
    ID      string   `xml:"id,attr"`
    Title   string   `xml:"title"`
}
out, _ := xml.Marshal(Book{ID: "7", Title: "Go"})
// <book id="7"><title>Go</title></book>

s := base64.StdEncoding.EncodeToString([]byte("hi?")) // aGk/
raw, _ := base64.StdEncoding.DecodeString(s)
hex.EncodeToString([]byte{0xde, 0xad})                // dead
buf := binary.BigEndian.AppendUint32(nil, 258)        // [0 0 1 2]
```

Code Explained

- → `ReadAll` — all records; use `Read` in a loop for big files
- → quoted fields — commas and quotes handled for you
- → `w.Flush()` + `w.Error()` — catch write failures
- → `xml:"id,attr"` — encode as an attribute
- → `binary.BigEndian` — fixed byte-order integers

Coming From…

- **Python** — `csv` module ≈ `encoding/csv`
- **Python** — `base64.b64encode` → `base64.StdEncoding`
- **Java** — `struct.pack` ≈ `encoding/binary`

Packages

Package	For
encoding/csv	spreadsheets, exports
encoding/xml	XML documents, SOAP
encoding/base64	binary in text (email, JSON)
encoding/hex	hashes, debugging
encoding/binary	protocols, file headers
encoding/gob	Go ↔ Go (RPC, caches)

csv.Reader Options

Field	Effect
Comma	delimiter rune
Comment	skip lines starting with it
FieldsPerRecord	enforce column count (-1 = any)
LazyQuotes	tolerate bad quotes
TrimLeadingSpace	trim after delimiter

⚠ Common Mistakes

- Splitting CSV lines with `strings.Split(line, ",")`; quoted commas break it
- Forgetting `w.Flush()` on `csv.Writer`, losing the last rows
- Using `gob` for data other languages must read; it's Go-specific

✓ Tips & Best Practices

- Stream big CSV files with `r.Read()` in a loop and `r.ReuseRecord = true`
- Use `base64.RawURLEncoding` for tokens (no padding, URL-safe)
- Third-party YAML/TOML: `gopkg.in/yaml.v3`, `github.com/BurntSushi/toml`

Quick Summary

- `csv` — records as `[]string`
- `xml` — struct tags like JSON
- `base64` / `hex` — bytes ↔ text
- `binary` / `gob` — binary formats

Packages & Imports

PACKAGES & IMPORTS

A **package** is a directory of `.go` files sharing one `package` name, and it's the unit of compilation and visibility. You import packages by **import path** (module path + directory), but refer to them by **package name**. Import cycles are forbidden, which keeps dependency graphs clean.

CONVENTIONS

- Package names: short, lowercase, no underscores (`strconv` , `httputil`)
- Avoid `util` , `common` , `helpers` ; name by purpose
- `internal/` restricts importers to the parent tree
- `cmd/<name>/main.go` for each binary in a repo

IMPORT FORMS

```
import (
    "fmt"                                // stdlib
    "net/http"                           // name is "http"

    "github.com/google/uuid"             // third-party module
    "example.com/shop/internal/store"    // own module, internal

    mrand "math/rand/v2"                 // alias (avoid clash)
    _ "github.com/lib/pq"                // side effects only
    . "example.com/shop/testutil"        // dot import (avoid)
)

func main() {
    id := uuid.New()                     // pkg name, not path
    n := mrand.IntN(10)
    http.ListenAndServe(":8080", store.Handler())
}

// shop/
//   go.mod              module example.com/shop
//   cmd/api/main.go     package main
//   internal/store/     package store (shop-only)
//   pkg/client/         package client (importable)
```

CODE EXPLAINED

→ stdlib first, then third-party, then local (goimports groups)

→ `net/http` — import path; `http` — package name

→ `mrand` — alias when two packages share a name

→ `_` — run `init` only; `.` — names without prefix

→ `internal/store` — importable only within `shop/…`

COMING FROM…

- **Python** — directory = package; no `__init__.py`
- **Java** — import paths, but no one-class-per-file rule
- **JS** — no default exports; capitalized names export

IMPORT KINDS

FORM	USE
`"pkg/path"`	normal
`alias "pkg/path"`	rename / resolve clash
`_ "pkg/path"`	side effects (drivers)
`. "pkg/path"`	rarely: some test DSLs

LAYOUT DIRS

DIR	HOLDS
`cmd/<app>`	`package main` per binary
`internal/`	private packages
`pkg/ (optional)`	public libraries
`testdata/`	fixtures, ignored by build
`vendor/`	vendored deps

⚠ COMMON MISTAKES

- Import cycles (`a` imports `b` imports `a`): move shared types to a third package
- Stuttering names: `user.UserService` → `user.Service`
- One giant `utils` package that everything imports

✓ TIPS & BEST PRACTICES

- Run `goimports` to add, remove and group imports automatically
- Start flat (one package); split when a clear boundary appears
- Put shared-but-private code in `internal/` to keep your public API small

QUICK SUMMARY

- package — one directory, one name
- import path vs package name
- `alias` , `_` , `.` import forms
- `internal/` & `cmd/` — layout conventions

Modules & go.mod

MODULES & GO.MOD

A **module** is a versioned collection of packages defined by a `go.mod` file at its root. `go.mod` records the module path, the **minimum Go version**, an optional preferred `toolchain`, and dependency requirements. Go uses **minimal version selection**: builds use the lowest versions that satisfy every requirement, so they're reproducible.

NOTES

- Module path is usually the repo URL: `github.com/me/app`
- Major versions ≥ 2 put `/v2` in the module path
- `// indirect` marks deps needed only by other deps
- `go.sum` holds checksums; commit both files

A GO.MOD FILE

```
// $ go mod init github.com/me/shop

module github.com/me/shop

go 1.27                              // minimum language version
toolchain go1.27.1                   // preferred toolchain

require (
    github.com/go-chi/chi/v5 v5.2.1
    github.com/jackc/pgx/v5 v5.7.4
    golang.org/x/sync v0.14.0 // indirect
)

replace github.com/me/lib => ../lib   // local development
exclude github.com/bad/pkg v1.2.3     // never use this version
retract v1.0.1                        // "don't use our v1.0.1"

tool golang.org/x/tools/cmd/stringer  // 1.24: go tool stringer
```

CODE EXPLAINED

- → `module` — the import-path prefix for your packages
- → `go 1.27` — enables 1.27 language features (and requires it)
- → `require` — minimum versions of dependencies
- → `replace` — point a module at a local path or fork
- → `tool` — track dev tools; run with `go tool <name>` (1.24)

COMING FROM…

- **Node** — `go.mod` ≈ `package.json`; `go.sum` ≈ lockfile
- **Python** — ≈ `pyproject.toml` with pinned minimums
- **Java** — ≈ `pom.xml` dependencies, far simpler

DIRECTIVES

DIRECTIVE	PURPOSE
`module`	module path
`go`	language version / minimum
`toolchain`	suggested toolchain
`require`	dependency + version
`replace / exclude`	override / block versions
`retract / tool / godebug`	withdraw / tools / defaults

SEMANTIC VERSIONS

VERSION	MEANING
`v1.4.2`	release
`v2.0.0 → /v2 path`	breaking change
`v0.x.y`	unstable API
`v1.5.0-rc.1`	pre-release
`v0.0.0-2026…-abc123`	pseudo-version (commit)

⚠ COMMON MISTAKES

- Committing `replace ../local` lines; they break everyone else's builds
- Tagging `v2.0.0` without changing the module path to `/v2`
- Editing `go.mod` by hand and not running `go mod tidy` afterwards

✓ TIPS & BEST PRACTICES

- Use `go.work` instead of `replace` for multi-module local development
- Pin dev tools with the `tool` directive instead of a `tools.go` file
- Set the `go` line to the oldest version you actually support

QUICK SUMMARY

- `go mod init path` — create a module
- `go` / `toolchain` — version requirements
- `require` / `replace` — dependencies
- `/v2` — major version in path

Dependencies & Workspaces

MANAGING DEPENDENCIES

`go get` adds, upgrades or removes a dependency in `go.mod`; `go mod tidy` then syncs requirements with what the code actually imports. Modules are downloaded through a **proxy** (`proxy.golang.org`) and verified against the **checksum database**. **Workspaces** (`go.work`) let you develop several modules together.

NOTES

- `go get pkg@none` removes a dependency
- `GOPRIVATE=github.com/myorg/*` skips proxy + checksum DB
- `go mod vendor` copies deps into `vendor/` for offline builds
- Don't commit `go.work` in library repos (usually)

DEPENDENCY COMMANDS

```
go get github.com/go-chi/chi/v5          # add latest
go get github.com/go-chi/chi/v5@v5.1.0   # specific version
go get -u ./...                          # upgrade all (minor/patch)
go get -u=patch ./...                    # patch upgrades only
go get example.com/old@none              # remove
go mod tidy                              # add missing, drop unused

go list -m all                           # full module graph
go list -m -u all                        # show available updates
go mod why -m golang.org/x/text          # why is this needed?
go mod graph | grep chi                  # who requires what

go env -w GOPRIVATE='github.com/myorg/*'
go mod vendor                            # create vendor/
govulncheck ./...                        # known vulnerabilities

go work init ./api ./lib                 # workspace (go.work)
go work use ./tools                      # add another module
```

CODE EXPLAINED

- → `@version` — tag, branch, commit or `latest`
- → `-u` — upgrade deps of the named packages
- → `go mod tidy` — the go-to cleanup after changes
- → `go mod why` — trace an unexpected dependency
- → `go work` — local multi-module development

COMING FROM…

- **npm** — `go get` ≈ `npm install pkg`
- **npm** — `go mod tidy` ≈ prune + install
- **pip** — `govulncheck` ≈ `pip-audit`

VERSION QUERIES

QUERY	SELECTS
`@latest`	newest release
`@v1.2.3`	exact version
`@master / @main`	branch head
`@abc1234`	commit
`@none`	remove

ENV VARS

VAR	PURPOSE
`GOPROXY`	proxy list (`direct` = VCS)
`GOPRIVATE`	private module patterns
`GONOSUMDB`	skip checksum DB
`GOFLAGS`	default flags (`-mod=vendor`)
`GOINSECURE`	allow http (avoid)

⚠ COMMON MISTAKES

- Running `go get -u ./...` in CI and getting different builds each day
- Not setting `GOPRIVATE` for company repos: "410 Gone" / checksum errors
- Forgetting `go mod tidy`, so CI fails on a dirty `go.mod`

✓ TIPS & BEST PRACTICES

- In CI, run `go mod tidy -diff` (1.23) to fail on untidy modules
- Run `govulncheck` in CI; it reports only vulnerabilities your code actually calls
- Use Dependabot/Renovate to propose upgrades as reviewable PRs

QUICK SUMMARY

- `go get pkg@ver` — add / change
- `go mod tidy` — sync with imports
- `GOPRIVATE` — private modules
- `go work` — multi-module dev

math & math/rand/v2

MATH & MATH/RAND/V2

`math` provides float functions and constants, while `math/rand/v2` (Go 1.22) is the modern random package: it's **auto-seeded**, has clearer names (`IntN`), and offers a generic `rand.N` that works with any integer type, including `time.Duration`. For **secrets** (tokens, passwords) always use `crypto/rand`.

NOTES

- Top-level `rand` funcs are safe for concurrent use
- Use `rand.New(rand.NewPCG(s1, s2))` for reproducible sequences
- `crypto/rand.Text()` (1.24) returns a secure random string
- `math/big` handles arbitrary-precision ints, floats, rationals

MATH & RANDOMNESS

```
math.Sqrt(16)          // 4
math.Pow(2, 8)         // 256
math.Abs(-3.2)         // 3.2
math.Mod(10, 3)        // 1
math.Max(3, 7)         // 7 (floats; use max() for ints)
math.Log(math.E)       // 1
math.Pi, math.MaxInt   // 3.14159… 9223372036854775807

rand.IntN(6) + 1                      // dice roll 1-6
rand.Float64()                        // [0.0, 1.0)
rand.N(5 * time.Second)               // random duration < 5s
rand.Shuffle(len(s), func(i, j int) { s[i], s[j] = s[j], s[i] })
rand.Perm(4)                          // e.g. [2 0 3 1]

r := rand.New(rand.NewPCG(1, 42))     // seeded: same each run
r.IntN(100)

token := cryptorand.Text()            // crypto/rand, 1.24
b := make([]byte, 32); cryptorand.Read(b)

f := new(big.Int).Exp(big.NewInt(2), big.NewInt(100), nil) // 2^100
```

CODE EXPLAINED

- → `math` — float64 functions; ints need conversion
- → `rand.IntN(n)` — `[0, n)`
- → `rand.N` — generic over integer types
- → `rand.NewPCG(…)` — deterministic generator for tests/sims
- → `cryptorand.Text()` — secure token (import alias)

COMING FROM…

- **Python** — `random.randint` → `rand.IntN`
- **Python** — `secrets` → `crypto/rand`
- **JS** — `Math.random()` → `rand.Float64()`

MATH FUNCTIONS

FUNC	EXAMPLE
`Floor / Ceil / Round`	rounding
`Trunc`	drop fraction
`Hypot(3, 4)`	5
`Log2 / Log10`	logs
`Inf / NaN / IsNaN`	special values

RAND/V2 VS V1

V1	V2
`rand.Intn`	rand.IntN
`rand.Seed(…)`	auto-seeded (gone)
`rand.Int63n`	rand.Int64N
—	`rand.N[T]` generic
`rand.NewSource`	NewPCG / NewChaCha8

⚠ COMMON MISTAKES

- Generating session tokens or passwords with `math/rand`; it's predictable
- `rand.IntN(0)` panics; guard against empty ranges
- Using `math.Max` on ints with conversions; use the `max` builtin

✓ TIPS & BEST PRACTICES

- Seed a local generator in tests for reproducible randomized data
- Add jitter to retries: `base + rand.N(base/2)`
- Use `big.Rat` for exact fractions and `big.Int` for huge integers

QUICK SUMMARY

- `math` — float functions & constants
- `rand.IntN` / `Float64` / `Shuffle` — v2 API
- `rand.N` — generic random (durations too)
- `crypto/rand` — security-grade randomness

time

time.Now / Since / Until · time.Duration (5 time.Second) · reference layout "2006-01-02 15:04:05" · Format / Parse / time.RFC3339 · Add / Sub / AddDate · LoadLocation / UTC · Timer / Ticker / time.After · monotonic clock

TIME & DURATIONS

`time.Time` is an instant with a location; `time.Duration` is an `int64` count of nanoseconds. Formatting uses a **reference time** instead of %Y–%m–%d: write how **Mon Jan 2 15:04:05 MST 2006** would look in your format (01/02 03:04:05PM '06 -0700). `time.Now()` includes a **monotonic clock** reading, so `time.Since` is safe from wall-clock jumps.

NOTES

- Compare times with `Before`, `After`, `Equal`, not `==`
- Store and transmit times in UTC / RFC 3339
- `time.Time{}` zero value is year 1; test with `t.IsZero()`
- Import `_ "time/tzdata"` to embed time zones in the binary

WORKING WITH TIME

```
now := time.Now()
fmt.Println(now.Format("2006-01-02 15:04:05")) // 2026-10-16 14:30:00
now.Format(time.RFC3339)                       // 2026-10-16T14:30:00-05:00
now.Format("Jan 2, 2006 3:04 PM")              // Oct 16, 2026 2:30 PM

t, err := time.Parse("2006-01-02", "2026-12-25")
loc, _ := time.LoadLocation("America/Chicago")
t2, _ := time.ParseInLocation("2006-01-02 15:04", "2026-12-25 09:00", loc)

d := 90 * time.Minute                  // Duration
fmt.Println(d, d.Hours())              // 1h30m0s 1.5
later := now.Add(d)
tomorrow := now.AddDate(0, 0, 1)
age := now.Sub(t)                      // Duration between
start := time.Now(); work(); fmt.Println(time.Since(start))

dd, _ := time.ParseDuration("1h15m30s")
now.Truncate(time.Hour); now.Weekday(); now.YearDay()

timer := time.NewTimer(2 * time.Second) // fires once: <-timer.C
ticker := time.NewTicker(time.Second)   // repeats; defer ticker.Stop()
```

CODE EXPLAINED

- → layout strings — use the 2006-01-02 15:04:05 reference
- → `time.Parse` — UTC unless the layout has a zone
- → `ParseInLocation` — interpret in a named zone
- → `Add` / `Sub` / `AddDate` — arithmetic
- → `Timer` / `Ticker` — one-shot / repeating events

COMING FROM…

- **Python** — `strftime('%Y-%m-%d')` → `Format("2006-01-02")`
- **Java** — `Duration` ≈ `time.Duration`
- **JS** — `Date.now()` → `time.Now()`

LAYOUT TOKENS

TOKEN	MEANS
`2006 / 06`	year
`01 / Jan / January`	month
`02 / _2 / Mon`	day / weekday
`15 / 03 / PM`	hour 24h / 12h
`04 / 05 / .000`	min / sec / ms
`MST / -0700 / Z07:00`	zone

DURATIONS

EXPR	VALUE
`time.Millisecond`	1ms
`2 * time.Second`	2s
`d.Seconds()`	float64
`d.Round(time.Second)`	rounded
`time.Duration(n) * time.Second`	int → duration

⚠ COMMON MISTAKES

- Using `"YYYY-MM-DD"` layouts; Go needs `"2006-01-02"`
- `time.Sleep(5)` sleeps 5 **nanoseconds**; write `5 * time.Second`
- Comparing times with `==`; locations and monotonic readings differ, use `Equal`

✓ TIPS & BEST PRACTICES

- Use the predefined layouts: `time.RFC3339`, `time.DateOnly`, `time.TimeOnly`, `time.DateTime`
- Inject a clock (`func() time.Time`) for testable time-dependent code
- Since Go 1.23, unstopped Timers/Tickers are garbage-collected, but still `Stop` them

QUICK SUMMARY

- `time.Now` / `Since` / `Until` — current & elapsed
- `Format` / `Parse` — reference layout 2006-01-02
- `Duration` — `5 * time.Second`
- `Timer` / `Ticker` — scheduling

os, exec & Signals

OS, EXEC & SIGNALS

The `os` package exposes the process environment: arguments, environment variables, working directory, exit codes. `os/exec` runs **external commands** without a shell, and `os/signal` lets you react to Ctrl-C and SIGTERM, usually by canceling a context for a **graceful shutdown**.

NOTES

- `os.Exit` skips deferred functions
- `exec.Command` does **not** use a shell: no globs, pipes or `$VAR`
- `exec.LookPath` finds a binary on `PATH`
- `runtime.GOOS` / `GOARCH` tell you the platform

PROCESS & ENVIRONMENT

```
fmt.Println(os.Args[0], os.Args[1:])     // program, args
port := os.Getenv("PORT")                // "" if unset
if v, ok := os.LookupEnv("DEBUG"); ok { fmt.Println("debug:", v) }
os.Setenv("MODE", "test")
host, _ := os.Hostname()
wd, _ := os.Getwd()

out, err := exec.Command("git", "rev-parse", "HEAD").Output()
fmt.Println(strings.TrimSpace(string(out)))

cmd := exec.CommandContext(ctx, "ffmpeg", "-i", in, outFile)
cmd.Stdout, cmd.Stderr = os.Stdout, os.Stderr
cmd.Env = append(os.Environ(), "LANG=C")
if err := cmd.Run(); err != nil {
    var ee *exec.ExitError
    if errors.As(err, &ee) { fmt.Println("exit:", ee.ExitCode()) }
}

ctx, stop := signal.NotifyContext(context.Background(),
    os.Interrupt, syscall.SIGTERM)       // Ctrl-C / kill
defer stop()
<-ctx.Done()                             // wait, then clean up
fmt.Println(runtime.GOOS, runtime.NumCPU()) // darwin 10
```

CODE EXPLAINED

- → `os.LookupEnv` — distinguishes unset from empty
- → `Output()` — run and capture stdout
- → `CommandContext` — kill the process when ctx is done
- → `*exec.ExitError` — non-zero exit details
- → `signal.NotifyContext` — ctx canceled on signal

COMING FROM…

- **Python** — `sys.argv` → `os.Args`
- **Python** — `subprocess.run` → `exec.Command(...).Run`
- **Node** — `process.on('SIGTERM')` → `signal.NotifyContext`

EXEC.CMD METHODS

METHOD	DOES
`Run()`	start + wait
`Output()`	stdout as []byte
`CombinedOutput()`	stdout + stderr
`Start() / Wait()`	async
`StdoutPipe()`	stream output

EXIT CODES

CODE	CONVENTION
`0`	success
`1`	general error (`log.Fatal`)
`2`	usage error / panic exit
`130`	terminated by Ctrl-C

⚠ COMMON MISTAKES

- `exec.Command("ls *.go")` : treats the whole string as the program name. Split args, glob in Go
- Passing user input to `exec.Command("sh", "-c", input)` : command injection
- Calling `os.Exit` in library code; return an error instead

✓ TIPS & BEST PRACTICES

- Keep `main` as `if err := run(); err != nil { fmt.Fprintln(os.Stderr, err); os.Exit(1) }`
- Use `exec.CommandContext` with a timeout for external tools
- Read config from env with defaults: `cmp.Or(os.Getenv("PORT"), "8080")`

QUICK SUMMARY

- `os.Args` / `Getenv` / `LookupEnv` — inputs
- `exec.Command(…).Output()` — run programs
- `signal.NotifyContext` — graceful shutdown
- `os.Exit(code)` — skips defers

Command-Line Flags

COMMAND-LINE FLAGS

The `flag` package parses `–name value` style options. Define flags (each returns a pointer, or binds to a variable with `…Var`), call `flag.Parse()`, then read the remaining positional arguments with `flag.Args()`. `–h` / `–help` prints auto-generated usage. For **subcommands**, create one `flag.FlagSet` per command.

SYNTAX ACCEPTED

- `-port 8080`, `-port=8080`, `--port=8080` all work
- Booleans: `-v` or `-v=false` (not `-v false`)
- Parsing stops at the first non-flag argument or `--`
- For GNU-style `-abc` short flags use `spf13/pflag` or Cobra

DEFINING FLAGS

```
var (
    port    = flag.Int("port", 8080, "listen port")
    verbose = flag.Bool("v", false, "verbose output")
    timeout = flag.Duration("timeout", 5*time.Second, "timeout")
    name    string; tags []string
)

func main() {
    flag.StringVar(&name, "name", "world", "who to greet")
    flag.Func("tag", "repeatable tag", func(s string) error {
        tags = append(tags, s); return nil
    })
    flag.Usage = usage          // custom -h; calls flag.PrintDefaults()
    flag.Parse()
    fmt.Println(*port, *verbose, *timeout, name, tags, flag.Args())
}
// $ greet -port=9000 -v -tag a -tag b -timeout 2s x.txt
// 9000 true 2s world [a b] [x.txt]

serve := flag.NewFlagSet("serve", flag.ExitOnError) // subcommand
addr := serve.String("addr", ":80", "address")
serve.Parse(os.Args[2:])
```

CODE EXPLAINED

- → `flag.Int(…)` — returns `*int`; dereference after `Parse`
- → `flag.Duration` — parses `2s`, `1h30m`
- → `flag.Func` — custom parsing, repeatable flags
- → `flag.Usage` — customize `-h` output
- → `NewFlagSet` — separate flags per subcommand

COMING FROM…

- **Python** — `argparse` ≈ `flag`, simpler
- **Node** — ≈ `commander` / `yargs` basics
- **C** — ≈ `getopt`, single-dash style

FLAG TYPES

FUNC	VALUE
`String / StringVar`	string
`Int / Int64 / Uint`	integers
`Float64`	float
`Bool`	bool
`Duration`	time.Duration
`TextVar`	any TextUnmarshaler

HELPERS

FUNC	RETURNS
`flag.Args()`	positional args
`flag.NArg()`	count
`flag.Arg(i)`	i-th positional
`flag.NFlag()`	flags set
`flag.Visit(fn)`	only set flags

⚠ COMMON MISTAKES

- Reading `*port` before `flag.Parse()`; you get the default
- Putting flags after positional args (`app file.txt -v`); `-v` becomes a positional arg
- `-v false` for booleans; it sets `true` and treats `false` as an argument

✓ TIPS & BEST PRACTICES

- Use `flag.TextVar` for types like `net.IP` or `slog.Level`
- Combine flags with env fallbacks for 12-factor apps
- Move to Cobra (sheet 082) when you have many nested subcommands

QUICK SUMMARY

- `flag.Int/String/Bool` — define (pointers)
- `flag.Parse()` — then read values
- `flag.Args()` — positional args
- `flag.NewFlagSet` — subcommands

Logging: log & slog

LOGGING: LOG & SLOG

The classic `log` package writes timestamped lines to stderr; `log.Fatal` also exits. **`log/slog`** (Go 1.21) is the standard **structured logger**: each record has a level, a message and key-value **attributes**, rendered by a handler as text or JSON. Use `slog` for anything that runs in production.

NOTES

- Levels: `Debug` (-4), `Info` (0), `Warn` (4), `Error` (8)
- `slog.SetDefault` also routes the old `log` package through slog
- Typed attrs (`slog.Int` , `slog.String`) avoid allocations
- `LogValuer` controls how a type is logged (redact secrets)

LOG AND SLOG

```
log.SetFlags(log.LstdFlags | log.Lshortfile)
log.SetPrefix("[api] ")
log.Println("starting")  // [api] 2026/10/16 14:30:00 main.go:12: …
// log.Fatalf("bad config: %v", err) // logs, then os.Exit(1)

slog.Info("user login", "user", "ann", "attempts", 3)
// time=… level=INFO msg="user login" user=ann attempts=3

h := slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level:     slog.LevelDebug,
    AddSource: true,
})
logger := slog.New(h)
slog.SetDefault(logger)

reqLog := logger.With("request_id", rid)   // child logger
reqLog.Warn("slow query", slog.Duration("took", d),
    slog.Group("db", "table", "users", "rows", 120))
reqLog.ErrorContext(ctx, "save failed", "err", err)

type Token string
func (Token) LogValue() slog.Value { return slog.StringValue("REDACTED") }
```

CODE EXPLAINED

- → `log.Lshortfile` — add file:line to each line
- → `slog.Info(msg, k, v, …)` — alternating key-value pairs
- → `NewJSONHandler` — machine-readable output
- → `logger.With(…)` — attach attrs to every later record
- → `LogValue` — custom/redacted rendering

COMING FROM…

- **Python** — `logging` ≈ `log/slog`
- **Java** — SLF4J + Logback ≈ `slog` + handlers
- **JS** — pino / winston JSON ≈ `JSONHandler`

SLOG API

CALL	PURPOSE
`slog.Debug/Info/Warn/Error`	log at level
`…Context(ctx, …)`	pass ctx to handler
`slog.With(attrs…)`	child logger
`slog.Group(name, …)`	nested attrs
`slog.SetLogLoggerLevel`	level for `log` pkg

HANDLERS

HANDLER	OUTPUT
`TextHandler`	key=value lines
`JSONHandler`	one JSON object/line
`DiscardHandler`	drop all (1.24)
`custom`	implement `slog.Handler`

⚠ COMMON MISTAKES

- Odd number of key-value args: you get `!BADKEY` in the output (`go vet` checks this)
- `log.Fatal` inside libraries or goroutines; it kills the process and skips defers
- Logging secrets (tokens, passwords) as attributes; implement `LogValuer`

✓ TIPS & BEST PRACTICES

- Create one logger in `main` and pass it (or ctx) down; avoid globals in libraries
- Use JSON in production, Text in local dev
- Use `slog.LevelVar` to change log level at runtime

QUICK SUMMARY

- `log` — simple timestamped lines
- `slog` — structured, leveled logging
- `slog.With` — contextual child loggers
- `JSONHandler` / `TextHandler` — output format

Hashing, Crypto & UUIDs

HASHING, CRYPTO & UUIDS

Go's standard library includes production-grade cryptography: hashes (`crypto/sha256`), MACs (`crypto/hmac`), secure randomness (`crypto/rand`), TLS and more. Passwords need a slow hash like **bcrypt** or **argon2** from `golang.org/x/crypto`. **Go 1.27 added a `uuid` package** for generating and parsing UUIDs, replacing the most common third-party dependency.

NOTES

- Never use `md5` / `sha1` for security; fine for checksums
- `hash/fnv`, `hash/maphash` are fast **non-crypto** hashes
- Compare secrets with `subtle.ConstantTimeCompare` or `hmac.Equal`
- Go 1.27 adds `crypto/mldsa` (post-quantum signatures)

COMMON CRYPTO TASKS

```
sum := sha256.Sum256([]byte("hello"))       // [32]byte
fmt.Println(hex.EncodeToString(sum[:]))     // 2cf24dba…

h := sha256.New()                           // streaming
io.Copy(h, file)
digest := h.Sum(nil)

mac := hmac.New(sha256.New, secretKey)      // sign a webhook
mac.Write(body)
valid := hmac.Equal(mac.Sum(nil), gotSig)   // constant time

key := make([]byte, 32)
crand.Read(key)                             // crypto/rand
tok := crand.Text()                         // base32 token (1.24)

hash, _ := bcrypt.GenerateFromPassword([]byte(pw), bcrypt.DefaultCost)
err := bcrypt.CompareHashAndPassword(hash, []byte(pw)) // nil = ok

f := fnv.New64a(); f.Write([]byte("key")); f.Sum64() // fast non-crypto
id := uuid.NewV7()                          // uuid pkg (1.27), time-ordered
u, err := uuid.Parse("0b7c…")               // UUID, error; id.String()
```

CODE EXPLAINED

- → `sha256.Sum256` — one-shot digest as an array
- → `h.Sum(nil)` — finish a streaming hash
- → `hmac.Equal` — avoids timing attacks
- → `bcrypt` — salted, slow password hashing (x/crypto)
- → `fnv` — fast hashing for sharding/buckets

COMING FROM…

- **Python** — `hashlib.sha256` → `sha256.Sum256`
- **Python** — `uuid.uuid4()` → `uuid.New()` (1.27)
- **Node** — `crypto.randomBytes` → `crypto/rand.Read`

PICK THE RIGHT TOOL

NEED	USE
file checksum	crypto/sha256
API signature	crypto/hmac
password storage	x/crypto/bcrypt / argon2
tokens / keys	crypto/rand
unique IDs	uuid.New / NewV7 (1.27)
hash map / sharding	hash/maphash, fnv

ENCODING DIGESTS

FUNC	OUTPUT
hex.EncodeToString	64 hex chars (sha256)
base64.StdEncoding	44 chars
base64.RawURLEncoding	43, URL-safe
fmt.Sprintf("%x", sum)	hex

⚠ COMMON MISTAKES

- Storing passwords as SHA-256 hashes; fast hashes are easy to brute-force
- Comparing MACs with `==` / `bytes.Equal`; use `hmac.Equal`
- Seeding `math/rand` with time for tokens; use `crypto/rand`

✓ TIPS & BEST PRACTICES

- Prefer high-level APIs (`crypto/tls`, `x/crypto/nacl/secretbox`) over raw primitives
- Check the 1.27 `uuid` package before adding a third-party UUID library
- Hash files with `io.Copy(h, f)` so large files stream

QUICK SUMMARY

- `sha256.Sum256` / `New` — hashing
- `hmac` + `hmac.Equal` — message auth
- `crypto/rand` — secure random bytes
- `bcrypt` — passwords; `uuid` — IDs (1.27)

testify

TESTIFY

`github.com/stretchr/testify` is the most widely used Go testing helper library. `assert` functions report a failure and **keep going**; `require` functions report and **stop the test** immediately. It also provides test `suites` with setup/teardown and a `mock` package for hand-written mocks.

NOTES

- Argument order is `(t, expected, actual)`
- Use `require` for preconditions (no point continuing on nil)
- Every assertion accepts an optional message + args at the end
- `assert.New(t)` saves passing `t` repeatedly

ASSERTIONS, SUITES & MOCKS

```
func TestParse(t *testing.T) {
    cfg, err := Parse("port=80")
    require.NoError(t, err)                  // stop if err
    require.NotNil(t, cfg)
    assert.Equal(t, 80, cfg.Port)
    assert.ElementsMatch(t, []int{1, 2}, []int{2, 1}) // any order
    _, err = Parse("port=x")
    assert.ErrorIs(t, err, ErrBadPort)
    assert.Contains(t, err.Error(), "port")
}

type MockStore struct{ mock.Mock }
func (m *MockStore) Get(id int) (string, error) {
    args := m.Called(id)
    return args.String(0), args.Error(1)
}
st := new(MockStore)
st.On("Get", 7).Return("ann", nil).Once()
st.AssertExpectations(t)
```

CODE EXPLAINED

- → `require.NoError` — fail now; later lines would panic
- → `assert.Equal(t, want, got)` — prints a diff on failure
- → `ElementsMatch` — compare ignoring order
- → `ErrorIs` — wraps `errors.Is`
- → `mock.Mock` + `On(…).Return(…)` — scripted behavior

COMING FROM…

- **Python** — ≈ `pytest` asserts / `unittest.mock`
- **Java** — ≈ JUnit `assertEquals` + Mockito
- **JS** — ≈ Jest `expect` + mocks

COMMON ASSERTIONS

FUNC	CHECKS
`Equal / NotEqual`	deep equality
`NoError / Error`	err nil / non-nil
`ErrorIs / ErrorAs`	wrapped errors
`Nil / NotNil / Empty`	values
`True / False`	booleans

PACKAGES

PACKAGE	PURPOSE
`assert`	soft checks
`require`	hard checks (FailNow)
`suite`	SetupTest/TearDownTest
`mock`	mock objects

⚠ COMMON MISTAKES

- Swapping expected and actual; failure messages become misleading
- Using `assert.NoError` then dereferencing the result; use `require`
- Calling `require` from a goroutine other than the test's; `FailNow` must run on the test goroutine

✓ TIPS & BEST PRACTICES

- Prefer table-driven tests + `assert`; reach for `suite` only for heavy shared setup
- Generate mocks with `mockery` instead of writing them by hand
- Plain `if got != want { t.Errorf(…) }` is fine; testify is optional

QUICK SUMMARY

- `assert` — record failure, continue
- `require` — stop the test
- `(t, expected, actual)` — argument order
- `mock` / `suite` — mocks & fixtures

Cobra & Viper (CLIs)

COBRA & VIPER

Cobra (`spf13/cobra`) builds CLIs with nested subcommands, POSIX-style flags, help text and shell completion; it powers `kubectl`, `gh` and `hugo`. **Viper** (`spf13/viper`) loads configuration from files, environment variables and flags with a single lookup API. They're often used together.

NOTES

- Each command is a `*cobra.Command`; `AddCommand` nests them
- `PersistentFlags` are inherited by subcommands
- Use `RunE` to return errors instead of calling `os.Exit`
- `cobra-cli init` / `add` scaffold a project

A COBRA CLI WITH VIPER

```
var rootCmd = &cobra.Command{Use: "shop", Short: "Shop CLI"}

var serveCmd = &cobra.Command{
    Use:   "serve",
    Short: "Start the HTTP server",
    RunE: func(cmd *cobra.Command, args []string) error {
        port := viper.GetInt("port")
        return runServer(port)
    },
}

func init() {
    rootCmd.PersistentFlags().String("config", "", "config file")
    serveCmd.Flags().IntP("port", "p", 8080, "listen port")
    viper.BindPFlag("port", serveCmd.Flags().Lookup("port"))
    viper.SetEnvPrefix("SHOP"); viper.AutomaticEnv() // SHOP_PORT
    rootCmd.AddCommand(serveCmd)
}

func main() {
    if err := rootCmd.Execute(); err != nil { os.Exit(1) }
}
```

CODE EXPLAINED

- → `Use` / `Short` — usage line and help summary
- → `RunE` — command body that returns an error
- → `IntP("port", "p", …)` — `--port` and `-p`
- → `BindPFlag` + `AutomaticEnv` — flag, env or config file
- → `Execute()` — parses args and dispatches

COMING FROM…

- **Python** — Cobra ≈ Click / Typer
- **Node** — ≈ `commander` with subcommands
- **Python** — Viper ≈ `dynaconf`

COBRA PIECES

FIELD / FUNC	PURPOSE
`Use, Short, Long`	help text
`Args: cobra.ExactArgs(1)`	validate positionals
`PreRunE / PostRunE`	hooks
`Flags() / PersistentFlags()`	local / inherited
`GenBashCompletion…`	shell completion

VIPER LOOKUP ORDER

PRIORITY	SOURCE
1	explicit `viper.Set`
2	flag
3	environment variable
4	config file
5	default (`SetDefault`)

⚠ COMMON MISTAKES

- Using `Run` and calling `os.Exit` inside; tests can't capture errors
- Reading flag variables in `init()` before `Execute` has parsed them
- Expecting `AutomaticEnv` to map `server.port` to `SERVER_PORT` without `SetEnvKeyReplacer`

✓ TIPS & BEST PRACTICES

- Set `SilenceUsage: true` on root so runtime errors don't print the usage text
- Use `cmd.OutOrStdout()` so command output can be captured in tests
- For simple tools, the stdlib `flag` package is enough

QUICK SUMMARY

- `cobra.Command` — commands + subcommands
- `RunE` — return errors
- `viper` — config from flags/env/files
- `BindPFlag` / `AutomaticEnv` — wiring

Web Routers: chi, Gin & Echo

WEB ROUTERS

Since Go 1.22 the standard `http.ServeMux` supports **methods and path wildcards** (`GET /items/{id}`), so many services need no router library. Third-party routers still add conveniences: **chi** stays `net/http`-compatible with middleware groups; **Gin** and **Echo** use their own context type with binding, validation and rendering helpers.

NOTES

- chi handlers are plain `http.HandlerFunc` s
- Gin/Echo handlers take `*gin.Context` / `echo.Context`
- All support middleware chains and route groups
- Stdlib first; add a router when you need groups or binding

SAME ROUTE, THREE ROUTERS

```
// chi (github.com/go-chi/chi/v5)
r := chi.NewRouter()
r.Use(middleware.Logger, middleware.Recoverer)
r.Route("/api", func(r chi.Router) {
    r.Get("/users/{id}", func(w http.ResponseWriter, q *http.Request) {
        fmt.Fprint(w, chi.URLParam(q, "id"))
    })
})
http.ListenAndServe(":8080", r)

// Gin (github.com/gin-gonic/gin)
g := gin.Default()
g.GET("/api/users/:id", func(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"id": c.Param("id")})
})
g.POST("/api/users", func(c *gin.Context) {
    var u User
    if err := c.ShouldBindJSON(&u); err != nil { c.AbortWithStatus(400) }
})
g.Run(":8080")

// Echo: e.GET("/users/:id", func(c echo.Context) error {
//     return c.JSON(200, map[string]string{"id": c.Param("id")}) })
```

CODE EXPLAINED

- → chi `Route` — nested route groups
- → `chi.URLParam` — like stdlib `r.PathValue`
- → Gin `:id` path params + `c.Param`
- → `ShouldBindJSON` — decode + validate tags
- → Echo handlers return `error`

COMING FROM…

- **Python** — Gin ≈ Flask; chi ≈ minimal Flask
- **Node** — Echo / Gin ≈ Express
- **Java** — ≈ Spring MVC routing, without annotations

COMPARISON

ROUTER	STYLE
net/http ServeMux	stdlib, GET /x/{id}
chi	net/http-compatible
Gin	own context, fast, popular
Echo	own context, error returns
Fiber	fasthttp-based (not net/http)

PATH PARAMS

ROUTER	SYNTAX / READ
stdlib	{id} / r.PathValue("id")
chi	{id} / chi.URLParam(r, "id")
Gin	:id / c.Param("id")
Echo	:id / c.Param("id")

⚠ COMMON MISTAKES

- Picking Fiber and then finding `net/http` middleware and libraries don't plug in
- Forgetting `return` after writing an error response in Gin handlers
- Adding a router just for path params; Go 1.22's `ServeMux` already does it

✓ TIPS & BEST PRACTICES

- Start with stdlib `ServeMux`; move to chi for groups/middleware without lock-in
- Keep business logic out of handlers so switching routers is cheap
- Use Gin's `binding:"required"` tags for quick request validation

QUICK SUMMARY

- stdlib `ServeMux` — methods + wildcards (1.22)
- chi — net/http-compatible groups
- Gin / Echo — rich context APIs
- choose by ecosystem fit

Sheet 084

Database Libraries: pgx, sqlx, sqlc & GORM

pgxpool.New · pgx.CollectRows · sqlx Get / Select / db: tags · sqlc generated queries · GORM First / Create / Where · migrations (goose, golang-migrate) · choosing an approach

DATABASE LIBRARIES

Go's `database/sql` (sheet 087) is low-level. Popular libraries sit on top or beside it: **pgx** is the native PostgreSQL driver with pooling and rich types; **sqlx** adds struct scanning to `database/sql`; **sqlc** generates type-safe Go from SQL queries; **GORM** is a full ORM. Migrations are usually handled by **goose** or **golang-migrate**.

CHOOSING

- Like writing SQL? sqlc or sqlx
- PostgreSQL only? use pgx directly (`pgxpool`)
- Want an ORM with associations? GORM
- Always use placeholders, never string-built SQL

FOUR APPROACHES

```go
// pgx (github.com/jackc/pgx/v5)
pool, _ := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
rows, _ := pool.Query(ctx, "SELECT id, name FROM users WHERE age > $1", 18)
users, err := pgx.CollectRows(rows, pgx.RowToStructByName[User])

// sqlx (github.com/jmoiron/sqlx)
db := sqlx.MustConnect("postgres", dsn)
var u User                                  // `db:"name"` tags
err = db.Get(&u, "SELECT * FROM users WHERE id = $1", 7)
var all []User
err = db.Select(&all, "SELECT * FROM users ORDER BY id")

// sqlc: write SQL, generate Go
// -- name: GetUser :one
// SELECT * FROM users WHERE id = $1;
q := db.New(pool)
user, err := q.GetUser(ctx, 7)              // generated, typed

// GORM (gorm.io/gorm)
gdb, _ := gorm.Open(postgres.Open(dsn), &gorm.Config{})
gdb.Create(&User{Name: "Ann"})
gdb.Where("age > ?", 18).Order("name").Find(&all)
```

CODE EXPLAINED

- → `pgxpool` — concurrent connection pool
- → `RowToStructByName` — map columns to fields
- → `db.Get` / `db.Select` — one row / many rows
- → sqlc — compile-time checked queries
- → GORM — chainable query builder + models

COMING FROM…

- **Python** — GORM ≈ SQLAlchemy ORM
- **Python** — sqlx ≈ raw SQL with row mapping
- **Java** — sqlc ≈ jOOQ codegen; GORM ≈ Hibernate

LIBRARIES

LIBRARY	TYPE
pgx	Postgres driver + toolkit
sqlx	database/sql extension
sqlc	code generator
GORM / ent / Bun	ORMs
squirrel	SQL builder

MIGRATIONS

TOOL	COMMAND
goose	`goose up` / `goose create x sql`
golang-migrate	`migrate -path db -database $URL up`
atlas	declarative schema diff
GORM	`AutoMigrate` (dev only)

⚠ COMMON MISTAKES

- Building SQL with `fmt.Sprintf` and user input: SQL injection
- Relying on GORM `AutoMigrate` in production instead of versioned migrations
- Mixing `?` and `$1` placeholders; Postgres needs `$1` , MySQL `?`

✓ TIPS & BEST PRACTICES

- sqlc + pgx is a strong default for Postgres services
- Pass `ctx` to every query so requests can cancel slow work
- Embed migration files with `//go:embed` and run them at startup

QUICK SUMMARY

- pgx — native Postgres, pooling
- sqlx — struct scanning
- sqlc — SQL → typed Go
- GORM — ORM; goose/migrate — migrations

Regular Expressions

RE2 syntax (no backrefs or lookaround) · regexp.MustCompile · MatchString · FindString / FindAllString · FindStringSubmatch · named groups (?P<name>…) / SubexpNames · ReplaceAllString / ReplaceAllStringFunc · raw-string patterns

REGULAR EXPRESSIONS

Go's `regexp` package uses the **RE2** engine, which guarantees **linear-time** matching: no catastrophic backtracking, so it's safe on untrusted input. The trade-off is **no backreferences and no lookaround**. Compile once with `regexp.MustCompile` at package level and reuse the `*Regexp`; it's safe for concurrent use.

NOTES

- Use **raw strings** for patterns: `` `\d+` ``
- `Find…` returns the leftmost match; `FindAll…` takes a limit `n` (-1 = all)
- `Submatch` variants return capture groups; `Index` variants return positions
- `(?i)` case-insensitive, `(?m)` multiline, `(?s)` dot matches newline

REGEXP IN PRACTICE

```
var reDate = regexp.MustCompile(`(?P<year>\d{4})-(?P<month>\d{2})-(\d{2})`)

reDate.MatchString("due 2026-10-16")              // true
reDate.FindString("a 2026-10-16 b 2027-01-01")    // "2026-10-16"
reDate.FindAllString("2026-10-16 2027-01-01", -1) // [both]

m := reDate.FindStringSubmatch("2026-10-16")
fmt.Println(m[0], m[1], m[3])                     // 2026-10-16 2026 16
i := reDate.SubexpIndex("month")
fmt.Println(m[i])                                 // 10

reDate.ReplaceAllString("2026-10-16", "${month}/$3/${year}") // 10/16/2026
re := regexp.MustCompile(`\s+`)
re.ReplaceAllString("a   b \t c", " ")            // "a b c"
re.Split("a  b c", -1)                            // [a b c]

upper := regexp.MustCompile(`[a-z]+`).ReplaceAllStringFunc("go is fun",
    strings.ToUpper)                              // GO IS FUN

re2, err := regexp.Compile(userPattern)           // check user input
```

CODE EXPLAINED

- → `(?P<year>…)` — named group
- → `FindStringSubmatch` — `[full, g1, g2, …]`
- → `SubexpIndex("month")` — group index by name
- → `${month}` / `$3` — references in replacements
- → `Compile` — returns error for bad patterns (user input)

COMING FROM…

- **Python / JS** — no lookahead or backreferences (RE2)
- **Python** — `re.compile` → `regexp.MustCompile`
- **Java** — named groups `(?P<n>…)` (Python syntax)

SYNTAX

PATTERN	MATCHES
`\d \w \s`	digit, word char, space
`[a-z] [^0-9]`	class / negated
`* + ? {2,4}`	quantifiers
`*? +?`	lazy quantifiers
`^ $ \b`	anchors, word boundary
`(…) (?:…) (?P<n>…)`	groups

METHODS

METHOD	RETURNS
`MatchString`	bool
`FindString / FindAllString`	string / []string
`FindStringSubmatch`	[]string groups
`FindStringIndex`	[]int positions
`ReplaceAllString(Func)`	string
`Split(s, n)`	[]string

⚠ COMMON MISTAKES

- Compiling inside a loop or handler; compile once at package level
- Porting patterns with lookahead `(?=…)` or backrefs `\1`; RE2 rejects them
- Forgetting anchors: `MatchString("\d+")` is true for `"abc1"`; use `^\d+$`

✓ TIPS & BEST PRACTICES

- Prefer `strings` functions for fixed substrings; they're much faster
- Use the `[]byte` methods (`Find`, `ReplaceAll`) to avoid conversions on byte data
- Test patterns at regex101.com with the **Golang** flavor

QUICK SUMMARY

- RE2 — linear time, no backrefs/lookaround
- `MustCompile` once — reuse safely
- `Find…Submatch` — capture groups
- `ReplaceAllString` — `$1` / `${name}`

Reflection

REFLECTION

The `reflect` package inspects types and values at runtime: it's how `encoding/json`, `fmt` and ORMs work. `reflect.TypeOf` describes a type (name, fields, tags); `reflect.ValueOf` wraps a value you can read and, if it's **addressable**, set. Reflection is slow and loses compile-time safety, so use it only for generic infrastructure.

LAWS OF REFLECTION

- Interface value → reflection object (`ValueOf`)
- Reflection object → interface value (`v.Interface()`)
- To modify, the value must be **settable**: pass a pointer + `Elem()`
- `Kind` is the category (`Struct` , `Ptr` …); `Type` is the exact type

INSPECTING STRUCTS

```
type User struct {
    Name  string `json:"name" validate:"required"`
    Age   int    `json:"age"`
    email string
}

u := User{Name: "Ann", Age: 31}
t := reflect.TypeOf(u)
fmt.Println(t.Name(), t.Kind(), t.NumField()) // User struct 3

for i := range t.NumField() {
    f := t.Field(i)
    fmt.Println(f.Name, f.Type, f.Tag.Get("json"), f.IsExported())
}   // Name string name true / Age int age true / email string  false

v := reflect.ValueOf(&u).Elem()               // settable
v.FieldByName("Age").SetInt(32)
fmt.Println(u.Age, v.CanSet())                // 32 true

rt := reflect.TypeFor[map[string]int]()       // 1.22, no value needed
fmt.Println(rt.Key(), rt.Elem())              // string int
reflect.DeepEqual([]int{1}, []int{1})         // true
```

CODE EXPLAINED

- → `TypeOf(u)` — type metadata
- → `f.Tag.Get("json")` — read struct tags
- → `ValueOf(&u).Elem()` — addressable, so fields can be set
- → `SetInt` — panics if not settable or wrong kind
- → `reflect.TypeFor[T]()` — type without a value (1.22)

COMING FROM…

- **Java** — `reflect` ≈ `java.lang.reflect`
- **Python** — ≈ `getattr` / `type()` , but much stricter
- **C#** — ≈ `System.Reflection` ; struct tags ≈ attributes

TYPE METHODS

METHOD	GIVES
`Name() / String()`	type name
`Kind()`	Struct, Slice, Ptr…
`NumField() / Field(i)`	struct fields
`Elem()`	pointee / element type
`NumMethod() / Method(i)`	methods

VALUE METHODS

METHOD	GIVES
`Interface()`	back to any
`Int() / String() / Bool()`	typed read
`SetInt / SetString / Set`	write
`CanSet() / CanAddr()`	writability
`IsNil() / IsZero()`	checks

⚠ COMMON MISTAKES

- Setting fields on `reflect.ValueOf(u)` (a copy): `panic: using unaddressable value`
- Calling `Field(i).Interface()` on an unexported field: panics
- Using reflection where generics or an interface would be type-safe and faster

✓ TIPS & BEST PRACTICES

- Cache `reflect.Type` field info per type when processing many values
- Check `Kind()` before calling kind-specific methods
- Prefer `reflect.TypeFor[T]()` over `reflect.TypeOf((*T)(nil)).Elem()`

QUICK SUMMARY

- `TypeOf` / `ValueOf` — inspect
- `Tag.Get` — struct tag metadata
- pointer + `Elem()` — settable values
- slow & unsafe — use sparingly

database/sql

sql.Open(driver, dsn) + blank driver import · QueryContext / rows.Next / Scan · QueryRowContext · ExecContext / RowsAffected · ? vs $1 placeholders · BeginTx / Commit / Rollback · sql.Null[T] · pool settings

DATABASE/SQL

`database/sql` is a driver-agnostic API over a **connection pool**. You import a driver for its side effect (`_ "github.com/jackc/pgx/v5/stdlib"`), open a `*sql.DB` once, and share it everywhere. Always use **placeholders** for parameters, always `Close` rows, and pass a `context` so queries can be canceled.

NOTES

- `sql.Open` doesn't connect; call `db.PingContext` to verify
- `*sql.DB` is safe for concurrent use; don't open one per request
- `QueryRow` returns `sql.ErrNoRows` from `Scan` if nothing matched

QUERY, EXEC, TRANSACTIONS

```
db, err := sql.Open("pgx", os.Getenv("DATABASE_URL")) // check err, then:
defer db.Close()
db.SetMaxOpenConns(20); db.SetConnMaxIdleTime(5 * time.Minute)

var name string
err = db.QueryRowContext(ctx, "SELECT name FROM users WHERE id=$1", 7).
    Scan(&name)
if errors.Is(err, sql.ErrNoRows) { return ErrNotFound }

rows, err := db.QueryContext(ctx, "SELECT id, email FROM users")
if err != nil { return err }
defer rows.Close()
for rows.Next() {
    var id int; var email sql.Null[string]   // nullable
    if err := rows.Scan(&id, &email); err != nil { return err }
}
if err := rows.Err(); err != nil { return err }

res, err := db.ExecContext(ctx, "DELETE FROM users WHERE id=$1", 7)
n, _ := res.RowsAffected()

tx, err := db.BeginTx(ctx, nil)
defer tx.Rollback()                        // no-op after Commit
tx.ExecContext(ctx, "INSERT INTO audit(msg) VALUES ($1)", "x")
err = tx.Commit()
```

CODE EXPLAINED

- → `sql.Open("pgx", dsn)` — driver name + data source
- → `QueryRowContext(…).Scan` — single row
- → `defer rows.Close()` + `rows.Err()` — always both
- → `ExecContext` — INSERT/UPDATE/DELETE
- → `defer tx.Rollback()` — safe even after `Commit`

COMING FROM…

- **Python** — ≈ DB-API (`cursor.execute`)
- **Java** — ≈ JDBC `PreparedStatement` + pool
- **Node** — ≈ `pg` pool queries

PLACEHOLDERS

DATABASE	STYLE
PostgreSQL	`$1, $2`
MySQL / SQLite	`?`
SQL Server	`@p1` / `@name`

POOL SETTINGS

METHOD	PURPOSE
SetMaxOpenConns	cap total connections
SetMaxIdleConns	idle pool size
SetConnMaxLifetime	recycle connections
SetConnMaxIdleTime	close idle ones

⚠ COMMON MISTAKES

- Forgetting `rows.Close()` ; connections leak until the pool is exhausted
- Skipping `rows.Err()` ; network errors mid-iteration look like "fewer rows"
- Scanning a NULL into a plain `string` : "converting NULL to string is unsupported"

✓ TIPS & BEST PRACTICES

- Wrap `sql.ErrNoRows` into your own `ErrNotFound` at the repository layer
- Use `Prepare` for statements executed many times in a loop
- Keep transactions short and always pass `ctx`

QUICK SUMMARY

- `sql.Open` once — pooled `*sql.DB`
- `QueryRow` / `Query` / `Exec` (+Context)
- `rows.Close` + `rows.Err` — every time
- `BeginTx` + `defer Rollback` + `Commit`

Sheet 088

Build Tags & Cross-Compilation

//go:build linux && amd64 · _linux.go / _test.go suffixes · GOOS=windows GOARCH=arm64 go build · CGO_ENABLED=0 static binaries · -ldflags "-s -w -X main.version=…" · -trimpath · go generate · debug.ReadBuildInfo

BUILD TAGS & CROSS-COMPILING

Go cross-compiles by setting **GOOS** and **GOARCH**; no extra toolchain is needed for pure-Go code. **Build constraints** (`//go:build` lines and `_os_arch.go` filename suffixes) include or exclude files per platform or custom tag. `-ldflags -X` injects values like a version string at link time.

NOTES

- `//go:build` must be the first line(s), before `package`, followed by a blank line
- Filename suffixes: `_linux.go`, `_windows_amd64.go`, `_test.go`
- `CGO_ENABLED=0` → fully static binary, easy to ship in scratch containers
- `go tool dist list` prints every supported GOOS/GOARCH

CONSTRAINTS

```
//go:build linux && amd64      ← must be line 1

package sys

var version = "dev"            // set via -ldflags -X

// other constraint expressions:
//   //go:build !windows       all but Windows
//   //go:build integration    only with -tags integration
//   //go:build go1.27         only on Go ≥ 1.27
//   //go:build (linux || darwin) && !cgo

//go:generate stringer -type=Color
```

BUILD COMMANDS

```
GOOS=windows GOARCH=amd64 go build -o app.exe .
GOOS=darwin GOARCH=arm64 go build -o app-mac .
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build .

go build -tags integration ./...
go test -tags integration ./...

go build -trimpath \
  -ldflags "-s -w -X main.version=v1.4.0" .

go generate ./...              # run //go:generate
go version -m ./app            # embedded build info
```

COMMON TARGETS

GOOS / GOARCH	PLATFORM
`linux / amd64`	x86-64 servers
`linux / arm64`	Graviton, Raspberry Pi 4+
`darwin / arm64`	Apple Silicon
`windows / amd64`	Windows PCs
`js / wasm, wasip1 / wasm`	WebAssembly

LINKER & BUILD FLAGS

FLAG	EFFECT
`-ldflags "-s -w"`	strip symbols/DWARF (smaller)
`-ldflags "-X pkg.var=v"`	set string var
`-trimpath`	remove local paths
`-race`	race detector build
`-tags a,b`	enable build tags

⚠ COMMON MISTAKES

- Missing the blank line after `//go:build` and `package`; the line is treated as a doc comment
- `-X main.Version=…` on a `const` or a non-string var; it silently does nothing
- Cross-compiling with cgo enabled and getting "C compiler not found"

✓ TIPS & BEST PRACTICES

- Read build info at runtime with `debug.ReadBuildInfo()` (VCS revision, module versions)
- Use GoReleaser to build, archive and publish multi-platform binaries
- Gate slow tests behind `//go:build integration`

QUICK SUMMARY

- `GOOS` / `GOARCH` — cross-compile
- `//go:build expr` — file constraints
- `-ldflags -X` — inject version strings
- `CGO_ENABLED=0` — static binaries

Memory & Garbage Collection

MEMORY & GARBAGE COLLECTION

Go is garbage collected with a concurrent, low-pause collector; **Green Tea**, the default since Go 1.26, scans memory in page-sized batches for better CPU cache use. The compiler's **escape analysis** decides whether values live on the cheap **stack** or the GC-managed **heap**. Tuning knobs are `GOGC` (how much growth triggers a GC) and `GOMEMLIMIT` (a soft memory cap).

NOTES

- Values escape when they outlive the function (returned pointers, closures, interfaces)
- `GOGC=100` (default): collect when heap grows 100% since last GC
- `GOMEMLIMIT=1GiB` makes the GC work harder near the limit (containers)
- `runtime.AddCleanup` (1.24) replaces most `SetFinalizer` uses

ALLOCATION AWARENESS

```
// $ go build -gcflags=-m ./...     → escape analysis report
func sum(xs []int) int { t := 0; for _, x := range xs { t += x }; return t }
                                    // t stays on the stack
func newUser() *User { return &User{} } // &User{} escapes to heap

out := make([]Row, 0, len(in))      // preallocate: 1 allocation
m := make(map[string]int, 1000)     // size hint

var bufPool = sync.Pool{New: func() any { return new(bytes.Buffer) }}
b := bufPool.Get().(*bytes.Buffer)
b.Reset(); defer bufPool.Put(b)     // reuse hot temporary objects

var ms runtime.MemStats
runtime.ReadMemStats(&ms)
fmt.Println(ms.HeapAlloc/1e6, "MB", ms.NumGC, "GCs")

debug.SetMemoryLimit(512 << 20)     // same as GOMEMLIMIT=512MiB
runtime.AddCleanup(f, func(fd int) { syscall.Close(fd) }, f.fd) // 1.24
wp := weak.Make(bigObj)             // weak pointer (1.24)
if obj := wp.Value(); obj != nil { use(obj) }
```

CODE EXPLAINED

- → `-gcflags=-m` — shows what escapes and why
- → preallocation — avoids repeated growth copies
- → `sync.Pool` — reuse buffers under load
- → `ReadMemStats` — heap size and GC counts
- → `weak.Make` — reference that doesn't keep objects alive

COMING FROM…

- **Java** — no JVM heap flags; `GOMEMLIMIT` ≈ `-Xmx` (soft)
- **Python** — no refcounting; tracing GC only
- **C** — no `free`; escape analysis picks stack vs heap

KNOBS

SETTING	EFFECT
`GOGC=100`	default growth target
`GOGC=off`	disable GC (with memory limit)
`GOMEMLIMIT=2GiB`	soft limit
`GODEBUG=gctrace=1`	print each GC
`nogreenteagc`	GOEXPERIMENT opt-out (1.26)

REDUCE ALLOCATIONS

TECHNIQUE	EXAMPLE
`preallocate`	`make([]T, 0, n)`
`reuse buffers`	`sync.Pool`, `buf[:0]`
`avoid boxing`	concrete types over `any`
`strings.Builder`	instead of +=
`value receivers`	small structs

⚠ COMMON MISTAKES

- Optimizing allocations without a profile; measure with `-benchmem` and pprof first
- Putting objects back into `sync.Pool` while still using them
- Running in containers without `GOMEMLIMIT` and getting OOM-killed

✓ TIPS & BEST PRACTICES

- Set `GOMEMLIMIT` to ~90% of the container limit
- Use `testing.AllocsPerRun` or `-benchmem` to lock in allocation counts
- Since Go 1.25, `GOMAXPROCS` respects container CPU limits automatically

QUICK SUMMARY

- stack vs heap — escape analysis decides
- `GOGC` / `GOMEMLIMIT` — GC tuning
- Green Tea — default GC since 1.26
- `sync.Pool`, preallocation — fewer allocs

unsafe & cgo

UNSAFE & CGO

`unsafe` bypasses the type system: converting between pointer types, reading struct layout, and building strings or slices from raw pointers. **cgo** lets Go call C code via a special `import "C"` preceded by a C preamble comment. Both sacrifice portability and safety guarantees; use them only for performance-critical or interop code.

COSTS & RULES

- A cgo call costs far more than a Go call; batch work per call
- cgo breaks easy cross-compiling and static binaries
- C memory isn't GC'd: free what you `C.malloc` / `C.CString`

UNSAFE & CGO

```
type Header struct { A byte; B int64; C byte }
unsafe.Sizeof(Header{})           // 24 (padding!)
unsafe.Alignof(int64(0))          // 8
unsafe.Offsetof(Header{}.B)       // 8

b := []byte("hello")
s := unsafe.String(&b[0], len(b)) // no copy; b must not change
sl := unsafe.Slice(unsafe.StringData(s), len(s)) // []byte view

f := 1.5
bits := *(*uint64)(unsafe.Pointer(&f)) // = math.Float64bits(f)

/*
#include <stdlib.h>
#include <string.h>
static int add(int a, int b) { return a + b; }
*/
import "C"

r := C.add(2, 3)                  // C.int 5
cs := C.CString("go")             // malloc'd copy
defer C.free(unsafe.Pointer(cs))
n := C.strlen(cs)                 // 2
```

CODE EXPLAINED

- → `Sizeof` / `Alignof` / `Offsetof` — memory layout
- → `unsafe.String` — zero-copy `[]byte` → `string`
- → `(*uint64)(unsafe.Pointer(&f))` — type punning
- → C preamble comment directly above `import "C"`
- → `C.CString` + `C.free` — manual memory management

COMING FROM…

- **Python** — cgo ≈ C extensions / ctypes
- **Rust** — `unsafe` ≈ `unsafe {}` blocks
- **Java** — ≈ JNI, with similar call overhead

UNSAFE API

FUNC	DOES
`unsafe.Pointer`	generic pointer type
`Sizeof / Alignof / Offsetof`	layout info
`String / StringData`	string ↔ *byte
`Slice / SliceData`	slice ↔ *T
`Add(p, n)`	pointer offset

CGO TYPES

C	GO
`int / long`	C.int / C.long
`char*`	*C.char
`C string → Go`	C.GoString(p)
`Go → C string`	C.CString(s)
`void*`	unsafe.Pointer

⚠ COMMON MISTAKES

- Mutating a `[]byte` after creating a string from it with `unsafe.String`
- Storing pointers as `uintptr`; the GC doesn't track them and memory can move/free
- Blank line between the C preamble and `import "C"`; the preamble is ignored

✓ TIPS & BEST PRACTICES

- Use `math.Float64bits`, `binary.*` and `slices` before reaching for unsafe
- Check for pure-Go alternatives (e.g. `modernc.org/sqlite`) to avoid cgo
- Put cgo code in a small, well-tested package with a Go-friendly API

QUICK SUMMARY

- `unsafe` — bypass type safety
- `unsafe.String/Slice` — zero-copy views
- `import "C"` — call C via cgo
- costs — speed, portability, safety

Goroutines

go f() · go func() { … }() · cheap (KB-sized stacks) · main returning kills all goroutines · GOMAXPROCS (container-aware, 1.25) · runtime.NumGoroutine · "share memory by communicating" · unrecovered panic crashes the process

GOROUTINES

A goroutine is a function running **concurrently** with others, started with the `go` keyword. Goroutines are cheap (they start with a few KB of stack that grows as needed) and the runtime multiplexes them onto OS threads, so programs routinely run **thousands**. When `main` returns, the program exits, even if goroutines are still running.

NOTES

- `go f(x)` evaluates `x` now, runs `f` later
- No handle or return value: communicate via channels or shared state + sync
- `GOMAXPROCS` = OS threads running Go code; container-aware since 1.25
- An unrecovered panic in **any** goroutine crashes the whole program

STARTING & WAITING

```
func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    time.Sleep(100 * time.Millisecond)
    fmt.Println("worker", id, "done")
}
var wg sync.WaitGroup
for i := range 3 {
    wg.Add(1)
    go worker(i, &wg)              // pass pointer, never copy wg
}
wg.Wait()                          // block until all Done

var wg2 sync.WaitGroup
for _, url := range urls {
    wg2.Go(func() { fetch(url) })   // Go 1.25: Add + go + Done
}
wg2.Wait()
go func(msg string) { fmt.Println(msg) }("hello") // anonymous
fmt.Println(runtime.NumGoroutine(), runtime.GOMAXPROCS(0))
```

CODE EXPLAINED

- → `go worker(…)` — returns immediately; worker runs concurrently
- → `wg.Add(1)` before `go`, `defer wg.Done()` inside
- → `wg.Wait()` — waits for the counter to reach zero
- → `wg.Go(f)` — 1.25 shortcut, no Add/Done bookkeeping
- → `GOMAXPROCS(0)` — query without changing

COMING FROM…

- **Java** — goroutine ≈ virtual thread
- **Python** — real parallelism; no GIL
- **JS** — no event loop; blocking calls are fine

GOROUTINE VS THREAD

ASPECT	GOROUTINE
start cost	~2 KB stack, µs
scheduling	Go runtime (M:N)
count	100k+ is normal
identity	no public ID
stop	cooperatively (context)

COORDINATION

NEED	TOOL
wait for N tasks	`sync.WaitGroup`
collect errors	`errgroup.Group`
pass results	channels
cancel	`context.Context`
protect state	`sync.Mutex`

⚠ COMMON MISTAKES

- `wg.Add(1)` inside the goroutine; `Wait` may run before it and return early
- Exiting `main` without waiting, so goroutines are killed mid-work
- Starting goroutines with no way to stop them (leaks, sheet 098)

✓ TIPS & BEST PRACTICES

- Prefer `wg.Go(f)` (1.25) or `errgroup` over manual Add/Done
- Every goroutine should have a clear owner and exit condition
- Recover panics at the top of long-lived worker goroutines

QUICK SUMMARY

- `go f()` — start a goroutine
- `sync.WaitGroup` / `wg.Go` — wait for completion
- cheap & many — M:N scheduling
- `main` returns → program exits

Channels

CHANNELS

A channel is a typed pipe between goroutines: `ch <- v` sends and `v := <-ch` receives. An **unbuffered** channel synchronizes sender and receiver (each waits for the other); a **buffered** channel lets senders run ahead until it's full. The sender **closes** a channel to signal "no more values", which ends `for range ch` loops.

RULES

- Send on a closed channel → **panic**; close twice → panic
- Receive from a closed channel → zero value, `ok == false`
- Send/receive on a `nil` channel blocks forever
- Only the sender should close; receivers never close

CHANNEL BASICS

```
ch := make(chan int)                // unbuffered
go func() {
    for i := range 3 { ch <- i }  // blocks until received
    close(ch)                      // signal: done sending
}()
for v := range ch {                // ends when closed + drained
    fmt.Print(v, " ")              // 0 1 2
}

buf := make(chan string, 2)        // buffered, cap 2
buf <- "a"; buf <- "b"             // no receiver needed yet
fmt.Println(len(buf), cap(buf))    // 2 2
fmt.Println(<-buf)                 // a (FIFO)

v, ok := <-ch                      // 0 false: closed

func producer(out chan<- int) { out <- 42 }   // send-only
func consumer(in <-chan int)  { fmt.Println(<-in) } // receive-only

done := make(chan struct{})        // signal-only channel
go func() { work(); close(done) }()
<-done                             // wait for the signal
```

CODE EXPLAINED

- → unbuffered — each send waits for a matching receive
- → `close(ch)` + `range` — clean end of stream
- → buffered — sends block only when full
- → `chan<-` / `<-chan` — direction enforced by the compiler
- → `chan struct{}` + `close` — broadcast a one-time signal

COMING FROM…

- **Python** — ≈ `queue.Queue`, built into the language
- **Java** — ≈ `BlockingQueue`
- **Rust** — ≈ `mpsc` channels

CHANNEL STATES

OPERATION	NIL / OPEN / CLOSED
`send`	block / ok or block / panic
`receive`	block / value / zero, false
`close`	panic / ok / panic
`len / cap`	0 / n / n

BUFFERED OR NOT?

USE	CHOICE
`handoff / sync`	unbuffered
`bounded queue`	buffered(n)
`semaphore`	buffered(limit)
`one result from goroutine`	buffered(1)

⚠ COMMON MISTAKES

- Closing from the receiver side, or from multiple senders: "send on closed channel" panics
- Unbuffered send with no receiver in `main`: "all goroutines are asleep - deadlock!"
- Using buffer size to "fix" a deadlock; it usually just hides a design bug

✓ TIPS & BEST PRACTICES

- Use a buffered channel of size 1 so a goroutine can deliver its result and exit even if nobody reads
- Declare directional types in function params to document ownership
- With multiple senders, close via a `sync.WaitGroup` in a separate goroutine

QUICK SUMMARY

- `make(chan T[, n])` — create
- `ch <- v` / `<-ch` — send / receive
- `close` + `range` — end a stream
- `chan<-` / `<-chan` — directions

select

SELECT

`select` waits on **several channel operations** at once and runs whichever case is ready first; if several are ready, it picks one **at random**. A `default` case makes it non-blocking. Combined with `time.After` or `ctx.Done()`, `select` implements timeouts and cancellation.

NOTES

- `select {}` with no cases blocks forever
- A `nil` channel case is never ready: use it to disable a case
- `break` inside `select` exits the select, not the loop (label it)
- `time.After` in a hot loop allocates a timer per iteration; reuse a `Timer`

SELECT PATTERNS

```
select {                                // first ready wins
case msg := <-msgs: fmt.Println("got", msg)
case <-time.After(2 * time.Second):     // timeout
    return errors.New("timed out")
case <-ctx.Done():                      // cancellation
    return ctx.Err()
}

select { case jobs <- j: default: log.Println("full, dropped") } // try-send

for a != nil || b != nil {              // merge two channels
    select {
    case v, ok := <-a:
        if !ok { a = nil; continue }    // disable closed channel
        out <- v
    case v, ok := <-b:
        if !ok { b = nil; continue }
        out <- v
    }
}
```

CODE EXPLAINED

→ multiple ready cases — chosen at random (fairness)

→ `time.After` — channel that fires once after `d`

→ `ctx.Done()` — closed when the context is canceled

→ `default` — runs immediately if nothing is ready

→ `a = nil` — stop selecting on a closed channel

COMING FROM…

- **JS** — ≈ `Promise.race` over channels
- **Python** — ≈ `asyncio.wait(FIRST_COMPLETED)`
- **Rust** — ≈ `tokio::select!`

IDIOMS

GOAL	CASE
`timeout`	`case <-time.After(d):`
`cancel`	`case <-ctx.Done():`
`try send / recv`	`default:`
`periodic work`	`case <-ticker.C:`

GOTCHAS

CODE	EFFECT
`select {}`	block forever
`break in select`	leaves select only
`all cases nil, no default`	deadlock
`for+select without exit`	goroutine leak

⚠ COMMON MISTAKES

- Busy loop `for { select { … default: } }` burning 100% CPU
- Expecting case order to set priority; selection among ready cases is random
- Forgetting a `ctx.Done()` case, so the goroutine can't be stopped

✓ TIPS & BEST PRACTICES

- Always include `ctx.Done()` in long-running `for` / `select` loops
- Prefer `context.WithTimeout` over `time.After` when calling APIs that take a context
- For priority, check the urgent channel in a separate non-blocking select first

QUICK SUMMARY

- `select` — wait on multiple channels
- random among ready cases
- `default` — non-blocking
- timeouts & cancellation — `time.After`, `ctx.Done()`

sync Primitives

SYNC PRIMITIVES

When goroutines share memory, the `sync` package protects it. `Mutex` allows one goroutine at a time; `RWMutex` allows many readers or one writer. `WaitGroup` waits for goroutines, `Once` runs initialization exactly once, and `Pool` recycles temporary objects. All zero values are ready to use and **must not be copied** after use.

NOTES

- Lock, then `defer Unlock()` on the next line
- Embed the mutex next to the fields it guards
- `sync.OnceFunc` / `OnceValue` / `OnceValues` (1.21) wrap functions
- `go vet` copylocks catches accidental copies

GUARDING SHARED STATE

```
type Counter struct {
    mu sync.Mutex
    m  map[string]int          // guarded by mu
}
func (c *Counter) Inc(k string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.m[k]++
}

type Cache struct { mu sync.RWMutex; data map[string]string }
func (c *Cache) Get(k string) string {
    c.mu.RLock(); defer c.mu.RUnlock()   // many readers at once
    return c.data[k]
}

var wg sync.WaitGroup
for _, f := range files { wg.Go(func() { process(f) }) } // 1.25
wg.Wait()

var loadConfig = sync.OnceValue(mustLoad) // computed once
cfg := loadConfig()
```

CODE EXPLAINED

- → `Lock` / `defer Unlock` — critical section
- → `RLock` — shared read lock
- → `wg.Go` — spawn + track (1.25)
- → `sync.OnceValue` — lazy init with a result
- → `sync.Map` — specialized concurrent map

COMING FROM…

- **Java** — `Mutex` ≈ `ReentrantLock` (not reentrant!)
- **Python** — ≈ `threading.Lock`; `Once` ≈ `functools.cache`
- **Java** — `WaitGroup` ≈ `CountDownLatch`

PRIMITIVES

TYPE	USE
`Mutex`	exclusive access
`RWMutex`	read-heavy data
`WaitGroup`	wait for goroutines
`Once / OnceFunc`	run once
`Cond`	wait for condition (rare)
`Pool`	reuse temp objects

SYNC.MAP FITS WHEN

PATTERN	WHY
`write once, read many`	cache
`disjoint key sets per goroutine`	low contention
`otherwise`	map + Mutex is better

⚠ COMMON MISTAKES

- Passing a struct with a mutex **by value**; each copy has its own lock
- Forgetting to unlock on an early return; use `defer`
- Calling a method that locks from inside another method holding the same lock: deadlock (not reentrant)

✓ TIPS & BEST PRACTICES

- Keep critical sections tiny: copy data out, unlock, then do slow work
- Comment which fields a mutex guards
- Run tests with `-race` to find unguarded access

QUICK SUMMARY

- `Mutex` / `RWMutex` — protect shared data
- `WaitGroup` / `wg.Go` — wait
- `Once*` — one-time init
- never copy sync types

sync/atomic

ATOMIC OPERATIONS

`sync/atomic` performs **lock-free** reads and writes on single values. The typed wrappers (`atomic.Int64`, `atomic.Bool`, `atomic.Pointer[T]`) added in Go 1.19 are the preferred API: they can't be accessed non-atomically by accident and are correctly aligned. Use atomics for counters, flags and swapping immutable snapshots; use a mutex when several values must change together.

NOTES

- Every access to an atomic must be atomic (no mixing with plain reads)
- `CompareAndSwap` is the building block of lock-free updates
- `atomic.Pointer[T]` swaps whole immutable structs (config reload)
- `go fix` modernizer `atomictypes` (1.27) migrates old calls

TYPED ATOMICS

```
var requests atomic.Int64
requests.Add(1)                         // safe from many goroutines
fmt.Println(requests.Load())            // 1

var ready atomic.Bool; ready.Store(true)
old := hits.Swap(0)                     // read + reset (atomic.Uint32)

var state atomic.Int32
if state.CompareAndSwap(0, 1) {         // only one winner
    start()
}

type Config struct{ Limit int }
var cfg atomic.Pointer[Config]
cfg.Store(&Config{Limit: 10})
go func() { cfg.Store(&Config{Limit: 20}) }() // hot reload
limit := cfg.Load().Limit               // readers never block

var n int64
atomic.AddInt64(&n, 1)                  // old func style; prefer types
```

CODE EXPLAINED

- → `atomic.Int64` — methods `Add`, `Load`, `Store`, `Swap`, `CAS`
- → `atomic.Bool` — lock-free flag
- → `Swap` — set and return the previous value
- → `CompareAndSwap(old, new)` — succeed only if unchanged
- → `atomic.Pointer[T]` — publish new immutable snapshots

COMING FROM…

- **Java** — ≈ `AtomicLong`, `AtomicReference`
- **C++** — ≈ `std::atomic<T>`
- **Rust** — ≈ `AtomicI64`, `AtomicBool`

TYPES

TYPE	HOLDS
`atomic.Int32 / Int64`	signed ints
`atomic.Uint32 / Uint64 / Uintptr`	unsigned
`atomic.Bool`	bool
`atomic.Pointer[T]`	*T
`atomic.Value`	any (same type each Store)

ATOMIC OR MUTEX?

CASE	USE
`single counter`	atomic
`flag / state`	atomic
`read-mostly snapshot`	atomic.Pointer
`several fields together`	mutex
`map / slice updates`	mutex

⚠ COMMON MISTAKES

- Mixing `atomic.AddInt64(&n, 1)` with plain `n++` elsewhere: data race
- Check-then-act with `Load` then `Store` instead of `CompareAndSwap`
- Mutating the struct behind an `atomic.Pointer` instead of storing a new one

✓ TIPS & BEST PRACTICES

- Prefer typed atomics over the function API
- Expose metrics counters as `atomic.Int64` fields
- Benchmark: under low contention a mutex is often just as fast and clearer

QUICK SUMMARY

- `atomic.Int64` etc. — lock-free values
- `Add` / `Load` / `Store` / `Swap`
- `CompareAndSwap` — conditional update
- `atomic.Pointer[T]` — swap snapshots

context

CONTEXT

A `context.Context` carries a **cancellation signal**, an optional **deadline**, and request-scoped values across API boundaries and goroutines. Every function that does I/O or may block should accept `ctx context.Context` as its **first parameter** and stop when `ctx.Done()` closes. Derived contexts form a tree: canceling a parent cancels all its children.

RULES

- Never store a context in a struct; pass it explicitly
- Always call the `cancel` func (usually `defer cancel()`)
- Use `WithValue` only for request-scoped data (IDs), not optional params
- `ctx.Err()` is `context.Canceled` or `context.DeadlineExceeded`

CANCELLATION & DEADLINES

```
func fetch(ctx context.Context, url string) (*http.Response, error) {
    req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
    return http.DefaultClient.Do(req)        // aborts on cancel
}

ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()
resp, err := fetch(ctx, url)
if errors.Is(err, context.DeadlineExceeded) { /* too slow */ }

for {                                        // worker loop
    select {
    case <-ctx.Done(): return ctx.Err()      // stop promptly
    case j := <-jobs: handle(ctx, j)
    }
}

ctx2, cancelC := context.WithCancelCause(parent)
cancelC(errShutdown); context.Cause(ctx2)    // errShutdown
stop := context.AfterFunc(ctx, cleanup)      // run on cancel
type ctxKey struct{}; ctx = context.WithValue(ctx, ctxKey{}, reqID)
```

CODE EXPLAINED

- → `NewRequestWithContext` — HTTP honors cancellation
- → `WithTimeout` + `defer cancel()` — bounded work
- → `DeadlineExceeded` — distinguish timeouts
- → `ctx.Done()` in `select` — cooperative stop
- → unexported key type — avoids `WithValue` collisions

COMING FROM…

- **JS** — ≈ `AbortController` / `AbortSignal`
- **C#** — ≈ `CancellationToken`
- **Python** — ≈ `asyncio` task cancellation + timeouts

CONSTRUCTORS

FUNC	DONE WHEN
`Background() / TODO()`	never (roots)
`WithCancel(p)`	cancel() called
`WithTimeout(p, d)`	after d
`WithDeadline(p, t)`	at time t
`WithCancelCause(p)`	cancel(err)
`WithoutCancel(p)`	never (keeps values)

METHODS

METHOD	RETURNS
`Done()`	<-chan struct{}
`Err()`	nil / Canceled / DeadlineExceeded
`Deadline()`	time, ok
`Value(key)`	any
`context.Cause(ctx)`	cancel cause

⚠ COMMON MISTAKES

- Forgetting `cancel()`: timers and goroutines leak until the parent ends (`go vet lostcancel`)
- Passing `nil` as a context; use `context.TODO()`
- Using `context.Background()` deep inside handlers, losing the request's cancellation

✓ TIPS & BEST PRACTICES

- In HTTP handlers use `r.Context()`; it cancels when the client disconnects
- Use `context.WithoutCancel` for work that must finish after the request (audit logs)
- In tests, use `t.Context()` (1.24); it's canceled when the test ends

QUICK SUMMARY

- `ctx` first param — cancellation + deadline
- `WithTimeout` / `WithCancel` + `defer cancel()`
- `<-ctx.Done()` / `ctx.Err()` — react
- `WithValue` — request-scoped data only

Concurrency Patterns

worker pool · fan-out / fan-in · pipelines · semaphore via buffered channel · errgroup.Group / SetLimit · rate limiting (time.Ticker, x/time/rate) · graceful shutdown

CONCURRENCY PATTERNS

Most concurrent Go code is built from a few patterns: a **worker pool** reading jobs from a channel, **fan-out/fan-in** to parallelize and merge, **pipelines** of stages connected by channels, and **bounded concurrency** with a semaphore or `errgroup.SetLimit`. Each pattern needs a clear **shutdown** path via channel close or context cancel.

PRINCIPLES

- The goroutine that creates a channel usually closes it
- Bound concurrency; unbounded goroutines exhaust memory/sockets
- Propagate `ctx` so every stage can stop
- `golang.org/x/sync/errgroup` = WaitGroup + errors + cancel

WORKER POOL & ERRGROUP

```
func pool(ctx context.Context, jobs []Job, n int) []Result {
    in, out := make(chan Job), make(chan Result)
    var wg sync.WaitGroup
    for range n {                              // fan-out
        wg.Go(func() { for j := range in { out <- process(j) } })
    }
    go func() {                                // feed jobs
        defer close(in)
        for _, j := range jobs {
            select { case in <- j: case <-ctx.Done(): return }
        }
    }()
    go func() { wg.Wait(); close(out) }()      // close when all done
    var res []Result
    for r := range out { res = append(res, r) } // fan-in
    return res
}

g, ctx := errgroup.WithContext(ctx)
g.SetLimit(8)                                  // at most 8 at once
for _, u := range urls { g.Go(func() error { return fetch(ctx, u) }) }
err := g.Wait()
```

CODE EXPLAINED

- → N workers `range` over one input channel
- → producer closes `in`; workers then exit
- → `wg.Wait(); close(out)` — signals fan-in to stop
- → `errgroup.SetLimit` — bounded parallelism + first error
- → buffered chan — counting semaphore

COMING FROM…

- **Java** — worker pool ≈ `ExecutorService`
- **Python** — ≈ `ThreadPoolExecutor`
- **JS** — `errgroup` ≈ `Promise.all` with concurrency limit

PATTERNS

PATTERN	SHAPE
worker pool	1 chan → N workers
fan-out / fan-in	split → merge results
pipeline	stage → chan → stage
semaphore	buffered chan of size N
rate limit	`time.Ticker` / `x/time/rate`

SHUTDOWN SIGNALS

SIGNAL	HOW
no more input	`close(in)`
all workers done	`wg.Wait()` then close out
abort everything	`cancel()` ctx
OS signal	`signal.NotifyContext`

⚠ COMMON MISTAKES

- Closing the output channel from a worker while others still send
- Unbounded `go` per item on a million-item list
- Fan-in reader that stops early, leaving workers blocked on send forever

✓ TIPS & BEST PRACTICES

- Reach for `errgroup` first; hand-roll pools only when you need streaming results
- Use `x/time/rate.Limiter` for requests-per-second limits
- Keep results ordered by sending `{index, value}` and placing into a slice

QUICK SUMMARY

- worker pool — N goroutines, one job chan
- fan-out / fan-in — parallelize & merge
- `errgroup.SetLimit` — bounded + errors
- clean shutdown — close / cancel

Races, Deadlocks & Leaks

go test -race / go run -race · data-race examples · "all goroutines are asleep - deadlock!" · goroutine leaks (blocked sends) · goroutineleak pprof profile (1.27) · testing/synctest (1.25) · GOTRACEBACK

RACES, DEADLOCKS & LEAKS

Concurrency bugs come in three main forms. A **data race** is unsynchronized access to the same memory with at least one write; the **race detector** (`-race`) finds them at runtime. A **deadlock** is when goroutines wait on each other forever. A **goroutine leak** is a goroutine blocked forever that's never cleaned up. Go 1.27's **`goroutineleak` profile** and `testing/synctest` help find the last two.

NOTES

- `-race` slows code 2-20× but finds real bugs; use it in CI tests
- The runtime only detects deadlock when **all** goroutines are blocked
- Leaks show up as a growing `runtime.NumGoroutine()`

SPOT THE BUG

```
count := 0                                  // DATA RACE
for range 100 { go func() { count++ }() } // fix: atomic / mutex

ch := make(chan int)
ch <- 1     // fatal error: all goroutines are asleep - deadlock!

func search(q string) string {              // LEAK
    ch := make(chan string)                 // fix: make(chan string, 1)
    go func() { ch <- slowLookup(q) }()
    select {
    case r := <-ch: return r
    case <-time.After(time.Second): return "" // sender stuck
    }
}

func TestTimeout(t *testing.T) {            // deterministic time
    synctest.Test(t, func(t *testing.T) {
        go worker()
        time.Sleep(time.Hour)               // instant: fake clock
        synctest.Wait()                     // until all goroutines block
    })
}
// $ go test -race ./...    ·    /debug/pprof/goroutineleak (1.27)
```

CODE EXPLAINED

- → `count++` from many goroutines — read-modify-write race
- → send with no receiver — blocks main forever
- → timeout path leaves the sender blocked: leak
- → buffer of 1 lets the sender finish and exit
- → `synctest.Test` — virtual time, `Wait` for quiescence

COMING FROM…

- **Java** — `-race` has no built-in JVM equivalent
- **Rust** — races caught at compile time there; here at runtime
- **Python** — no GIL protection; races are real

TOOLS

TOOL	FINDS
`go test -race`	data races
`go vet`	copylocks, lostcancel
`pprof goroutine`	stack of every goroutine
`goroutineleak profile`	leaked goroutines (1.27)
`testing/synctest`	timing-dependent tests

PREVENTION

BUG	HABIT
`race`	share via channels or lock
`deadlock`	consistent lock order
`leak`	ctx + buffered result chan
`flaky test`	synctest, not Sleep

⚠ COMMON MISTAKES

- Running `-race` only locally; enable it in CI where tests exercise concurrency
- Assuming "it works on my machine" means no race; races are timing-dependent
- Fixing flaky concurrent tests with longer `time.Sleep` calls

✓ TIPS & BEST PRACTICES

- Expose `net/http/pprof` on an internal port to inspect goroutines in production
- Use `go.uber.org/goleak` in `TestMain` to fail tests that leak goroutines
- Lock multiple mutexes in a fixed global order to avoid deadlocks

QUICK SUMMARY

- `-race` — detect data races
- deadlock — all goroutines blocked
- leak — blocked goroutine never exits
- `synctest` / `goroutineleak` — newer tools

Testing — Basics

TESTING BASICS

Go has testing built in. Tests live in `_test.go` files next to the code, are functions named `TestXxx(t *testing.T)`, and run with `go test`. There's no assertion library in the stdlib: compare values and call `t.Errorf` (continue) or `t.Fatalf` (stop). The idiomatic style is **table-driven tests** with **subtests** via `t.Run`.

NOTES

- `package foo` tests see unexported names; `package foo_test` tests only the public API
- Test results are cached; `-count=1` forces a re-run
- `-run 'TestAdd/neg'` selects tests and subtests by regex
- `t.Errorf` messages: `got X, want Y`

TABLE-DRIVEN TEST

```go
package mathx                              // math_test.go

func TestAdd(t *testing.T) {
    tests := []struct {
        name       string
        a, b, want int
    }{
        {"positive", 2, 3, 5},
        {"negative", -2, -3, -5},
        {"zero", 0, 0, 0},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            if got := Add(tt.a, tt.b); got != tt.want {
                t.Errorf("Add(%d,%d) = %d, want %d", tt.a, tt.b, got, tt.want)
            }
        })
    }
}
```

CODE EXPLAINED

- → `_test.go` — compiled only by `go test`
- → `[]struct{…}` — one row per case
- → `t.Run(name, …)` — named subtest, filterable with `-run`
- → `t.Errorf` — record failure and keep going
- → `got … want` — the conventional message order

COMING FROM…

- **Python** — no `assert`; use `t.Errorf` (or testify)
- **Java** — no annotations; `TestXxx` names are found automatically
- **JS** — table tests ≈ `test.each`

GO TEST FLAGS

FLAG	EFFECT
`./...`	all packages
`-v`	verbose, list every test
`-run Regex`	select tests
`-count=1`	bypass cache
`-short`	skip long tests (`testing.Short()`)
`-failfast`	stop at first failure

FAILURE METHODS

METHOD	BEHAVIOR
`t.Error / Errorf`	fail, continue
`t.Fatal / Fatalf`	fail, stop this test
`t.Log / Logf`	print with -v or on fail
`t.Skip / Skipf`	mark skipped
`t.Fail / FailNow`	fail without message

⚠ COMMON MISTAKES

- Calling `t.Fatal` from a goroutine started by the test; it must run on the test goroutine
- Tests that depend on execution order or shared global state
- Vague messages like `"wrong result"`; include inputs, got and want

✓ TIPS & BEST PRACTICES

- Use `go test -cover` to see coverage per package
- Put fixtures in `testdata/`; the go tool ignores that directory
- Compare structs with `reflect.DeepEqual` or `github.com/google/go-cmp/cmp.Diff`

QUICK SUMMARY

- `func TestX(t *testing.T)` in `_test.go`
- `t.Errorf` / `t.Fatalf` — report failures
- table-driven + `t.Run` — idiomatic style
- `go test ./... -run -v -count=1`

Testing — Helpers & Fixtures

t.Helper · t.Cleanup · t.Parallel · t.TempDir / t.Setenv / t.Chdir · t.Context (1.24) · testdata/ & golden files · TestMain · httptest.NewRecorder / NewServer

TEST HELPERS & FIXTURES

The `testing.T` type has helpers that make tests clean and isolated: `t.Helper` fixes failure line numbers, `t.Cleanup` registers teardown, `t.TempDir` / `t.Setenv` / `t.Chdir` provide auto-reverted fixtures, and `t.Context` (1.24) gives a context canceled at test end. `httptest` tests HTTP handlers and clients without a real network.

NOTES

- `t.Parallel()` runs a test alongside other parallel tests
- `t.Setenv` can't be used with `t.Parallel`
- `TestMain(m *testing.M)` runs package-wide setup/teardown
- Golden files: compare output with `testdata/*.golden`, update with a flag

HELPERS IN ACTION

```
func newTestDB(t *testing.T) *DB {
    t.Helper()                          // errors point at the caller
    db, err := Open(filepath.Join(t.TempDir(), "test.db"))
    if err != nil { t.Fatalf("open: %v", err) }
    t.Cleanup(func() { db.Close() })    // runs after the test
    return db
}

func TestHandler(t *testing.T) {
    t.Setenv("MODE", "test")            // restored; not with Parallel
    req := httptest.NewRequest("GET", "/users/7", nil)
    rec := httptest.NewRecorder()
    NewServer(newTestDB(t)).ServeHTTP(rec, req)
    if rec.Code != http.StatusOK { t.Fatalf("status %d", rec.Code) }
}

func TestClient(t *testing.T) {
    srv := httptest.NewServer(okHandler)  // any http.Handler
    defer srv.Close()
    got, err := Fetch(t.Context(), srv.URL)  // t.Context: 1.24
}
```

CODE EXPLAINED

- → `t.Helper()` — report failures at the caller's line
- → `t.TempDir()` — unique dir, removed automatically
- → `t.Cleanup` — LIFO teardown, even on failure
- → `NewRecorder` — capture a handler's response
- → `NewServer` — real local HTTP server for client tests

COMING FROM…

- **Python** — `t.TempDir` ≈ `tmp_path` fixture
- **Java** — `t.Cleanup` ≈ `@AfterEach`
- **JS** — `httptest` ≈ supertest / nock

TESTING.T HELPERS

METHOD	PROVIDES
`TempDir()`	auto-removed directory
`Setenv(k, v)`	env var, restored
`Chdir(dir)`	cwd, restored (1.24)
`Context()`	ctx canceled at end (1.24)
`Cleanup(f)`	teardown
`Parallel()`	run concurrently

HTTPTEST

FUNC	USE
`NewRequest`	fake *http.Request
`NewRecorder`	fake ResponseWriter
`NewServer`	local HTTP server
`NewTestServer`	new in 1.27

⚠ COMMON MISTAKES

- Forgetting `t.Helper()` in helpers, so every failure points inside the helper
- Using `defer` in a helper for cleanup; it runs when the helper returns. Use `t.Cleanup`
- Hitting real external services in unit tests; use `httptest.NewServer`

✓ TIPS & BEST PRACTICES

- Return cleanup-free fixtures from helpers by registering `t.Cleanup` inside them
- Add `-update` flag handling for golden files: `var update = flag.Bool("update", false, "")`
- Use `TestMain` sparingly (e.g. starting one container for the whole package)

QUICK SUMMARY

- `t.Helper` / `t.Cleanup` — clean helpers
- `t.TempDir` / `Setenv` / `Chdir` — fixtures
- `t.Context` — test-scoped ctx (1.24)
- `httptest` — handlers & clients

Benchmarks, Fuzzing & Examples

BENCHMARKS, FUZZING & EXAMPLES

The `testing` package also runs **benchmarks** (`BenchmarkXxx(b *testing.B)`), **fuzz tests** (`FuzzXxx(f *testing.F)`) that feed random mutated input to find crashes, and **examples** (`ExampleXxx`) that double as documentation and are verified against `// Output:`. Go 1.24's `b.Loop()` is the preferred benchmark loop.

NOTES

- Benchmarks run only with `-bench`; `-benchmem` reports allocations
- `b.Loop()` keeps setup out of timing and prevents dead-code elimination
- Fuzz failures are saved to `testdata/fuzz/` and become regression tests
- Compare runs statistically with `benchstat`

BENCH, FUZZ, EXAMPLE

```
func BenchmarkJoin(b *testing.B) {
    parts := strings.Fields("a b c d e f")  // setup: not timed
    for b.Loop() { strings.Join(parts, ",") } // Go 1.24
}

func FuzzReverse(f *testing.F) {
    f.Add("hello")                          // seed corpus
    f.Fuzz(func(t *testing.T, s string) {
        if Reverse(Reverse(s)) != s && utf8.ValidString(s) {
            t.Errorf("round trip failed for %q", s)
        }
    })
}

func ExampleReverse() {
    fmt.Println(Reverse("abc"))
    // Output: cba
}

// $ go test -bench=. -benchmem -count=6 > new.txt
// $ benchstat old.txt new.txt
// $ go test -fuzz=FuzzReverse -fuzztime=30s
```

CODE EXPLAINED

- → `for b.Loop()` — runs the body enough times to measure
- → `f.Add` — seed inputs; `f.Fuzz` — property to check
- → `ExampleReverse` — shown in docs, checked by `go test`
- → `-count=6` + `benchstat` — compare with confidence
- → `go tool cover -html` — visual coverage report

COMING FROM…

- **Python** — benchmarks ≈ `pytest-benchmark`, built in
- **Python** — fuzzing ≈ Hypothesis
- **Java** — benchmarks ≈ JMH

BENCHMARK OUTPUT

COLUMN	MEANING
`BenchmarkJoin-10`	name + GOMAXPROCS
`12345678`	iterations
`95.2 ns/op`	time per op
`24 B/op`	bytes allocated
`1 allocs/op`	allocations

USEFUL FLAGS

FLAG	EFFECT
`-bench=Join`	select benchmarks
`-benchtime=3s / 100x`	duration / count
`-cpuprofile cpu.out`	profile while benchmarking
`-fuzz=Name -fuzztime`	run fuzzer
`-cover / -coverpkg`	coverage

⚠ COMMON MISTAKES

- Old-style `for i := 0; i < b.N; i++` with setup inside the loop being timed
- Comparing single benchmark runs; noise can exceed the difference
- Examples without `// Output:`; they compile but are never checked

✓ TIPS & BEST PRACTICES

- Run `go fix` to migrate `b.N` loops to `b.Loop()`
- Fuzz parsers and decoders: they find edge cases humans miss
- Use `testing.AllocsPerRun` to assert a function doesn't allocate

QUICK SUMMARY

- `BenchmarkX` + `b.Loop()` — performance
- `FuzzX` + `f.Fuzz` — random inputs
- `ExampleX` + `// Output:` — tested docs
- `-benchmem`, `benchstat`, `-cover`

Profiling & Tracing

PROFILING & TRACING

pprof shows where a program spends CPU and memory; the **execution tracer** shows what goroutines, the scheduler and GC were doing over time. Profiles come from tests (`-cpuprofile`), from code (`runtime/pprof`), or from a live server via `net/http/pprof`. Go 1.25 added a **flight recorder** that keeps the last few seconds of trace in memory, and **PGO** feeds a profile back into the compiler.

NOTES

- Profile before optimizing; intuition about hot spots is often wrong
- Expose `/debug/pprof` only on an internal port
- `default.pgo` in the main package enables PGO automatically (2-14% gains)
- Go 1.27 adds the `goroutineleak` profile

COLLECTING PROFILES

```
import _ "net/http/pprof"           // registers /debug/pprof/*
go http.ListenAndServe("localhost:6060", nil)

f, _ := os.Create("cpu.out")        // programmatic CPU profile
pprof.StartCPUProfile(f)
defer pprof.StopCPUProfile()

fr := trace.NewFlightRecorder(trace.FlightRecorderConfig{
    MinAge: 5 * time.Second,        // keep last ~5s (1.25)
})
fr.Start()
if slowRequest { fr.WriteTo(traceFile) } // snapshot on anomaly

// $ go test -bench=. -cpuprofile cpu.out -memprofile mem.out
// $ go tool pprof -http=:8080 cpu.out          # web UI + flame graph
// $ go tool pprof http://localhost:6060/debug/pprof/heap
// $ curl -o trace.out localhost:6060/debug/pprof/trace?seconds=5
// $ go tool trace trace.out
// $ cp cpu.out ./cmd/app/default.pgo && go build ./cmd/app  # PGO
```

CODE EXPLAINED

- → `_ "net/http/pprof"` — adds handlers to the default mux
- → `StartCPUProfile` — sample CPU until stopped
- → `FlightRecorder` — rolling in-memory trace buffer
- → `pprof -http` — interactive graphs and flame graph
- → `default.pgo` — profile-guided optimization

COMING FROM…

- **Python** — ≈ `cProfile` / py-spy
- **Java** — ≈ async-profiler / JFR
- **Node** — ≈ `--prof`, Chrome DevTools

PROFILES

PROFILE	SHOWS
`profile (CPU)`	where time goes
`heap / allocs`	live memory / allocations
`goroutine`	all goroutine stacks
`block / mutex`	contention (enable rate)
`goroutineleak`	leaked goroutines (1.27)
`trace`	timeline of events

PPROF COMMANDS

COMMAND	DOES
`top10`	hottest functions
`list Func`	annotated source
`web`	call graph (graphviz)
`-diff_base old.out`	compare profiles
`-http=:8080`	browser UI

⚠ COMMON MISTAKES

- Exposing `/debug/pprof` on a public port; it leaks internals and can be abused
- Profiling a debug build or a tiny input that doesn't reflect production
- Reading `flat` vs `cum` wrong: `cum` includes time in callees

✓ TIPS & BEST PRACTICES

- Start with the flame graph view; it makes wide (expensive) stacks obvious
- Enable `runtime.SetMutexProfileFraction` / `SetBlockProfileRate` only when investigating
- Commit a representative `default.pgo` and refresh it periodically

QUICK SUMMARY

- `pprof` — CPU, heap, goroutine profiles
- `net/http/pprof` — live server profiling
- `go tool trace` + flight recorder — timelines
- PGO — `default.pgo` speeds up builds

Formatting, Vet & Linters

gofmt -l -w · goimports · go vet analyzers · staticcheck · golangci-lint run · go fix modernizers · govulncheck · gopls diagnostics

FORMATTING, VET & LINTERS

Go's tooling removes style debates: `gofmt` defines **the** format, `goimports` also manages imports, and `go vet` reports suspicious code like bad printf verbs or copied locks. **staticcheck** and the **golangci-lint** aggregator add hundreds more checks. `go fix` applies **modernizers** that rewrite code to newer idioms after a Go upgrade.

NOTES

- `go vet` runs automatically (a subset) during `go test`
- gopls shows vet/staticcheck diagnostics in the editor
- Go 1.27 `gofmt` alignment changed slightly; reformat after upgrading
- `govulncheck` reports only vulnerabilities your code actually reaches

TOOLING COMMANDS

```
gofmt -l .                          # list unformatted files
gofmt -w .                          # rewrite in place
gofmt -s -w .                       # also simplify code
go install golang.org/x/tools/cmd/goimports@latest
goimports -w .                      # format + fix imports

go vet ./...                        # standard analyzers
go vet -printf=false ./...          # disable one analyzer

go install honnef.co/go/tools/cmd/staticcheck@latest
staticcheck ./...

golangci-lint run ./...             # many linters, config in .golangci.yml
golangci-lint run --fix

go fix ./...                        # modernizers (min/max, any, b.Loop…)
govulncheck ./...                   # known CVEs on reachable code
go mod tidy -diff                   # fail CI if go.mod is untidy
```

CODE EXPLAINED

- → `gofmt -l` — use in CI to fail on unformatted code
- → `goimports` — add/remove imports + format
- → `go vet` — fast, no false-positive-heavy checks
- → `golangci-lint` — runs staticcheck, errcheck, revive…
- → `go fix` — automated modernization

COMING FROM…

- **Python** — `gofmt` ≈ Black, but universal
- **JS** — ≈ Prettier + ESLint
- **Rust** — ≈ `rustfmt` + `clippy`

VET ANALYZERS

ANALYZER	CATCHES
printf	verb/arg mismatch
copylocks	copied sync.Mutex
lostcancel	unused context cancel
structtag	malformed tags
unusedresult	ignored `fmt.Sprintf` etc.
stdversion	API newer than go.mod (1.27 in test)

POPULAR LINTERS

LINTER	FINDS
staticcheck	bugs, simplifications
errcheck	unchecked errors
revive	style (golint successor)
gosec	security issues
ineffassign	useless assignments

⚠ COMMON MISTAKES

- Disabling linters globally instead of one `//nolint:name // reason` on a line
- Running `gofmt` in CI but not `goimports`, causing import-order churn
- Treating every staticcheck suggestion as optional; many are real bugs

✓ TIPS & BEST PRACTICES

- Configure editor format-on-save with gopls (`gofumpt` option for stricter style)
- Pin golangci-lint's version in CI for reproducible results
- Run `go fix ./...` right after bumping the `go` line in `go.mod`

QUICK SUMMARY

- `gofmt` / `goimports` — canonical format
- `go vet` — built-in static checks
- `staticcheck` / `golangci-lint` — deeper linting
- `go fix` / `govulncheck` — modernize & secure

Debugging with Delve

DEBUGGING WITH DELVE

Delve (`dlv`) is Go's debugger: it understands goroutines, interfaces and Go's runtime, which `gdb` doesn't. You can debug a package, a test, a running process or a core dump, from the terminal or through VS Code / GoLand, which use Delve underneath. For quick checks, print debugging with `%+v` and `slog` is still common.

NOTES

- Install: `go install github.com/go-delve/delve/cmd/dlv@latest`
- Optimizations make variables `<optimized out>`; build with `-gcflags=all="-N -l"`
- `dlv test` debugs tests; pass test flags after `--`
- Remote/headless: `dlv debug --headless --listen=:2345`

A DELVE SESSION

```
dlv debug ./cmd/app -- -port 8080   # build + start, args after --
dlv test ./store -- -test.run TestSave
dlv attach $(pgrep app)             # running process

(dlv) break main.go:42              # or: b store.(*DB).Save
(dlv) condition 1 id == 7           # conditional breakpoint
(dlv) continue                      # c: run to breakpoint
(dlv) next                          # n: step over
(dlv) step                          # s: step into
(dlv) stepout                       # finish current function
(dlv) print user                    # p: inspect value
(dlv) locals                        # all local variables
(dlv) goroutines                    # list goroutines
(dlv) goroutine 18 bt               # stack of one goroutine
(dlv) set count = 0                 # change a variable
(dlv) restart                       # r: start over

go build -gcflags=all="-N -l" -o app-dbg ./cmd/app
```

CODE EXPLAINED

- → `dlv debug` — compile with debug info and launch
- → `b pkg.(*T).Method` — break on a method
- → `condition` — stop only when an expression is true
- → `goroutines` / `goroutine N bt` — inspect concurrency
- → `-N -l` — disable optimizations and inlining

COMING FROM…

- **Python** — ≈ `pdb`
- **C** — ≈ `gdb`, but goroutine-aware
- **Java** — ≈ IDE debugger via jdwp

START MODES

COMMAND	DEBUGS
`dlv debug`	main package
`dlv test`	package tests
`dlv exec ./bin`	prebuilt binary
`dlv attach PID`	running process
`dlv core bin core`	core dump

PRINT DEBUGGING

TOOL	USE
`fmt.Printf("%+v\n", x)`	struct with fields
`fmt.Printf("%#v\n", x)`	Go syntax
`slog.Debug(…)`	leveled, structured
`debug.PrintStack()`	current stack
`SIGQUIT (Ctrl-\)`	dump all goroutines

⚠ COMMON MISTAKES

- Debugging an optimized build and seeing skipped lines / missing variables
- Forgetting `--` before program args, so Delve tries to parse them
- Leaving debug `fmt.Println` calls in committed code; use `slog.Debug`

✓ TIPS & BEST PRACTICES

- In VS Code, use `launch.json` with `"mode": "debug"` or `"test"`
- Use `trace` (`dlv trace pkg Func`) to log calls without stopping
- Send SIGQUIT to a hung program to print every goroutine's stack

QUICK SUMMARY

- `dlv debug` / `test` / `attach` — start
- `break`, `continue`, `next`, `step` — control
- `print`, `locals`, `goroutines` — inspect
- `-gcflags=all="-N -l"` — debuggable builds

HTTP Server — Basics

NET/HTTP SERVER

Go's standard library ships a production-ready HTTP server. A **handler** writes to an `http.ResponseWriter` and reads the `*http.Request`. Since **Go 1.22**, `http.ServeMux` patterns can include a **method** and **wildcards** like `GET /items/{id}`, read with `r.PathValue`. Each request runs in its own goroutine.

PATTERN RULES

- `/path/` (trailing slash) matches the whole subtree
- `{$}` matches the exact path only: `GET /{$}`
- `{rest...}` captures the remaining path segments
- The most specific pattern wins; conflicts panic at registration

A MINIMAL SERVER

```
func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /{$}", home)                    // exact "/"
    mux.HandleFunc("GET /items/{id}", getItem)          // method + wildcard
    mux.HandleFunc("POST /items", createItem)
    mux.HandleFunc("/files/{path...}", serveFile)       // any method
    log.Fatal(http.ListenAndServe(":8080", mux))
}

func home(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "hello")
}

func getItem(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
    item, ok := store[id]
    if !ok {
        http.Error(w, "not found", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(item)
}
```

CODE EXPLAINED

- → `NewServeMux` — avoid the global `DefaultServeMux` in apps
- → `"GET /items/{id}"` — 405 is returned for other methods
- → `r.PathValue("id")` — wildcard value
- → `http.Error` — plain-text error + status; then `return`
- → set headers **before** writing the body

COMING FROM…

- **Node** — no framework needed; ≈ Express built in
- **Python** — ≈ Flask routes with `"GET /x/{id}"`
- **Java** — embedded server, no servlet container

REQUEST DATA

NEED	READ
path wildcard	`r.PathValue("id")`
query string	`r.URL.Query().Get("q")`
header	`r.Header.Get("Authorization")`
form field	`r.FormValue("name")`
body	`json.NewDecoder(r.Body)`
method / ctx	`r.Method` / `r.Context()`

RESPONSE

ACTION	CALL
header	`w.Header().Set(k, v)`
status	`w.WriteHeader(201)`
body	`w.Write(b)` / `fmt.Fprint(w, …)`
error	`http.Error(w, msg, code)`
redirect	`http.Redirect(w, r, url, 302)`

⚠ COMMON MISTAKES

- Forgetting `return` after `http.Error`; the handler keeps writing
- Setting headers after `Write` / `WriteHeader`; they're already sent and ignored
- Using `http.ListenAndServe` with no timeouts in production (sheet 109)

✓ TIPS & BEST PRACTICES

- Register routes in one function (`routes(mux)`) so they're easy to review
- Use `http.StatusText(code)` / named constants instead of bare numbers
- Test handlers with `httptest.NewRecorder` (sheet 100)

QUICK SUMMARY

- `HandleFunc("GET /x/{id}", h)` — routing (1.22)
- `r.PathValue` / `Query` / `FormValue` — input
- `w.Header` → `WriteHeader` → `Write` — output order
- `http.Error` + `return` — errors

Handlers & Middleware

HANDLERS & MIDDLEWARE

Everything in `net/http` revolves around one interface, `http.Handler` with `ServeHTTP(w, r)`. `http.HandlerFunc` adapts a plain function into a Handler. **Middleware** is a function that takes a Handler and returns a new one wrapping it, adding logging, auth, recovery or CORS around every request.

NOTES

- Middleware order matters: the outermost runs first
- Pass request-scoped data via `r.WithContext(ctx)`
- `net/http` already recovers panics per request, but logs noisily
- Wrap `ResponseWriter` to capture the status code for logging

WRITING MIDDLEWARE

```
func logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)                    // call the next handler
        slog.Info("req", "path", r.URL.Path, "took", time.Since(start))
    })
}

type userKey struct{}
func auth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        user, err := verify(r.Header.Get("Authorization"))
        if err != nil { http.Error(w, "unauthorized", 401); return }
        ctx := context.WithValue(r.Context(), userKey{}, user)
        next.ServeHTTP(w, r.WithContext(ctx))  // pass data on
    })
}

mux.Handle("GET /admin/", auth(adminHandler))  // per-route
mux.Handle("/static/", http.StripPrefix("/static/", http.FileServer(http.Dir("public"))))
http.ListenAndServe(":8080", logging(recoverer(mux))) // global chain
```

CODE EXPLAINED

- → `func(http.Handler) http.Handler` — middleware shape
- → `next.ServeHTTP` — continue the chain (or don't, to block)
- → `r.WithContext(ctx)` — attach values for later handlers
- → `StripPrefix` + `FileServer` — static files
- → `logging(recoverer(mux))` — logging runs outermost

COMING FROM…

- **Node** — middleware ≈ Express `(req, res, next)`
- **Python** — ≈ WSGI middleware / decorators
- **Java** — ≈ servlet filters

COMMON MIDDLEWARE

MIDDLEWARE	DOES
logging	method, path, status, latency
recover	turn panics into 500
auth	verify token, set user
CORS	cross-origin headers
request ID	trace correlation

ADAPTERS

TYPE	PURPOSE
http.HandlerFunc	func → Handler
http.StripPrefix	trim path prefix
http.TimeoutHandler	per-request time limit
http.MaxBytesHandler	limit body size
http.FileServerFS	serve an fs.FS

⚠ COMMON MISTAKES

- Writing to `w` after calling `next` and expecting to change the status (already sent)
- Using string keys for context values; collide across packages. Use an unexported key type
- Applying auth middleware to the whole mux and accidentally protecting health checks

✓ TIPS & BEST PRACTICES

- Build a tiny `chain(h, m1, m2, …)` helper to make order explicit
- Wrap `ResponseWriter` in a struct that records `WriteHeader` for status logging
- Use `http.NewResponseController(w)` (1.20) for flush/deadline control

QUICK SUMMARY

- `http.Handler` — `ServeHTTP(w, r)`
- `HandlerFunc` — functions as handlers
- middleware — `func(Handler) Handler`
- `r.WithContext` — per-request values

Building JSON APIs

BUILDING JSON APIS

A JSON API handler decodes the request body into a struct, validates it, does the work, and encodes a response with the right **status code** and `Content-Type`. Limit body size with `http.MaxBytesReader`, reject unknown fields to catch client typos, and return errors in a **consistent JSON shape**.

NOTES

- Order: `Header().Set` → `WriteHeader` → body
- 201 Created for POST, 204 No Content for DELETE
- Map domain errors to statuses in one place
- `r.Body` is closed by the server; no need to close it

DECODE, VALIDATE, RESPOND

```
type createReq struct {
    Name  string `json:"name"`
    Price int    `json:"price"`
}
func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(v)
}
func createItem(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20)  // 1 MB max
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    var req createReq
    if err := dec.Decode(&req); err != nil {
        writeJSON(w, 400, errBody(err)); return
    }
    if req.Name == "" { writeJSON(w, 422, errMsg("name required")); return }
    item, err := store.Create(r.Context(), req.Name, req.Price)
    if errors.Is(err, ErrDuplicate) { writeJSON(w, 409, errBody(err)); return }
    writeJSON(w, http.StatusCreated, item)
}
```

CODE EXPLAINED

- → `writeJSON` — one helper for headers, status, body
- → `MaxBytesReader` — 413-style failure for huge bodies
- → `DisallowUnknownFields` — reject `{"nmae": …}` typos
- → 422 for validation, 409 for conflicts
- → `r.Context()` — cancel DB work if the client leaves

COMING FROM…

- **Python** — ≈ FastAPI handler, manual validation
- **Node** — ≈ Express + `res.status().json()`
- **Java** — ≈ `@RestController` without annotations

STATUS CODES

CODE	WHEN
`200 OK`	successful GET/PUT
`201 Created`	POST created a resource
`204 No Content`	DELETE / empty success
`400 / 422`	bad JSON / invalid data
`401 / 403`	not logged in / not allowed
`404 / 409 / 500`	missing / conflict / server bug

INPUT SOURCES

SOURCE	READ
`JSON body`	`json.NewDecoder(r.Body)`
`path`	`r.PathValue("id")`
`query`	`r.URL.Query().Get("page")`
`form`	`r.FormValue("q")`
`header`	`r.Header.Get("X-Key")`

⚠ COMMON MISTAKES

- Calling `WriteHeader` before setting `Content-Type` ; the header is lost
- Returning raw internal errors (SQL text) to clients; log them, send a generic message
- Decoding without a size limit, letting clients exhaust memory

✓ TIPS & BEST PRACTICES

- Define `type apiError struct{ Error string json:"error" }` for one error shape
- Validate with a library like `go-playground/validator` once rules grow
- Version your API in the path (`/v1/…`) from the start

QUICK SUMMARY

- decode — `MaxBytesReader` + `DisallowUnknownFields`
- validate — 400 / 422
- respond — header → status → body
- consistent JSON error shape

HTTP Client

HTTP CLIENT

`http.Get` and friends use `http.DefaultClient`, which has **no timeout**, so production code should create its own `http.Client{Timeout: …}` and reuse it (it pools connections). Build requests with `http.NewRequestWithContext` to set headers and support cancellation, **always close** `resp.Body`, and check `resp.StatusCode` yourself: non-2xx is not an error.

NOTES

- One `http.Client` per app; it's safe for concurrent use
- Not reading/closing the body prevents connection reuse
- `err` is only for transport failures (DNS, timeout, refused)
- Use `url.Values` / `url.Parse` to build query strings safely

MAKING REQUESTS

```
var client = &http.Client{Timeout: 10 * time.Second}

func getUser(ctx context.Context, id int) (*User, error) {
    u, _ := url.Parse("https://api.example.com/users")
    u.RawQuery = url.Values{"id": {strconv.Itoa(id)}}.Encode()
    req, err := http.NewRequestWithContext(ctx, "GET", u.String(), nil)
    if err != nil { return nil, err }
    req.Header.Set("Authorization", "Bearer "+token)

    resp, err := client.Do(req)
    if err != nil { return nil, err }           // network error
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        b, _ := io.ReadAll(io.LimitReader(resp.Body, 1024))
        return nil, fmt.Errorf("status %d: %s", resp.StatusCode, b)
    }
    var user User
    return &user, json.NewDecoder(resp.Body).Decode(&user)
}
body, _ := json.Marshal(newUser)                // POST JSON
resp, err := client.Post(apiURL, "application/json", bytes.NewReader(body))
```

CODE EXPLAINED

- → `Timeout` — covers connect, headers and body read
- → `url.Values{…}.Encode()` — escaped query string
- → `NewRequestWithContext` — cancellable request
- → `defer resp.Body.Close()` — after the error check
- → check `StatusCode`; 404/500 return `err == nil`

COMING FROM…

- **Python** — ≈ `requests`, but no default timeout
- **JS** — ≈ `fetch`; non-2xx isn't an error either
- **Java** — ≈ `HttpClient`

CLIENT OPTIONS

FIELD	PURPOSE
`Timeout`	whole request limit
`Transport`	pooling, TLS, proxy
`CheckRedirect`	redirect policy
`Jar`	cookies

TRANSPORT TUNING

FIELD	DEFAULT
`MaxIdleConns`	100
`MaxIdleConnsPerHost`	2 (raise for one API)
`IdleConnTimeout`	90s
`TLSHandshakeTimeout`	10s
`ForceAttemptHTTP2`	true (default transport)

⚠ COMMON MISTAKES

- Using `http.Get` in production: no timeout, a slow server hangs your goroutine forever
- Creating a new `http.Client` (and Transport) per request; connections aren't reused
- Treating `err == nil` as success without checking `resp.StatusCode`

✓ TIPS & BEST PRACTICES

- Raise `MaxIdleConnsPerHost` when calling one host heavily
- Add retries with backoff for idempotent requests (or use `hashicorp/go-retryablehttp`)
- Test clients against `httptest.NewServer`

QUICK SUMMARY

- `http.Client{Timeout}` — create once, reuse
- `NewRequestWithContext` + headers
- `defer resp.Body.Close()` — always
- check `StatusCode` — non-2xx isn't an error

Production HTTP Servers

PRODUCTION HTTP SERVERS

For production, configure an `http.Server` struct instead of calling `http.ListenAndServe`: set **timeouts** so slow clients can't exhaust resources, and implement **graceful shutdown** so in-flight requests finish when the process gets `SIGTERM`. Go serves HTTP/2 automatically over TLS, and Go 1.25's `http.CrossOriginProtection` blocks CSRF-style cross-origin requests.

NOTES

- `ListenAndServe` returns `http.ErrServerClosed` after `Shutdown`; that's normal
- `Shutdown` stops accepting, then waits for active requests
- Set `ReadHeaderTimeout` at minimum (Slowloris protection)
- Behind a load balancer, add `/healthz` and `/readyz` endpoints

GRACEFUL SERVER

```
srv := &http.Server{
    Addr:              ":8080",
    Handler:           http.NewCrossOriginProtection().Handler(mux),
    ReadHeaderTimeout: 5 * time.Second,
    ReadTimeout:       15 * time.Second,
    WriteTimeout:      30 * time.Second,
    IdleTimeout:       120 * time.Second,
}
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

go func() {
    err := srv.ListenAndServe()            // or ListenAndServeTLS
    if !errors.Is(err, http.ErrServerClosed) { log.Fatal(err) }
}()
<-ctx.Done()                               // wait for signal
sctx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
defer cancel()
if err := srv.Shutdown(sctx); err != nil { log.Print(err) } // drain
```

CODE EXPLAINED

- → timeouts — bound every phase of a connection
- → `CrossOriginProtection` — rejects unsafe cross-origin requests (1.25)
- → `signal.NotifyContext` — ctx canceled on Ctrl-C / SIGTERM
- → `ErrServerClosed` — expected after `Shutdown`
- → `Shutdown(ctx)` — drain in-flight requests, with a deadline

COMING FROM…

- **Node** — ≈ `server.close()` on SIGTERM
- **Java** — ≈ Spring graceful shutdown
- **Python** — ≈ gunicorn timeouts + graceful stop

SERVER TIMEOUTS

FIELD	LIMITS
`ReadHeaderTimeout`	reading request headers
`ReadTimeout`	headers + body
`WriteTimeout`	until response written
`IdleTimeout`	keep-alive idle time
`MaxHeaderBytes`	header size (1 MB default)

OPS CHECKLIST

ITEM	HOW
`health checks`	`GET /healthz` → 200
`metrics`	Prometheus `promhttp`
`profiling`	pprof on internal port
`TLS`	`ListenAndServeTLS` / autocert
`limits`	`MaxBytesReader`, rate limits

⚠ COMMON MISTAKES

- No timeouts: slow or malicious clients hold connections open indefinitely
- Calling `os.Exit` on SIGTERM, killing in-flight requests mid-write
- Treating `http.ErrServerClosed` as a fatal error during shutdown

✓ TIPS & BEST PRACTICES

- Set the shutdown timeout below your orchestrator's grace period (e.g. 20s < 30s)
- Use `http.TimeoutHandler` for per-route limits shorter than `WriteTimeout`
- Flip readiness to failing first, then call `Shutdown`, so load balancers drain traffic

QUICK SUMMARY

- `http.Server{…Timeout}` — always set timeouts
- `signal.NotifyContext` + `Shutdown` — graceful stop
- `ErrServerClosed` — normal on shutdown
- `CrossOriginProtection` (1.25), TLS, health checks

TCP, UDP & net

TCP, UDP & the net package

Below HTTP, the `net` package provides **TCP** and **UDP** sockets. A TCP server calls `net.Listen`, then `Accept`s connections in a loop and handles each in its **own goroutine**. Clients use `net.Dial` / `DialTimeout`. A `net.Conn` is an `io.Reader` + `io.Writer`, so `bufio`, `io.Copy` and encoders all work on it.

Notes

- Set deadlines (`SetDeadline`) or a stalled peer blocks forever
- `net/netip` (1.18) has small, comparable `Addr` / `AddrPort` types
- `net.Dialer{Timeout, KeepAlive}` for fine-grained control
- Network names: `"tcp"` , `"tcp4"` , `"udp"` , `"unix"`

TCP echo server & client

```
ln, err := net.Listen("tcp", ":9000")
if err != nil { log.Fatal(err) }
for {
    conn, err := ln.Accept()
    if err != nil { continue }
    go func(c net.Conn) {                    // one goroutine per conn
        defer c.Close()
        c.SetDeadline(time.Now().Add(time.Minute))
        sc := bufio.NewScanner(c)
        for sc.Scan() { fmt.Fprintf(c, "echo: %s
", sc.Text()) }
    }(conn)
}

conn, err := net.DialTimeout("tcp", "localhost:9000", 3*time.Second)
fmt.Fprintln(conn, "hello")
reply, _ := bufio.NewReader(conn).ReadString('
') // echo: hello

pc, _ := net.ListenPacket("udp", ":9001")       // UDP
n, addr, _ := pc.ReadFrom(buf); pc.WriteTo(buf[:n], addr)
ap := netip.MustParseAddrPort("10.0.0.1:443")  // ap.Addr(), ap.Port()
```

Code explained

- → `Accept` loop — handle each connection concurrently
- → `SetDeadline` — absolute time limit for reads + writes
- → `bufio.Scanner` over `conn` — line-based protocol
- → `ListenPacket` / `ReadFrom` / `WriteTo` — UDP datagrams
- → `netip` — value-type IP addresses

Coming from…

- **Python** — ≈ `socket` module, plus goroutines
- **Node** — ≈ `net.createServer`
- **C** — ≈ BSD sockets, without `select` / `epoll`

Functions

FUNC	PURPOSE
`net.Listen`	TCP/Unix server
`net.Dial / DialTimeout`	client connect
`net.ListenPacket`	UDP socket
`net.LookupHost / LookupIP`	DNS
`net.SplitHostPort`	"h:p" → h, p

Conn methods

METHOD	DOES
`Read / Write`	io.Reader/Writer
`SetDeadline(t)`	read + write limit
`SetReadDeadline`	read only
`RemoteAddr()`	peer address
`Close()`	close socket

⚠ Common mistakes

- Handling connections inline in the Accept loop; one slow client blocks all others
- No deadlines on connections; idle or malicious peers leak goroutines
- Assuming one `Read` returns one full message; TCP is a byte stream, frame your protocol

✓ Tips & best practices

- Use `net.JoinHostPort` to build addresses (handles IPv6 brackets)
- Prefix messages with a length (`binary.BigEndian`) to frame them
- Prefer HTTP/gRPC unless you need a custom protocol

Quick summary

- `net.Listen` + `Accept` + `go handle`
- `net.Dial` / `DialTimeout` — clients
- `SetDeadline` — avoid hangs
- `ListenPacket` — UDP; `netip` — addresses

Sheet 111

Quick Reference — Syntax & Types

declarations · types & zero values · format verbs · operators & precedence · control flow · slice / map / string idioms · methods & interfaces · generics

DECLARATIONS

SYNTAX	DOES	#
`var x int`	zero-valued var	008
`x := 1`	short decl (funcs)	008
`const N = 10`	constant	009
`iota`	0,1,2… per const block	009
`type T U`	defined type	014
`type A = B`	alias	014
`_ = f()`	discard value	008

TYPES & ZERO VALUES

SYNTAX	DOES	#
`int / int64 / uint8`	integers, wrap on overflow	010
`float64`	default float	011
`bool`	false, no truthiness	012
`string`	"", immutable bytes	016
`byte / rune`	uint8 / int32	015
`*T, []T, map, chan`	nil	012
`T(v)`	explicit conversion	013

FORMAT VERBS

SYNTAX	DOES	#
`%v %+v %#v`	default / fields / Go	006
`%T`	type	006
`%d %x %b`	dec / hex / binary	006
`%.2f %e %g`	floats	006
`%s %q`	string / quoted	006
`%-8s %08d`	width, pad	006
`%w`	wrap error (Errorf)	034

OPERATORS

SYNTAX	DOES	#
`+ - * / %`	arithmetic; int / truncates	022
`== != < >`	comparison	023
`&& || !`	logic, short-circuit	023
`& | ^ &^ << >>`	bitwise; &^ = bit clear	024
`+= &^= <<=`	compound assign	025
`&x *p`	address / deref	026
`<-ch`	receive	092

CONTROL FLOW

SYNTAX	DOES	#
`if v, ok := m[k]; ok {}`	init statement	027
`switch { case x > 0: }`	tagless switch	028
`fallthrough`	into next case	028
`for i := 0; i < n; i++`	classic loop	029
`for cond {} / for {}`	while / forever	029
`for i, v := range s`	range loop	030
`break L / continue L`	labeled	031
`defer f()`	run at return, LIFO	032

STRINGS

SYNTAX	DOES	#
`len(s)`	bytes, not chars	016
`for i, r := range s`	runes	017
`strings.Cut(s, sep)`	split once	018
`strings.Fields(s)`	split on space	018
`strings.Join(xs, ",")`	join	019
`strings.Builder`	efficient build	020
`strconv.Atoi / Itoa`	string ↔ int	021

SLICES & MAPS

SYNTAX	DOES	#
`make([]T, len, cap)`	allocate slice	046
`s = append(s, x)`	always reassign	046
`s[a:b:c]`	full slice expr	047
`slices.Sort / Index`	slices pkg	048
`v, ok := m[k]`	comma-ok lookup	049
`delete(m, k) / clear(m)`	remove	049
`map[T]struct{}`	set	050
`slices.Sorted(maps.Keys(m))`	ordered keys	050

METHODS & INTERFACES

SYNTAX	DOES	#
`func (t *T) M()`	pointer receiver	055
`struct{ Embedded }`	promotion	056
`type I interface{ M() }`	implicit satisfaction	058
`v, ok := x.(T)`	type assertion	059
`switch v := x.(type)`	type switch	059
`var _ I = (*T)(nil)`	compile-time check	058
`String() string`	fmt.Stringer	061

GENERICS

SYNTAX	DOES	#
`func F[T any](x T)`	generic func	062
`type S[T any] struct`	generic type	062
`comparable`	== allowed	062
`cmp.Ordered`	< allowed	063
`~int | ~float64`	type set	063
`func (x *T) M[U any]()`	generic method (1.27)	064
`iter.Seq[V]`	iterator	054

⚠ COMMON MISTAKES

- `:=` in an inner block shadows outer vars (esp. `err`)
- `append` without reassigning; writes to nil maps
- Typed nil pointer returned as `error` is non-nil

✓ TIPS & BEST PRACTICES

- Run `gofmt`, `go vet` and `-race` on every change
- Prefer `slices` / `maps` / `cmp` helpers to hand loops
- Check the version tags: `range n` 1.22, iterators 1.23

QUICK SUMMARY

- `x := v` / `var x T` — declare
- `for` — the only loop; `range` everywhere
- interfaces — implicit; generics — `[T C]`
- errors — values, checked with `if err != nil`

Sheet 112

Quick Reference — Concurrency, Stdlib & Tooling

goroutines / channels / select · sync & context · error idioms · common stdlib calls · go command · test flags · build & cross-compile flags · version matrix

GOROUTINES & CHANNELS

SYNTAX	DOES	#
`go f()`	start goroutine	091
`wg.Go(f); wg.Wait()`	wait (1.25)	091
`make(chan T, n)`	buffered channel	092
`close(ch); for v := range ch`	end a stream	092
`select { case …: default: }`	multiplex	093
`case <-ctx.Done():`	cancellation	093
`chan<- T / <-chan T`	directions	092

SYNC & CONTEXT

SYNTAX	DOES	#
`mu.Lock(); defer mu.Unlock()`	mutex	094
`sync.OnceValue(f)`	lazy init	094
`atomic.Int64`	lock-free counter	095
`context.WithTimeout`	deadline + cancel	096
`defer cancel()`	always	096
`errgroup.SetLimit(n)`	bounded + errors	097
`go test -race`	find races	098

ERROR IDIOMS

SYNTAX	DOES	#
`if err != nil { return err }`	propagate	033
`fmt.Errorf("op: %w", err)`	wrap	034
`errors.Is(err, ErrX)`	match value	034
`errors.AsType[*E](err)`	match type (1.26)	034
`errors.Join(a, b)`	combine	034
`var ErrX = errors.New(…)`	sentinel	035
`defer func(){ recover() }()`	stop panic	036

COMMON STDLIB

SYNTAX	DOES	#
`os.ReadFile / WriteFile`	whole file	066
`bufio.NewScanner(f)`	lines	066
`json.Marshal / NewDecoder`	JSON	070
`t.Format(time.RFC3339)`	time format	076
`slog.Info(msg, k, v)`	structured log	079
`mux.HandleFunc("GET /x/{id}", h)`	routing (1.22)	105
`http.Client{Timeout: d}`	HTTP client	108

GO COMMAND

SYNTAX	DOES	#
`go run . / go build -o bin`	run / build	003
`go mod init / tidy`	modules	073
`go get pkg@v1.2.3`	add dependency	074
`go install pkg@latest`	install tool	003
`go vet ./... / go fix ./...`	check / modernize	103
`go doc pkg.Symbol`	docs	003
`go work init ./a ./b`	workspace	074

TESTING FLAGS

SYNTAX	DOES	#
`go test ./...`	all tests	099
`-run 'TestX/case'`	select	099
`-v -count=1`	verbose, no cache	099
`-bench=. -benchmem`	benchmarks	101
`-fuzz=FuzzX`	fuzzing	101
`-cover -coverprofile`	coverage	101
`-cpuprofile cpu.out`	profiling	102

BUILD & CROSS-COMPILE

SYNTAX	DOES	#
`GOOS=linux GOARCH=arm64`	target platform	088
`CGO_ENABLED=0`	static binary	088
`-ldflags "-s -w"`	smaller binary	088
`-ldflags "-X main.v=1.2"`	inject version	088
`-trimpath`	reproducible paths	088
`//go:build linux`	build constraint	088
`//go:embed dir`	embed files	069

VERSION MATRIX

VER	KEY ADDITIONS
1.18	generics, fuzzing, workspaces
1.21	min/max/clear, slices/maps/cmp, slog, toolchain
1.22	per-iteration loop vars, range int, ServeMux patterns, rand/v2
1.23	range-over-func iterators, iter
1.24	generic aliases, b.Loop, t.Context, os.Root, omitzero, tool
1.25	WaitGroup.Go, synctest, FlightRecorder, CrossOriginProtection
1.26	new(expr), errors.AsType, Green Tea GC default
1.27	generic methods, json/v2, uuid, goroutineleak, CutLast

⚠ COMMON MISTAKES

- No `cancel()`, no `Close()`, no `Flush()`: leaks and lost data
- `http.Get` without a timeout in production
- Unbounded goroutines; use `errgroup.SetLimit`

✓ TIPS & BEST PRACTICES

- Pass `ctx` first; select on `ctx.Done()` in loops
- Set `go` in go.mod to the oldest version you support
- Check `go.dev/doc/go1.N` release notes when upgrading

QUICK SUMMARY

- goroutines + channels + `select` — concurrency
- `sync` / `context` / `errgroup` — coordination
- `go` command — build, test, modules, tools
- version matrix — what your toolchain allows